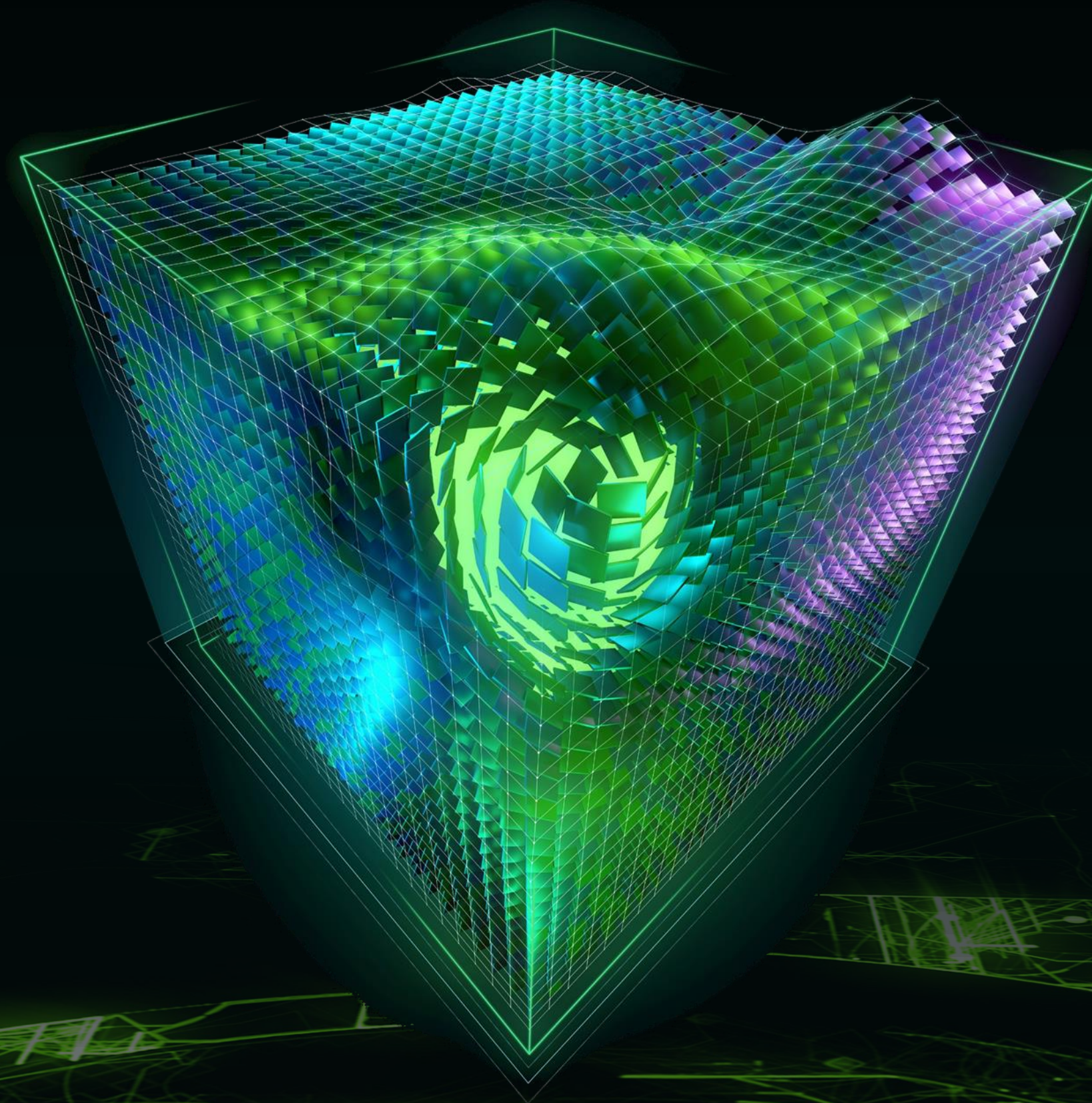
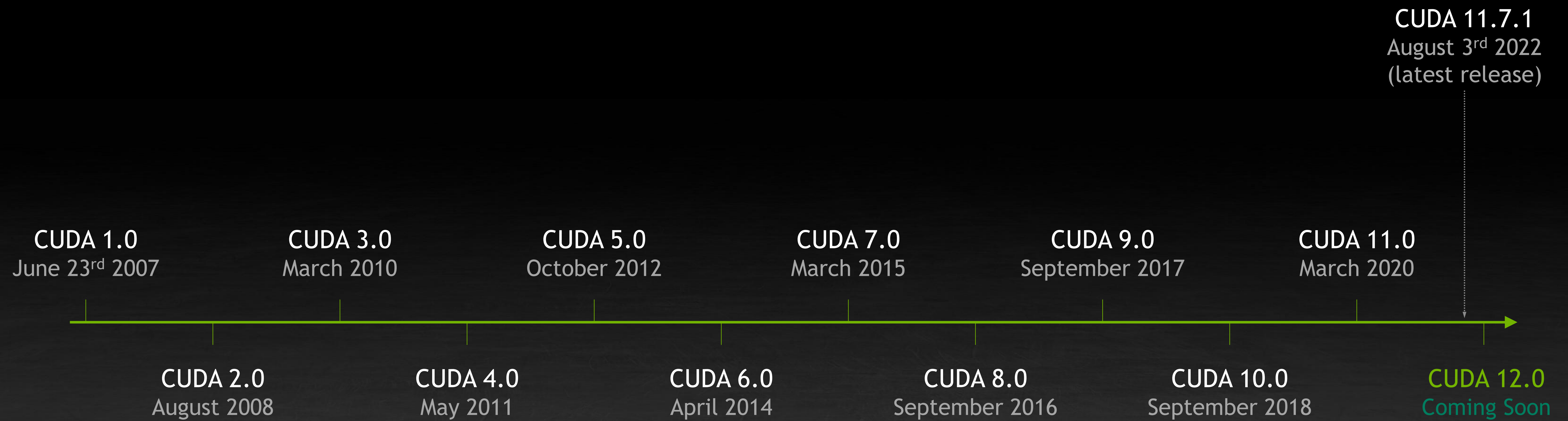


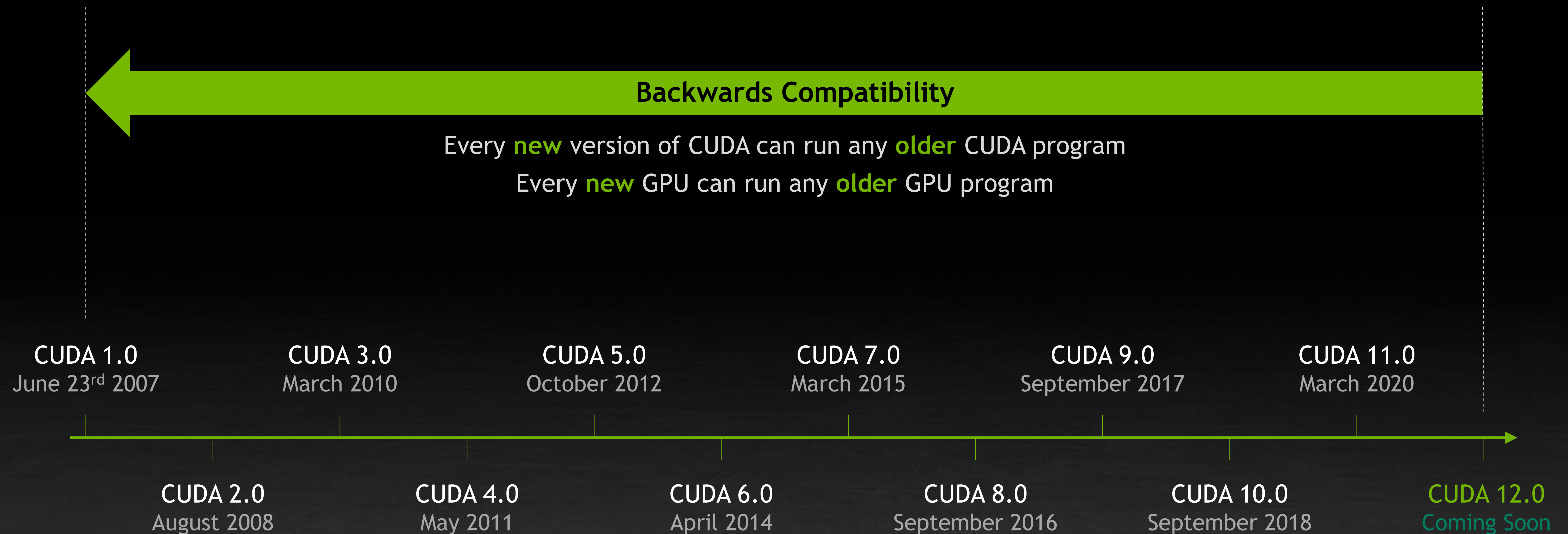
CUDA: NEW FEATURES AND BEYOND

Stephen Jones, GTC Fall 2022





CUDA MAINTAINS BACKWARDS COMPATIBILITY



ANATOMY OF A CUDA BINARY

Hello world example

```
__global__ void kernel() {  
    printf("Hello, CUDA\n");  
}
```

} GPU code section

```
void main() {  
    kernel<<< 1, 1 >>>();  
    cudaDeviceSynchronize();  
}
```

} CPU code section

nvcc example.cu -o example

"example"
executable

ANATOMY OF A CUDA BINARY

Hello world example

```
__global__ void kernel() {  
    printf("Hello, CUDA\n");  
}
```

} GPU code section

```
void main() {  
    kernel<<< 1, 1 >>>();  
    cudaDeviceSynchronize();  
}
```

} CPU code section

nvcc example.cu -o example

example.ptx

} PTX assembly code section

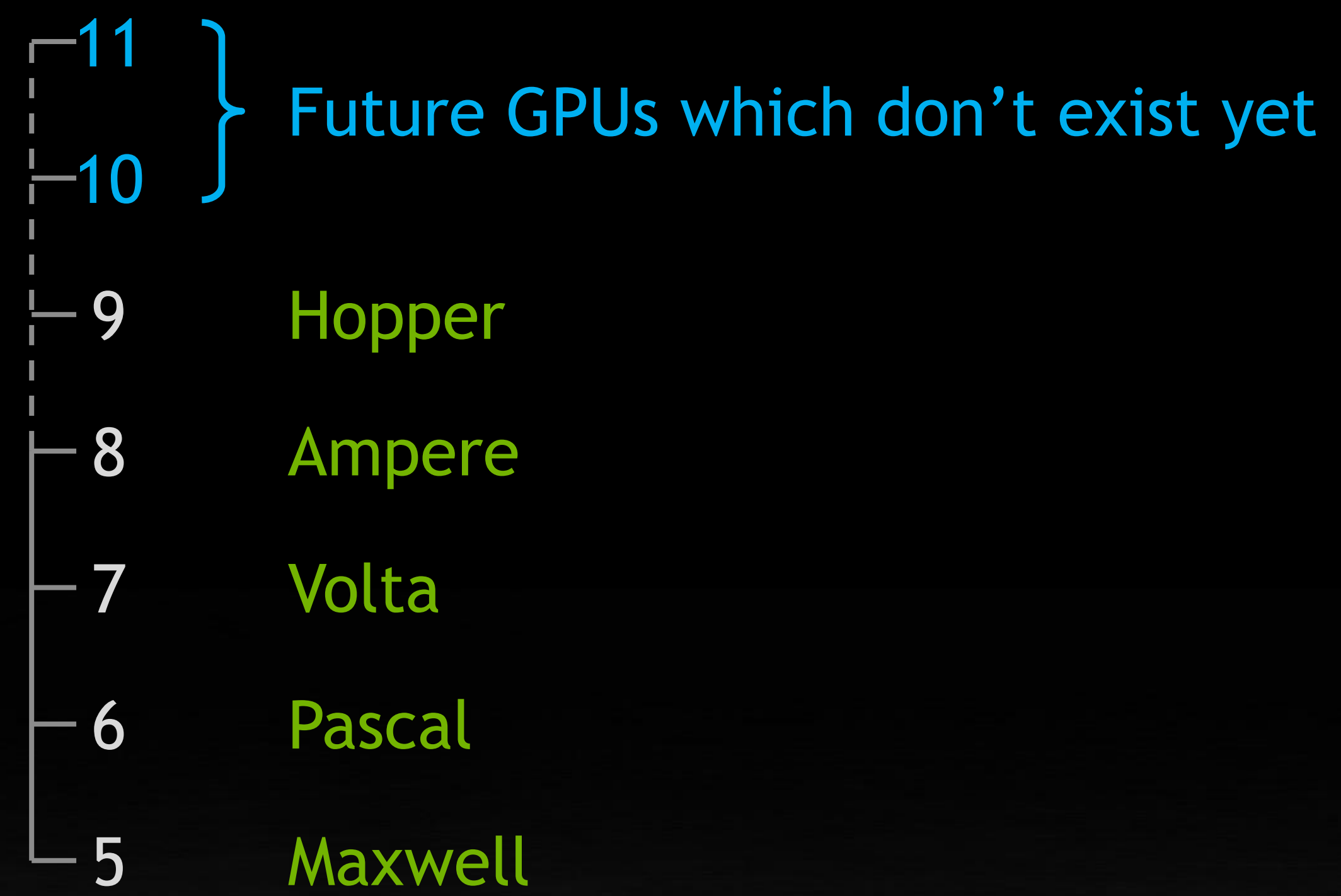
kernel()

} GPU binary code section

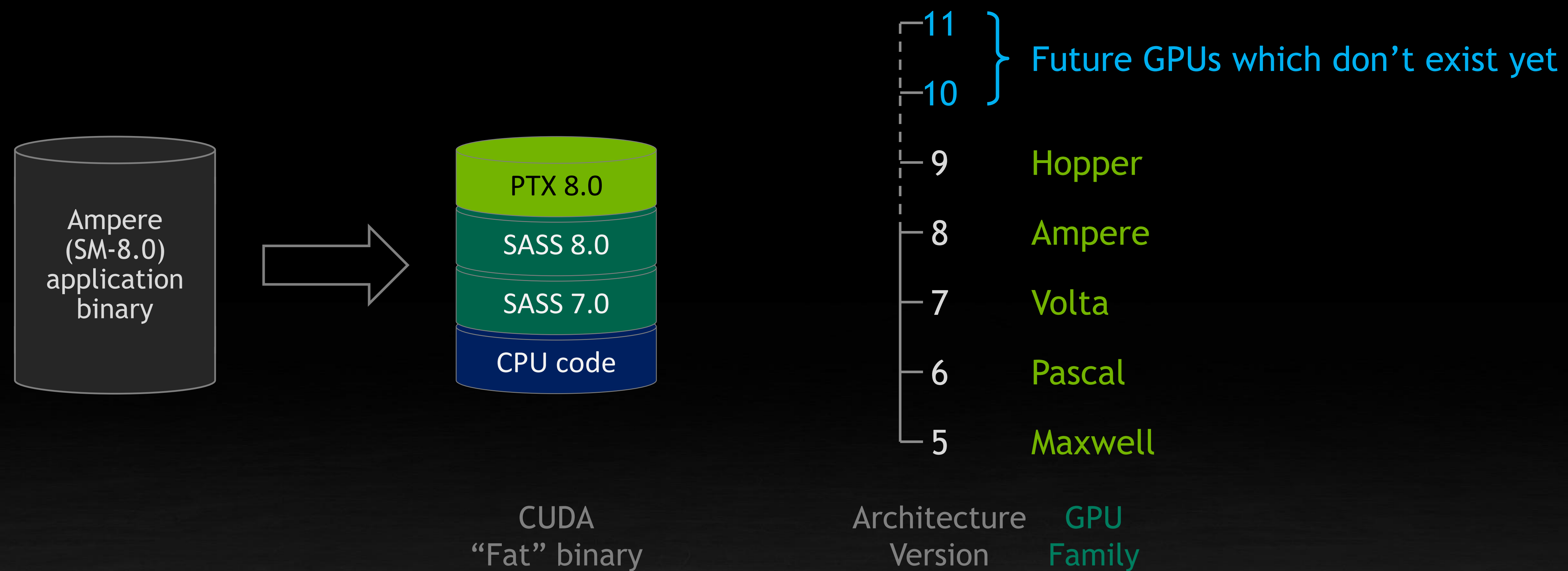
main()

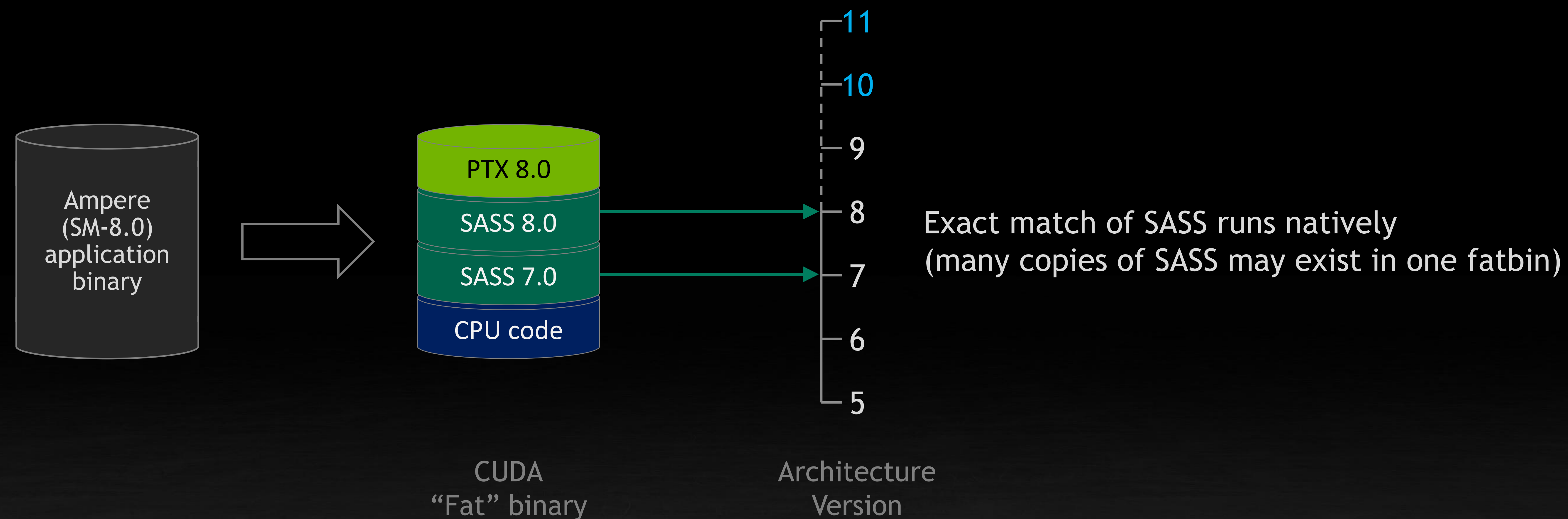
} CPU binary code section

CUDA
"Fat" binary



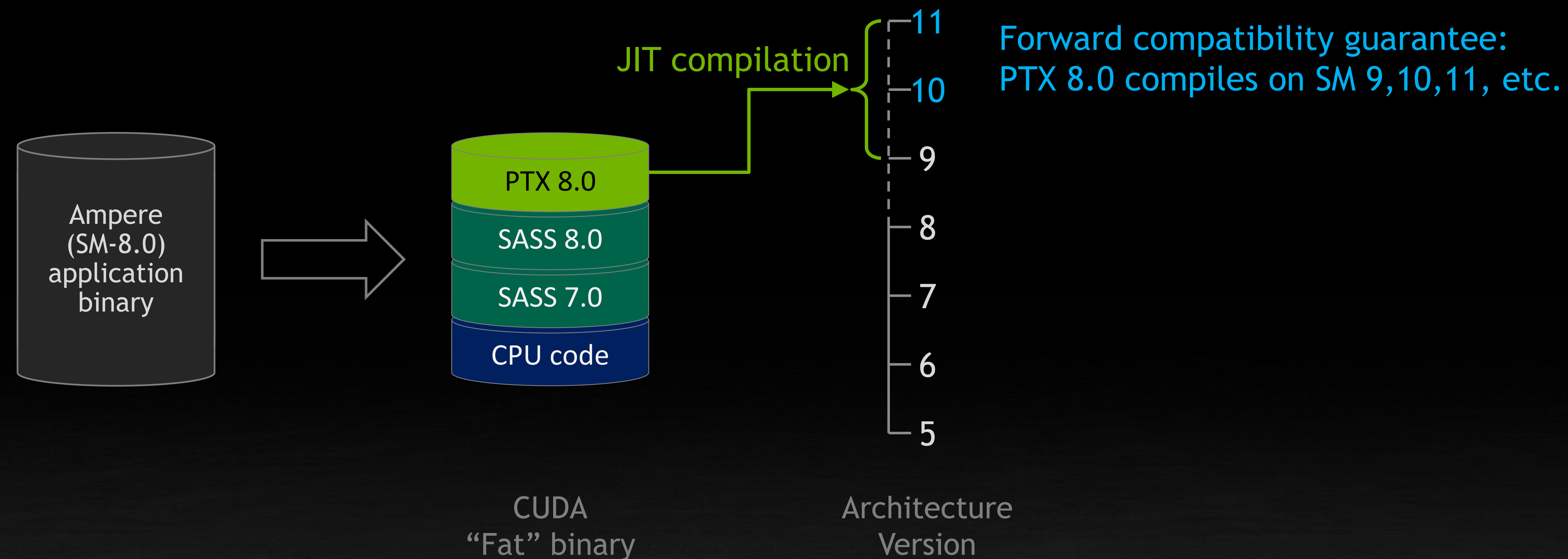
Architecture GPU
Version Family





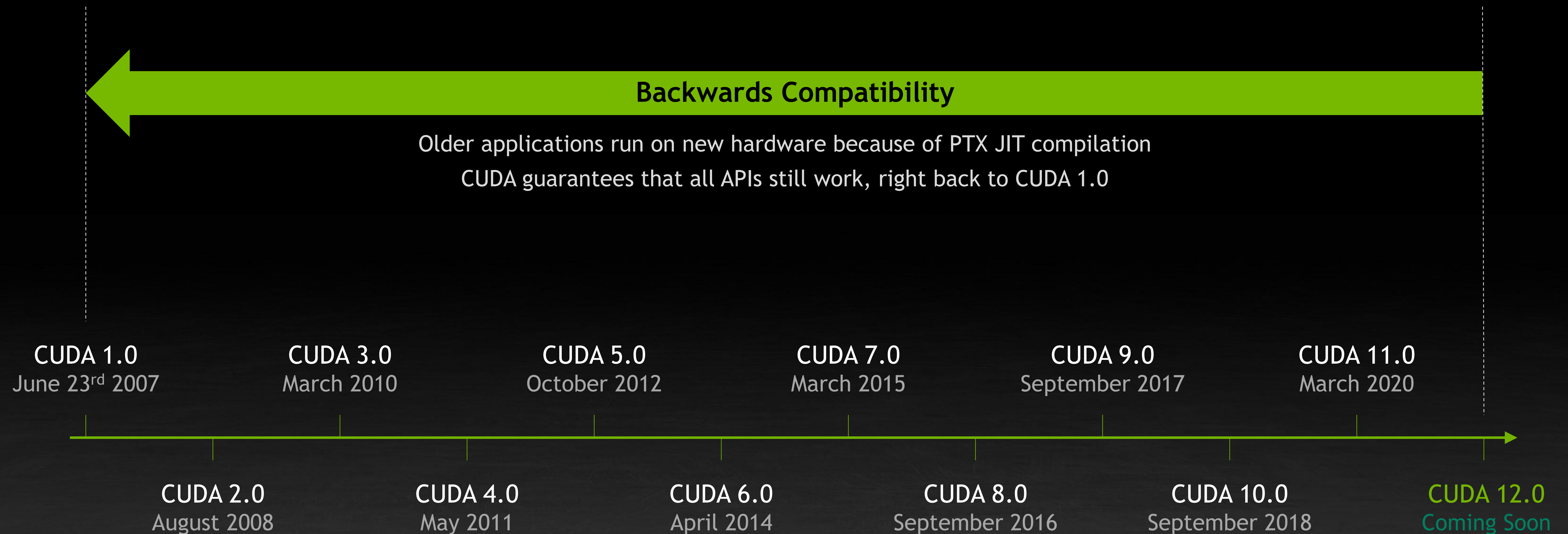
Running this binary on an Ampere (SM-8.0) GPU
or a Volta (SM-7.0) GPU will just work

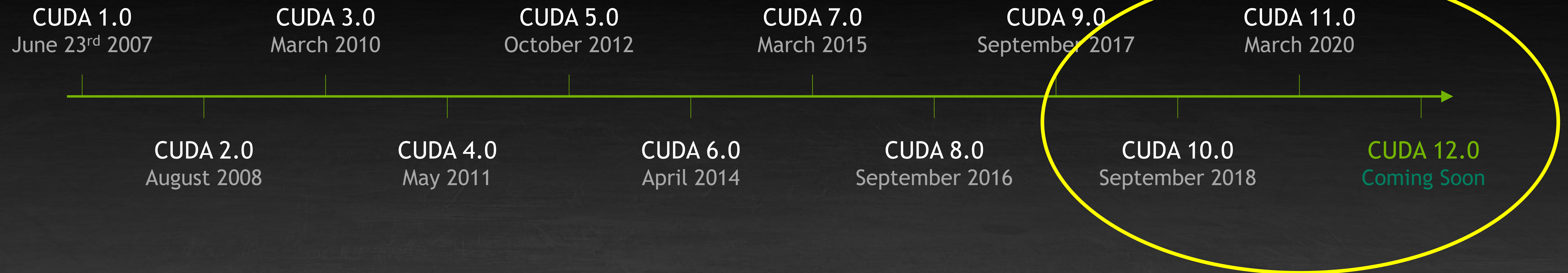
PORTABILITY VIA PTX JUST-IN-TIME COMPILATION

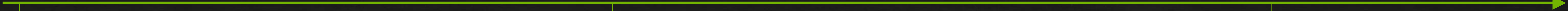


When running on a GPU for which SASS does not exist, CUDA's built-in PTX compiler recompiles for the new GPU

CUDA MAINTAINS BACKWARDS COMPATIBILITY





A horizontal timeline arrow pointing to the right, with three vertical tick marks.

CUDA 10.0
September 2018

CUDA 11.0
March 2020

CUDA 12.0
Coming Soon



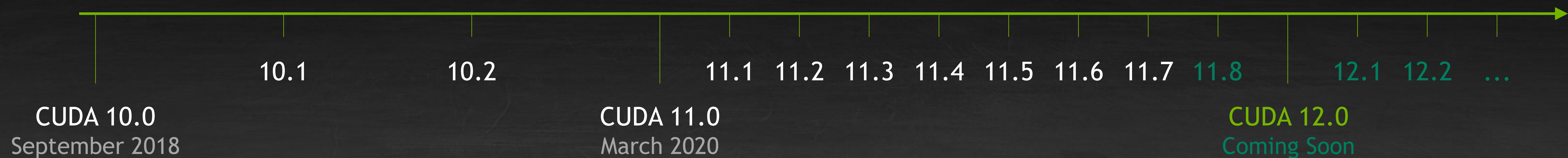
SEMANTIC VERSIONING - STARTING WITH 11.0

Version number MAJOR.MINOR.PATCH

Rules of Semantic Versioning:

1. MAJOR version changes when you make incompatible API changes
2. MINOR version changes when you add functionality in a backwards compatible manner
3. PATCH version changes when you make backwards compatible bug fixes

Semantic Versioning
introduced

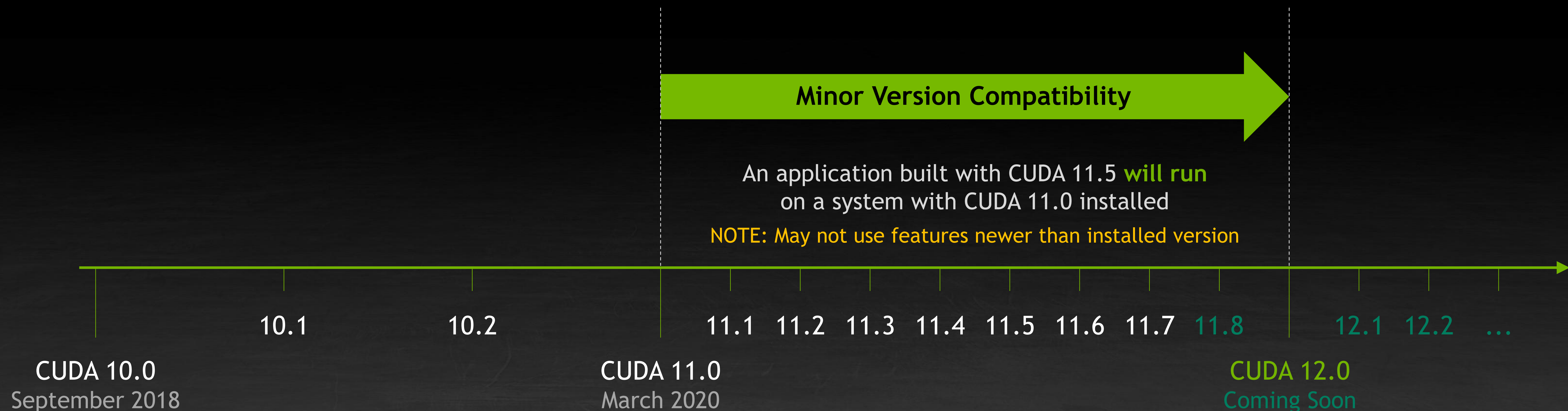


SEMANTIC VERSIONING - STARTING WITH 11.0

Version number MAJOR.MINOR.PATCH

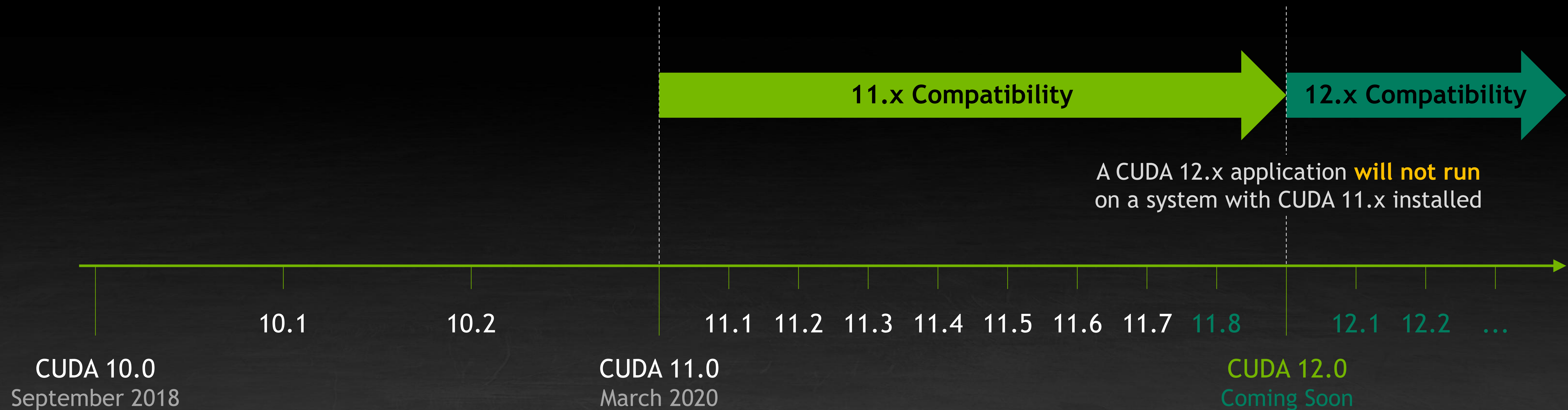
Rules of Semantic Versioning:

1. MAJOR version changes when you make incompatible API changes
2. MINOR version changes when you add functionality in a backwards compatible manner
3. PATCH version changes when you make backwards compatible bug fixes

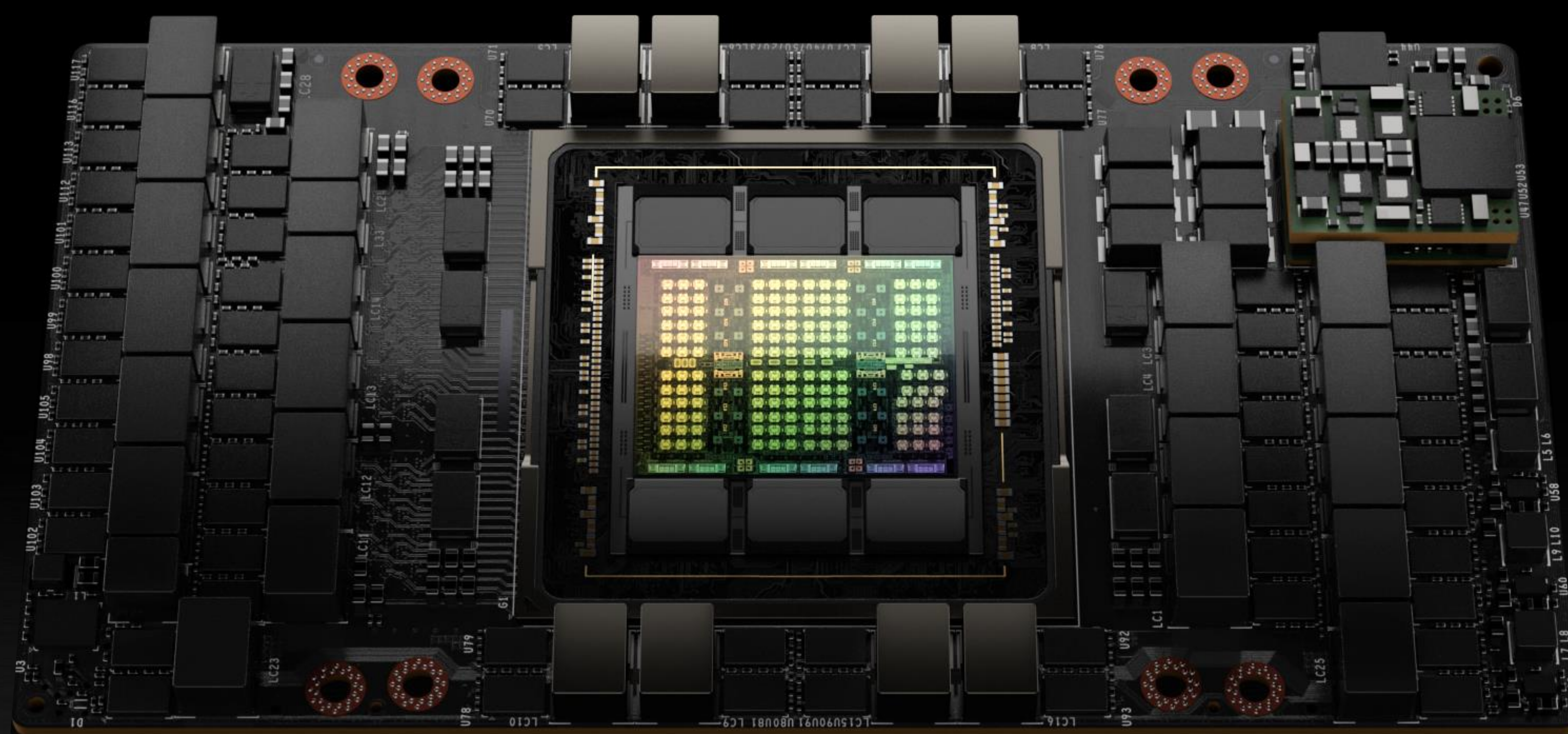


MINOR VERSION COMPATIBILITY

Resets at each new major version



INTRODUCING HOPPER



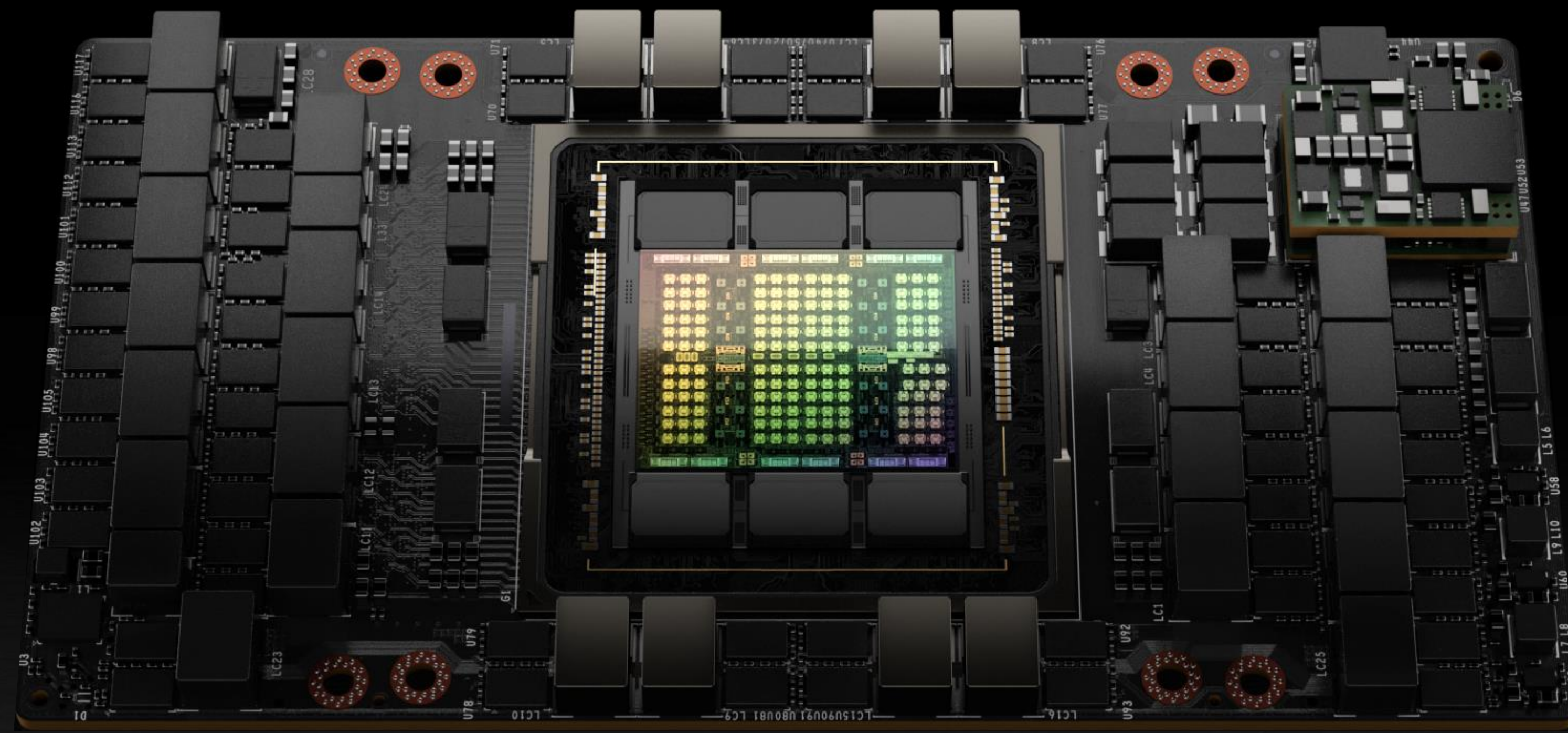
New GPU hardware

New functionality required

New major CUDA version 12.0

Minor version compatibility is reset

INTRODUCING HOPPER



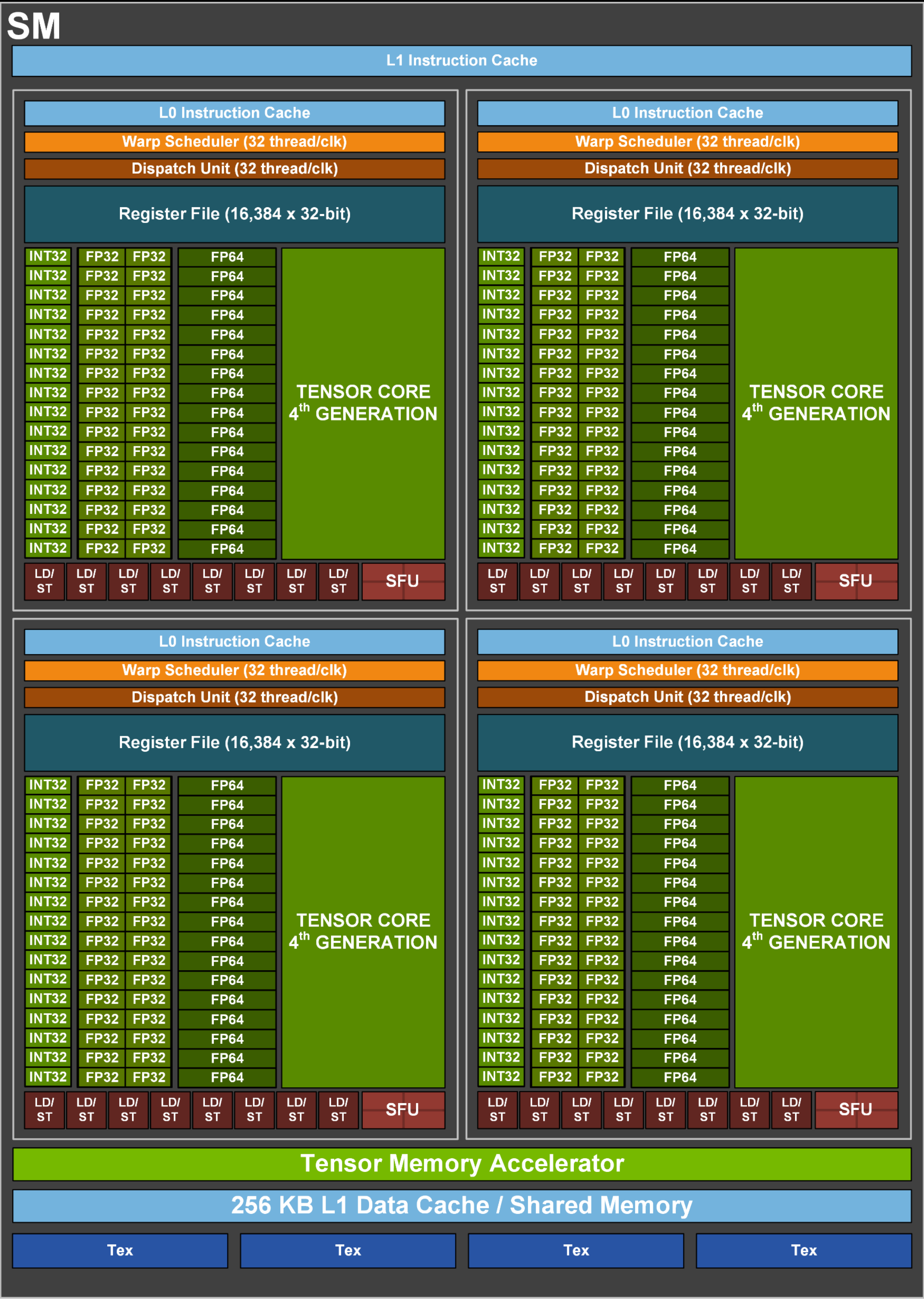
H100 Physical Architectural Features

- 132 Streaming Multiprocessors (SMs)
- PCIe Gen5 with PCIe Atomics
- HBM3 Memory with 3TB/sec Bandwidth
- 50MB L2 Cache
- 4th Generation NVLink @ 900GB/sec total bandwidth

H100 Next-Generation Capabilities

- Thread Block Clusters
- Distributed Shared Memory
- Tensor Memory Accelerator (TMA)
- Tensor Core Transformer Engine
- Confidential Computing Support
- Asynchronous Architecture

INTRODUCING HOPPER



H100 SM

256kB Shared Memory / L1-Cache

DPX Instruction Set

4th Generation Tensor Core

Fully-Asynchronous Architecture

Thread Block Clusters

Tensor Memory Accelerator

Asynchronous Transaction Barriers

SOME HISTORY: THE KEPLER GK110 GPU, 2012



Kepler GK110 Full Chip

15 SMs

THE HOPPER H100 GPU, 2022



Hopper H100 Full Chip

132 SMs

9x SMs OVER 10 YEARS



Hopper H100 Full Chip

132 SMs

Kepler GK110 Full Chip
15 SMs

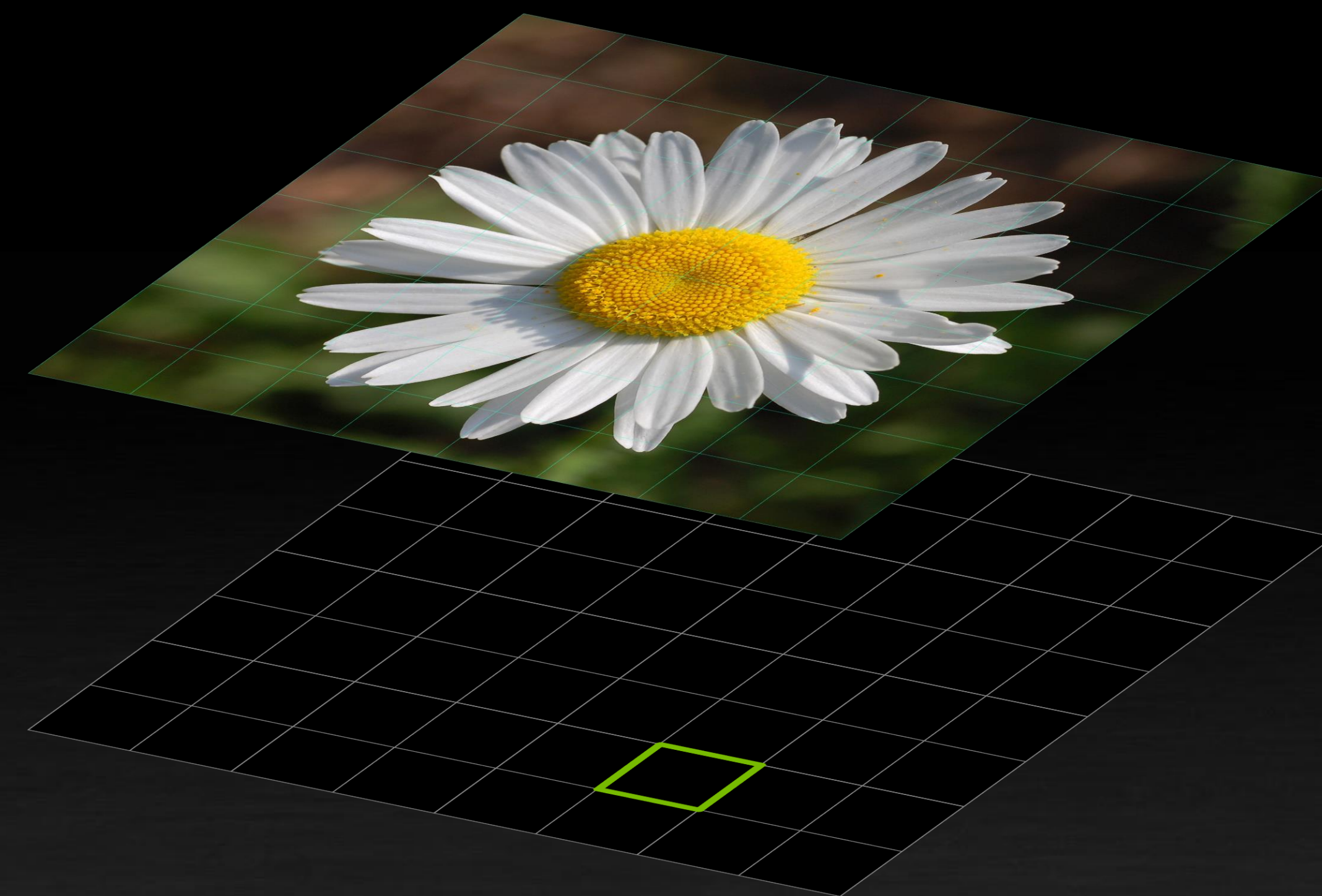
THE CUDA PROGRAMMING MODEL: GRID → BLOCKS → THREADS



Grid
of work

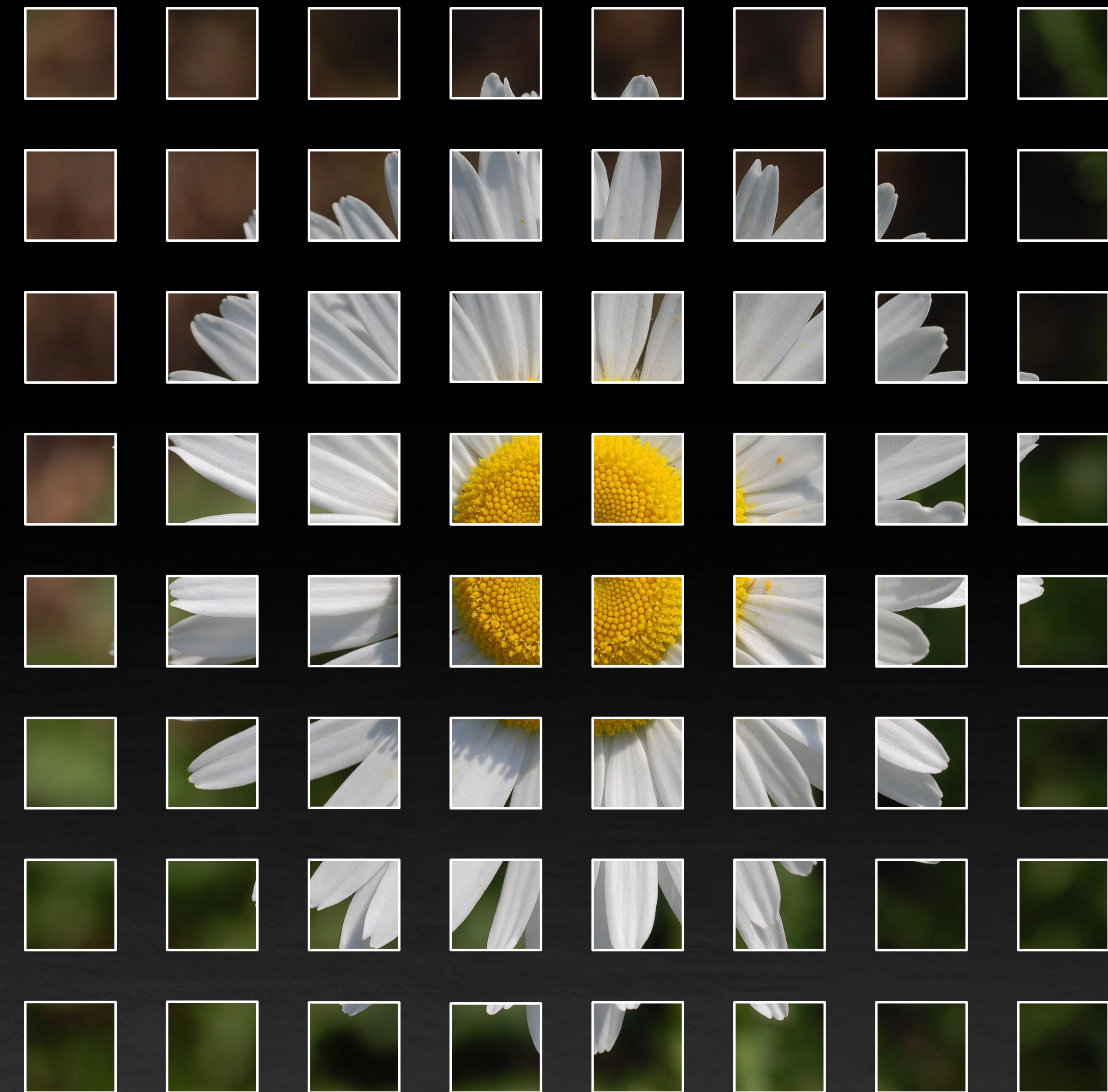


DIVIDE THE WORK INTO A GRID OF EQUAL BLOCKS

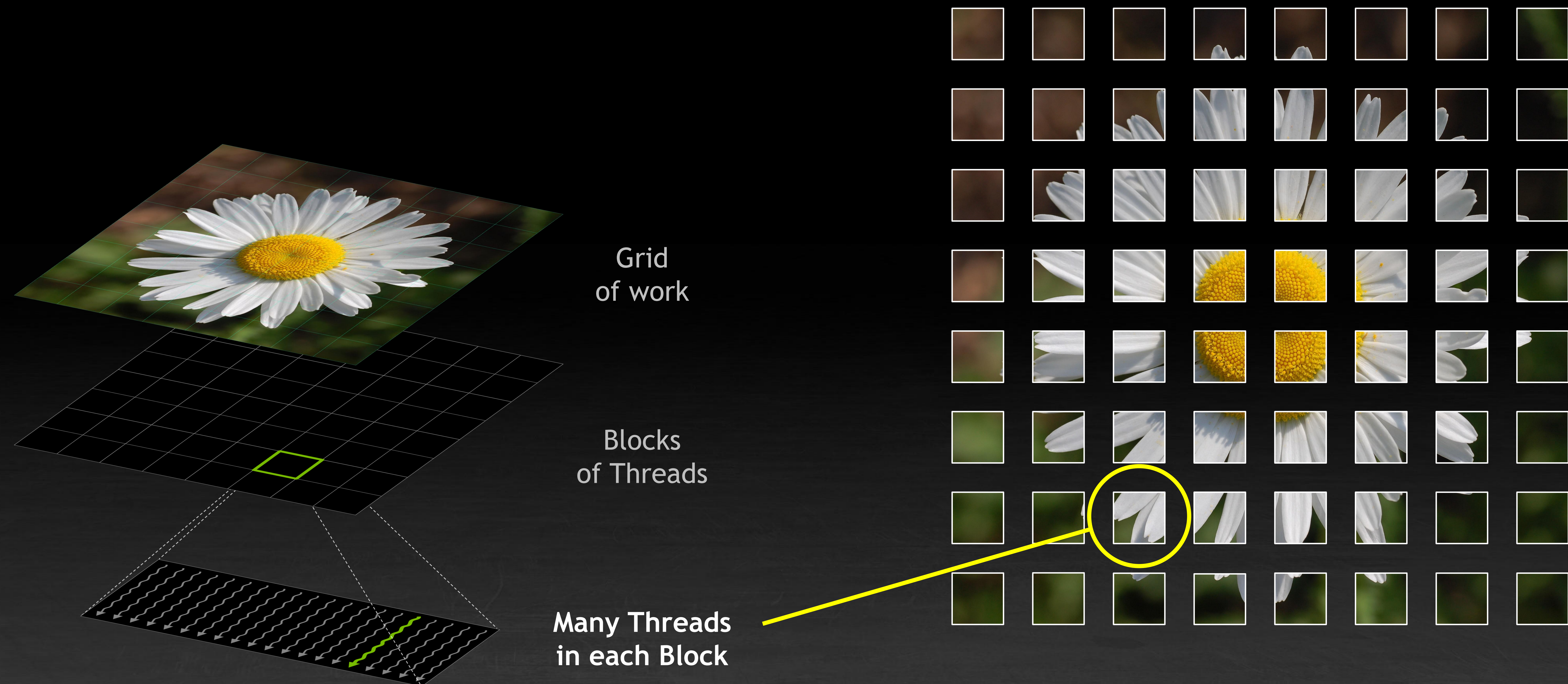


Grid
of work

Divide into
many Blocks

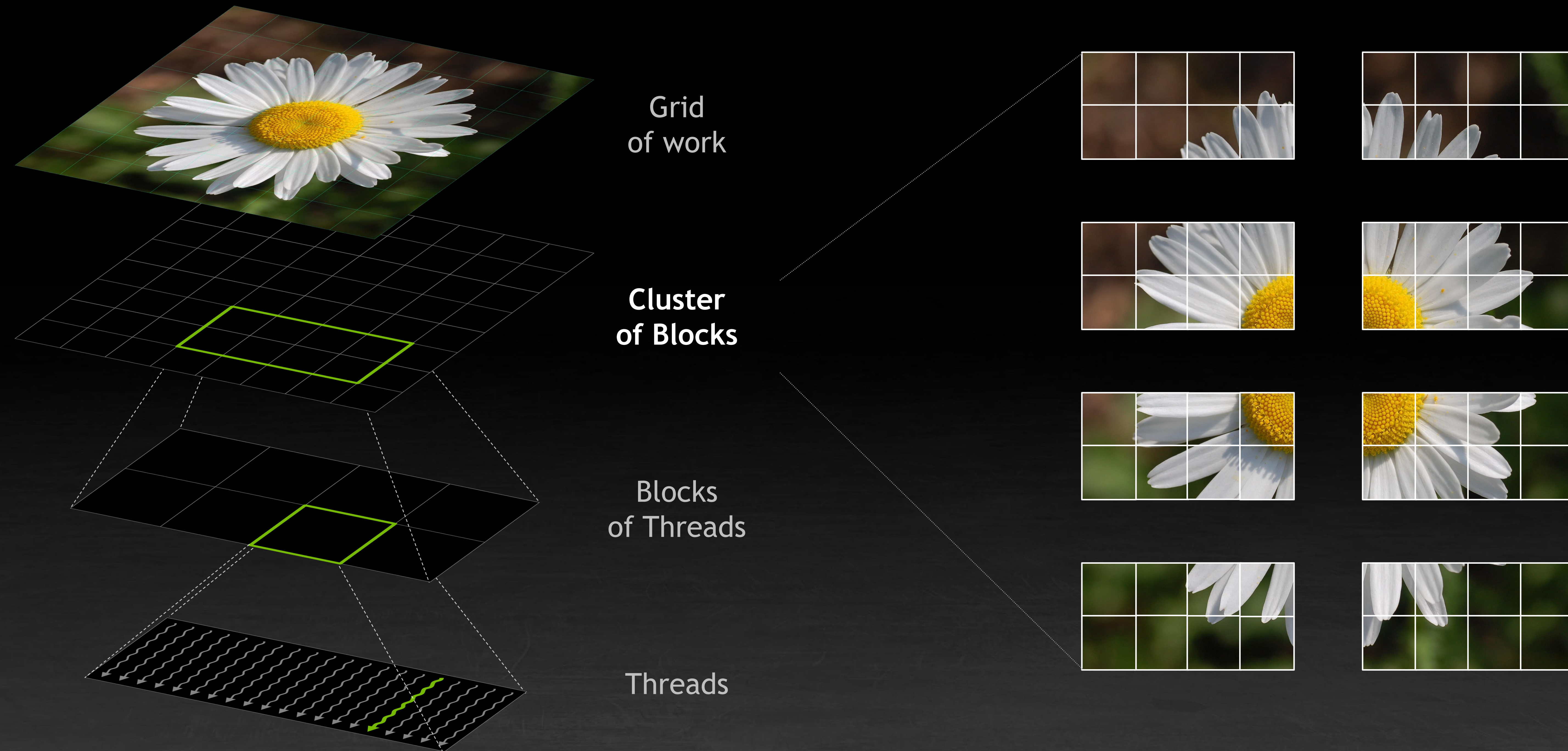


EACH BLOCK RUNS AS IF IT'S AN INDEPENDENT PROGRAM

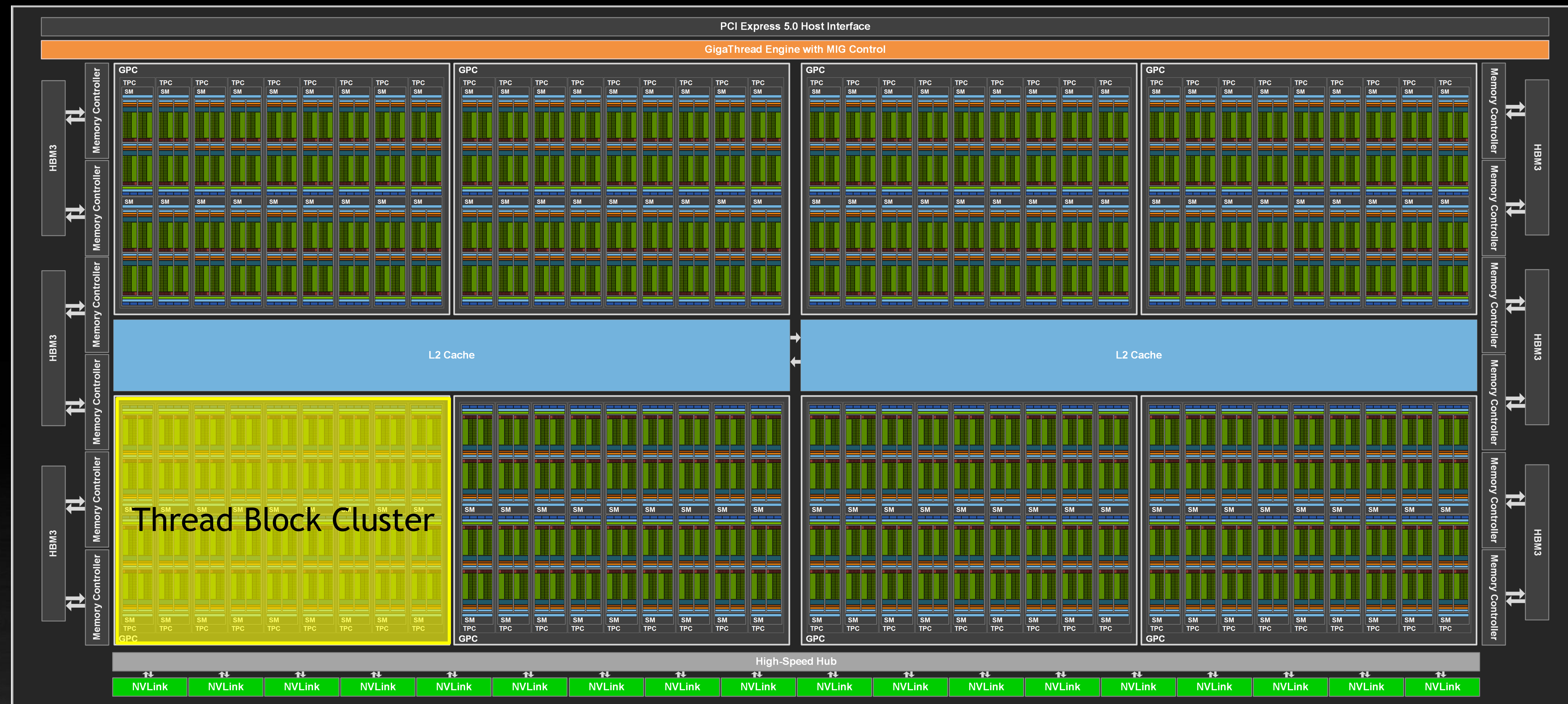


THREAD BLOCK CLUSTER

A collective of blocks, co-scheduled on adjacent multiprocessors



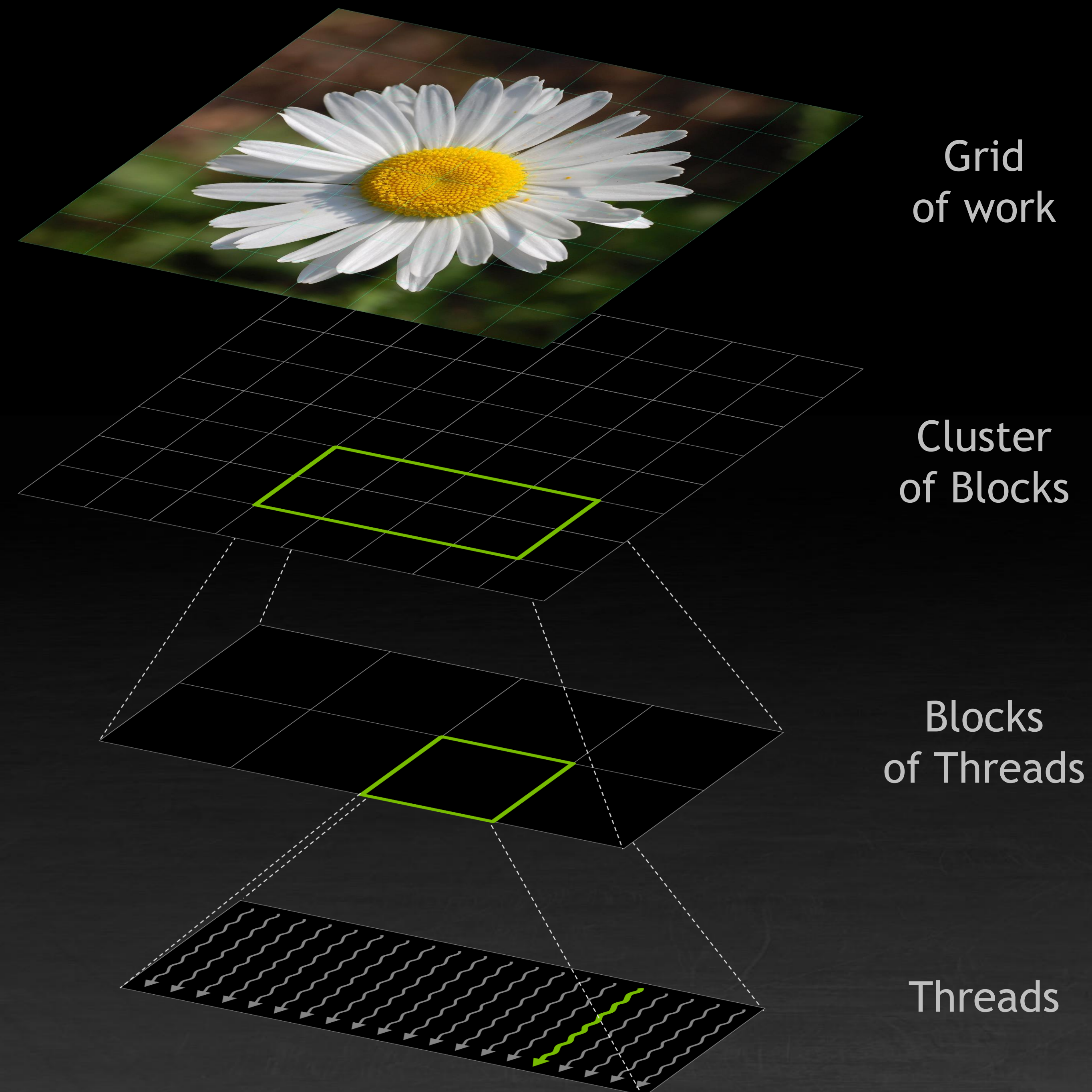
TAKING ADVANTAGE OF LOCALITY AT A GPU SCALE



Guaranteed co-located blocks
New tier of guaranteed concurrency
Fast data exchange & sync

THREAD BLOCK CLUSTER

Building hierarchy into a program



A cluster is a collective of up to 16 blocks

Guaranteed to be on different SMs

Guaranteed to be running at the same time

1D, 2D or 3D, just like blocks

Annotate a kernel with its required cluster size

New cluster dimension annotation for `__global__` functions:

`__cluster_dims__(xdim, [ydim, [zdim]])`

```
__cluster_dims__(4, 2, 1) // 8-block cluster of size 4x2x1 blocks
__global__ void helloCluster()
{
    cooperative_groups::cluster_group cluster = this_cluster();
    cluster.sync();

    printf("Hello from cluster elem %d\n", cluster.cluster_rank());
}
```

Plus: [New extensible launch API](#) allows configuration at launch time

CLUSTER DISTRIBUTED SHARED MEMORY (DSMEM)

Blocks within a cluster are able to access each others' shared memory directly



Full load/store/atomic access to all shared memory
between blocks within a cluster

CLUSTER DISTRIBUTED SHARED MEMORY (DSMEM)

Blocks within a cluster are able to access each others' shared memory directly

Distributed Shared Memory (DSMEM) Programming Model

All blocks in a cluster can collaborate using DSMEM

Blocks in a cluster can synchronize together via **barriers in DSMEM**

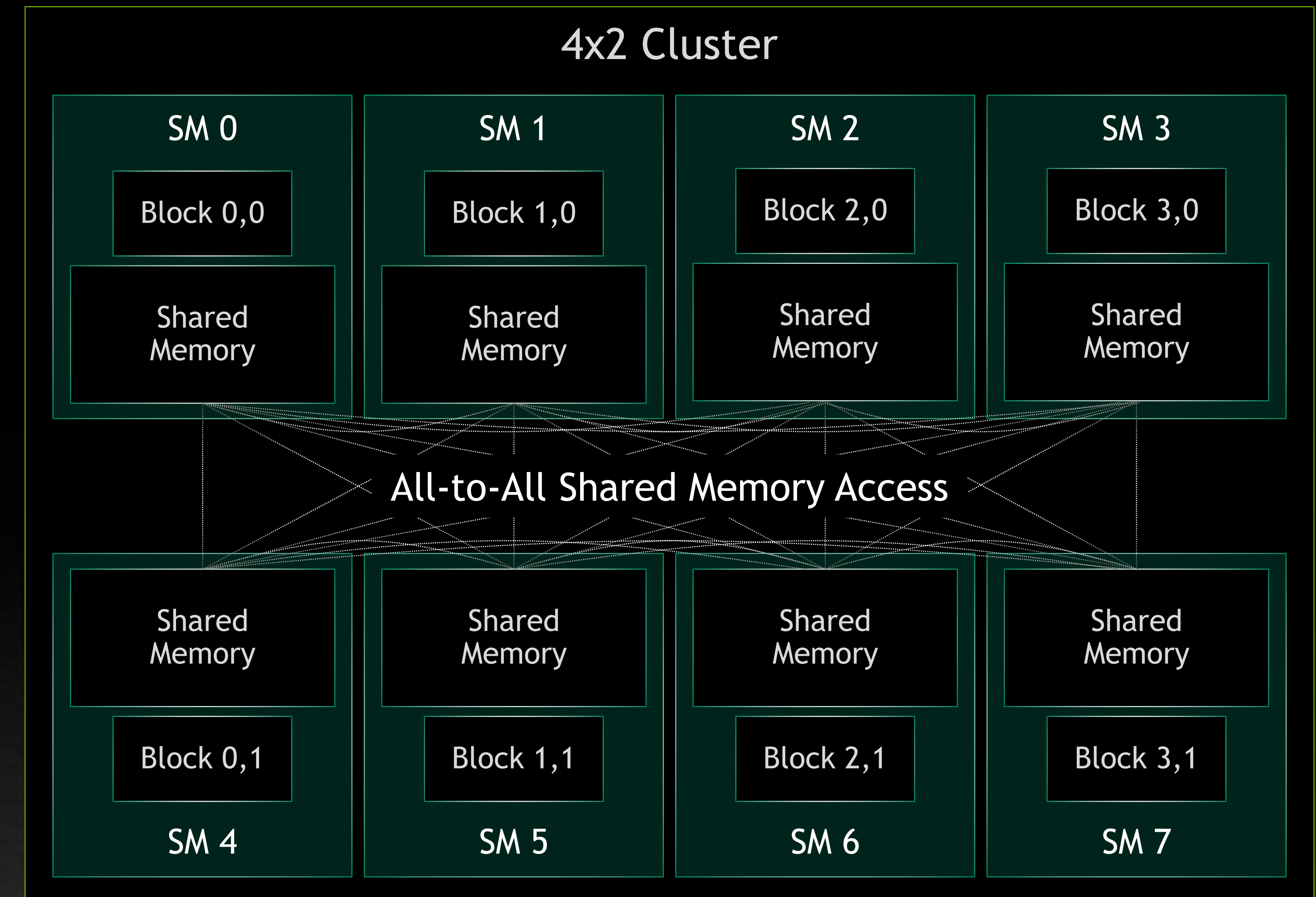
DSMEM is laid out as a **Partitioned Global Address Space**

DSMEM may be indexed by block **rank** - a 1D expansion of cluster index - or by (xyz) index in cluster

```
// Example of getting DSMEM pointer from cluster neighbours
__shared__ int x;

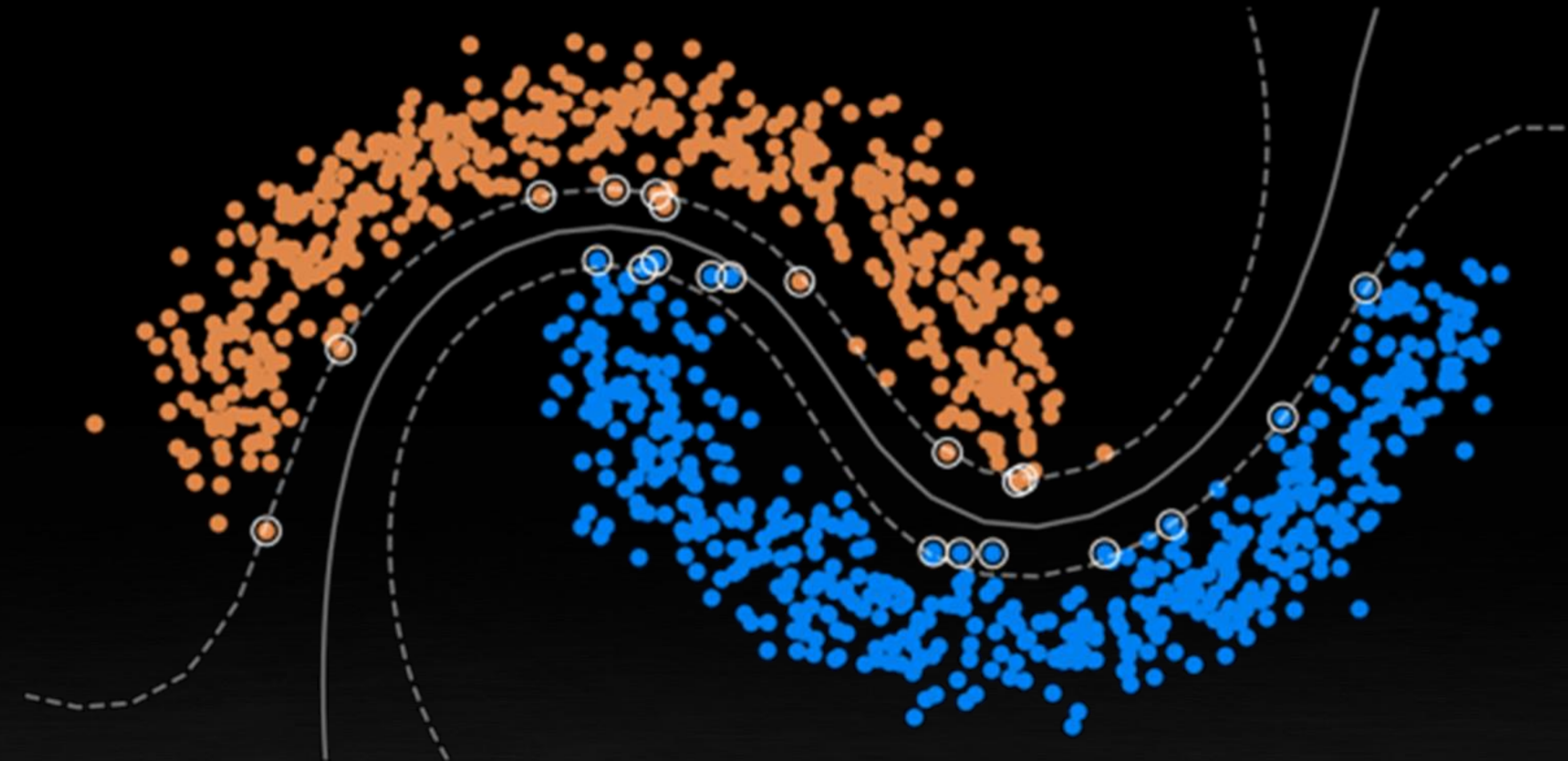
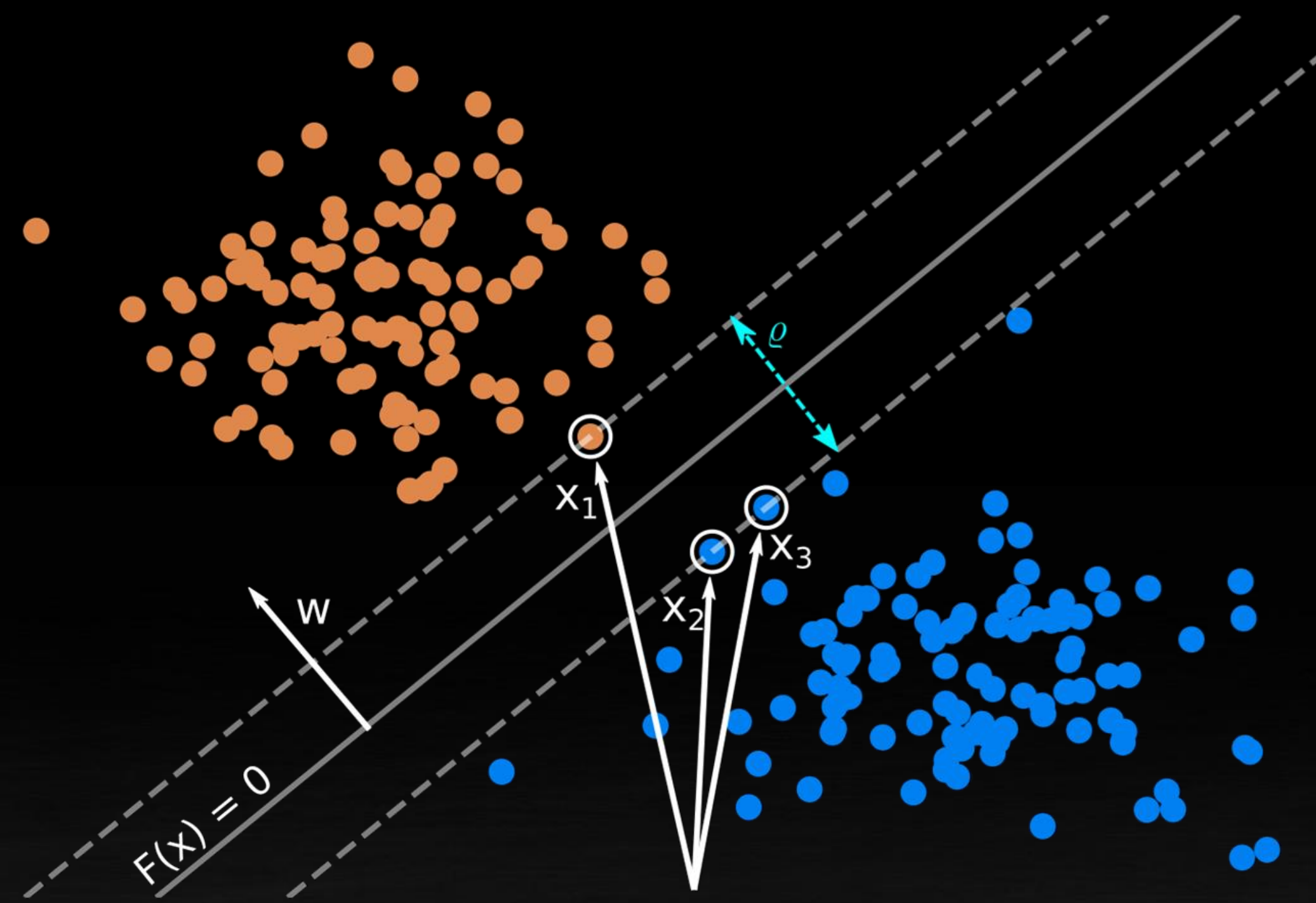
cg::cluster_group = this_cluster();
int *x_ptr = cluster.map_shared_rank(&x, 2);
*x_ptr = 5;

cluster_barrier_arrive();           // Hardware-managed barrier
cluster_barrier_wait();
```



Full load/store/atomic access to all shared memory
between blocks within a cluster

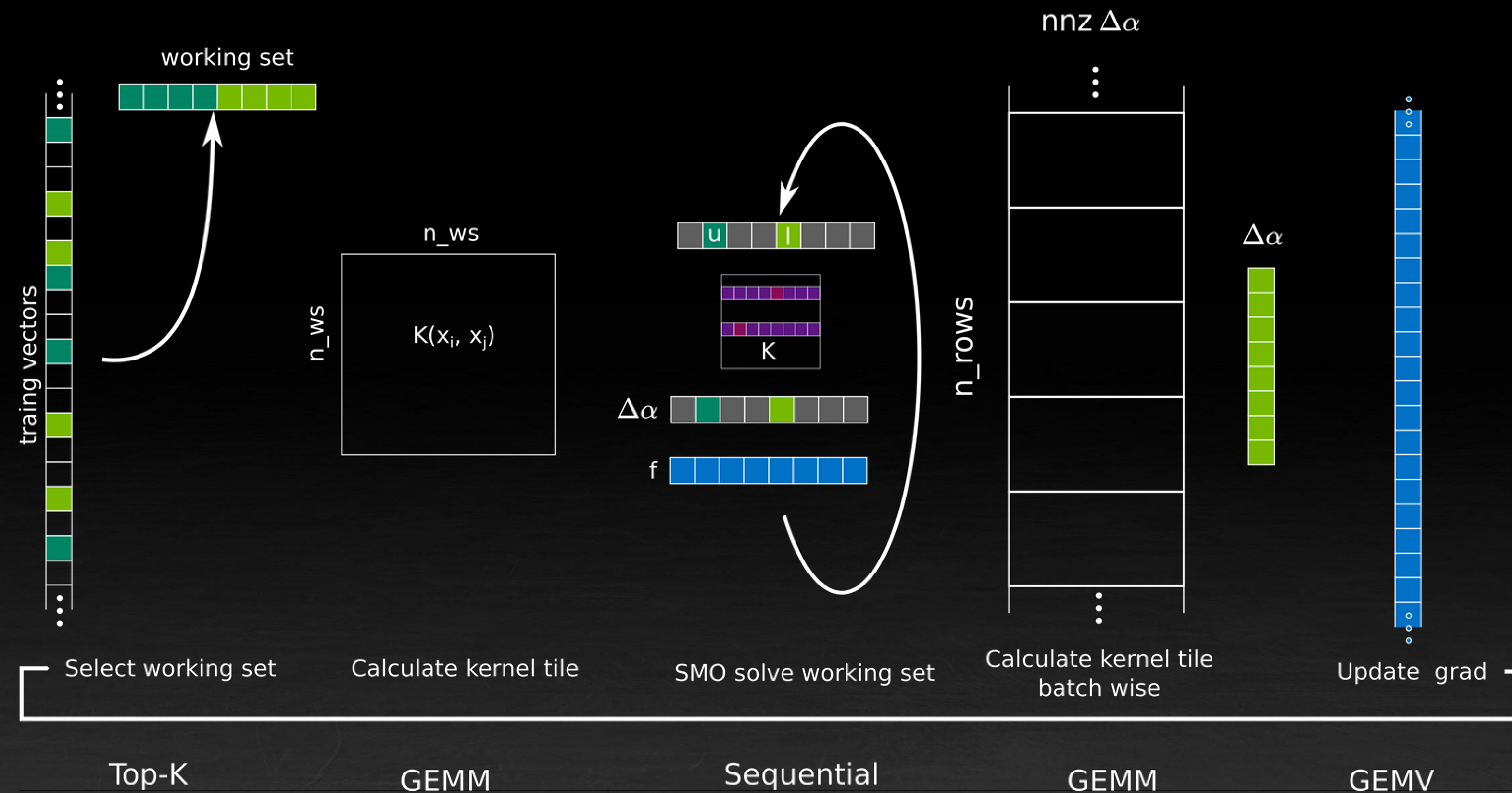
SUPPORT VECTOR MACHINES (SVM)



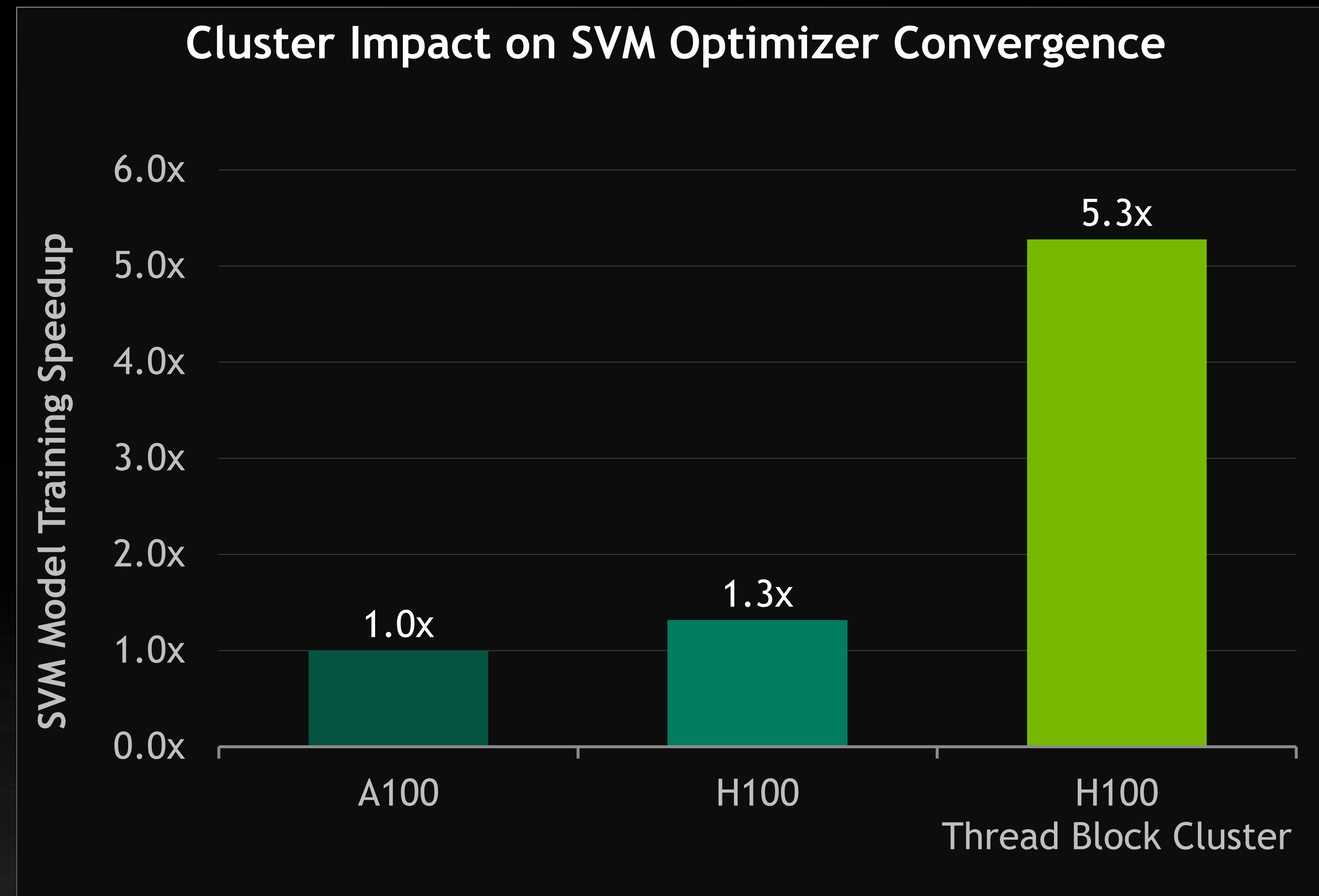
$$F(\vec{x}) = \sum_i \alpha_i y_i K(\vec{x}_i, \vec{x}) + b.$$

SVM ALGORITHM

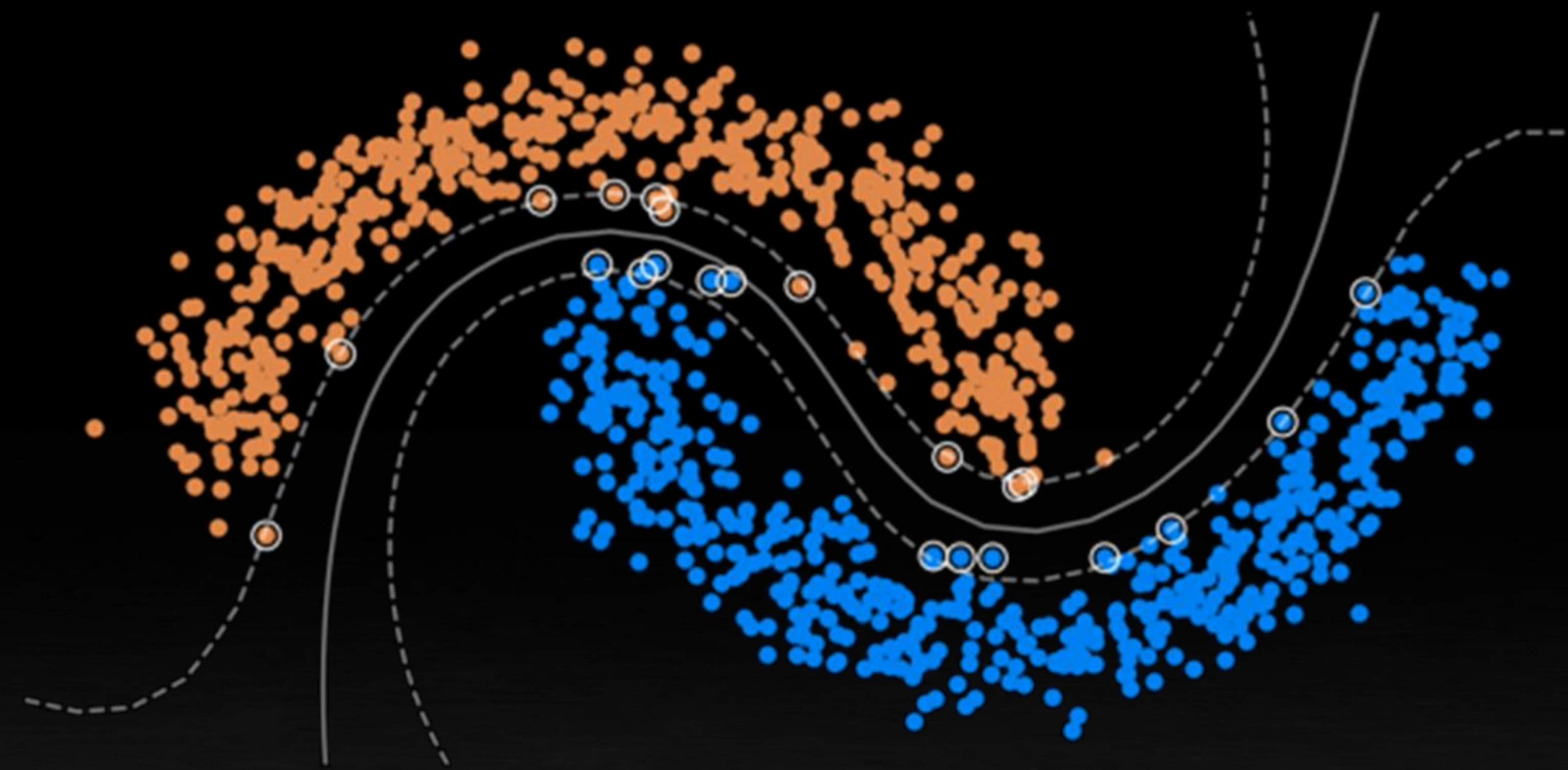
SMO solve step is key to overall performance



EXAMPLE: SUPPORT VECTOR MACHINE (SVM)



A9A dataset (preprocessed version of the UCI Adult data set)
39k samples, 123 features RBF kernel, C=500, gamma=1/123



$$F(\vec{x}) = \sum_i \alpha_i y_i K(\vec{x}_i, \vec{x}) + b.$$

THREAD BLOCK CLUSTERS IN NSIGHT COMPUTE

FileConnectionDebugProfileToolsWindowHelp

ConnectDisconnectTerminateProfile KernelBaselines

cask.ncu-rep

Page: DetailsResult: 2 - 26908 - warp_specialized_kernelLAdd BaselineApply RulesOccupancy Calculator

	Result	Time	Cycles	Regs	GPU	SM Frequency	CC	Process
Current	26908 - warp_specialized_kernel_gmma_utmaldg (1, 2, 1)x(70, 2, 1)x(384, 1, ...	88,992.00 nsecond	142,518	168	0 - NVIDIA Graphics Device	1,597,445,556.90 cycle/second	9.0	[7165] cask-sdk-cask-tester

Launch Statistics

Cluster Size	2.00	Cluster Scheduling Policy	cudaClusterSchedulingPolicySpread
Grid Size	140.00	Registers Per Thread [register/thread]	168.00
Block Size	384.00	Static Shared Memory Per Block [byte/block]	0.00
Threads [thread]	107,520.00	Dynamic Shared Memory Per Block [byte/block]	218,368.00
Waves Per SM	1.00	Driver Shared Memory Per Block [byte/block]	1,024.00
Function Cache Configuration	cudaFuncCachePreferNone	Shared Memory Configuration Size [byte]	233,472.00

Occupancy

Theoretical Occupancy [%]	18.75	Block Limit Registers [block]	1.00
Theoretical Active Warps per SM [warp]	12.00	Block Limit Shared Mem [block]	1.00
Achieved Occupancy [%]	15.56	Block Limit Warps [block]	5.00
Achieved Active Warps Per SM [warp]	9.96	Block Limit SM [block]	32.00
Max Cluster Size	16.00	Max Active Clusters	70.00

Source Counters

Branch Instructions [inst]	1,576,420.00	Branch Efficiency [%]	99.86
Branch Instructions Ratio [%]	0.05	Avg. Divergent Branches	4.00

Hopper SM 90

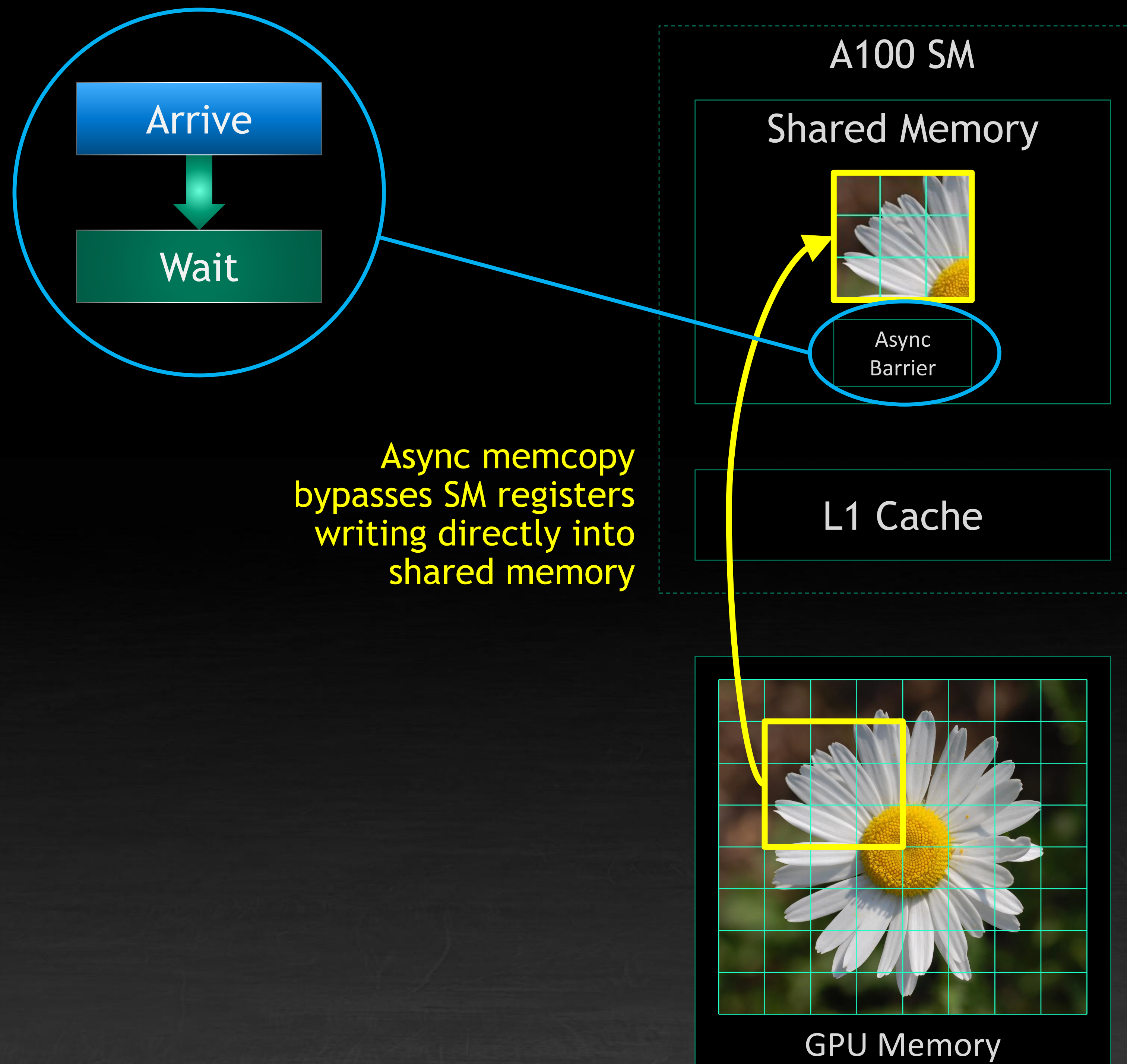
Cluster Scheduling Policy

Cluster Launch Size

Cluster Occupancy

A100 RECAP: ASYNCHRONOUS COPY TO SHARED MEMORY

cuda::memcpy_async



cuda::memcpy_async()

Bypasses SM registers to write directly to shared

Uses **Asynchronous Barrier** to signal completion

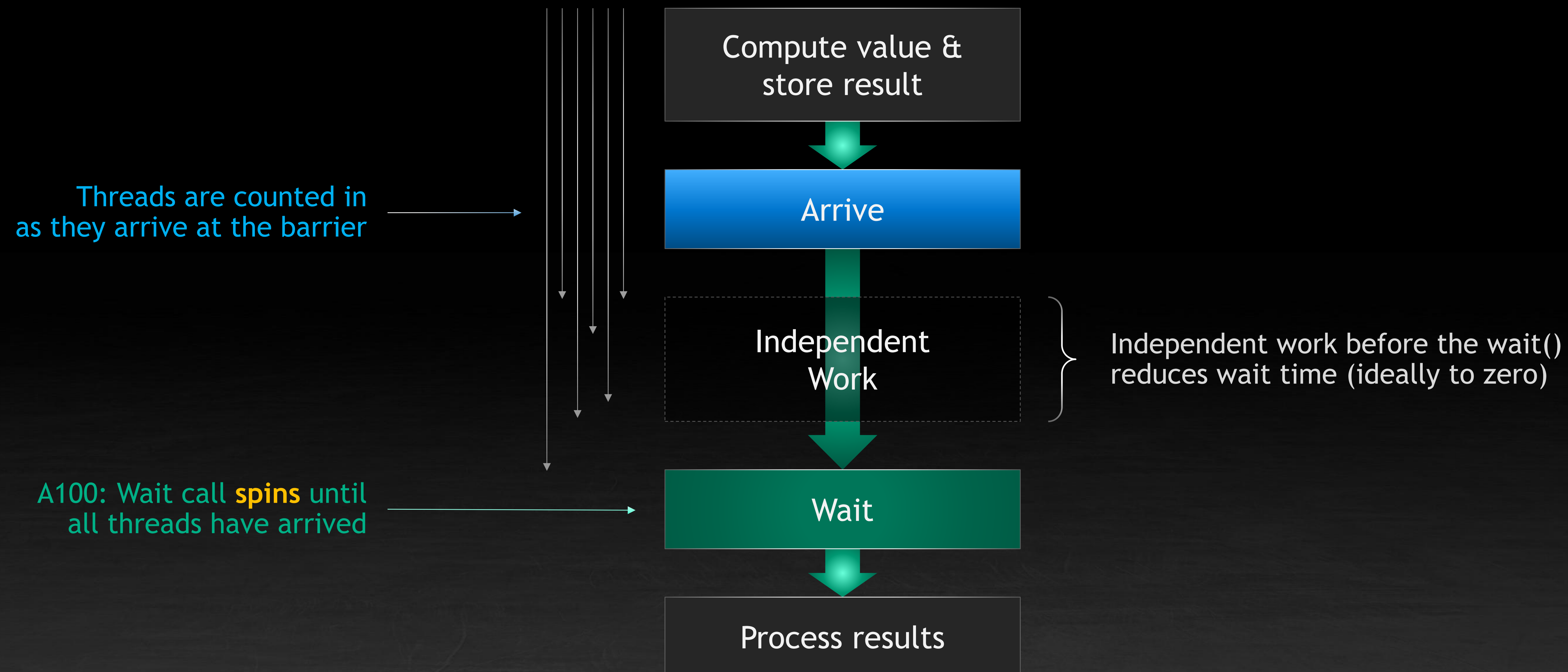
Threads work together to copy data in parallel

```
// Async block-wide memcpy using Cooperative Groups
cg::thread_block tb = cg::this_thread_block();
size_t copy_count, index = 0;
while (index < elementsPerThreadBlock) {
    cg::memcpy_async(
        tb,
        local_smem,
        elementsInShared,
        global_data + index,
        elementsPerThreadBlock - index);
    copy_count = min(elementsInShared,
                     elementsPerThreadBlock - index);
    cg::wait(tb);

    // ... use data copied into shared memory ...
    index += copyCount;
}
```

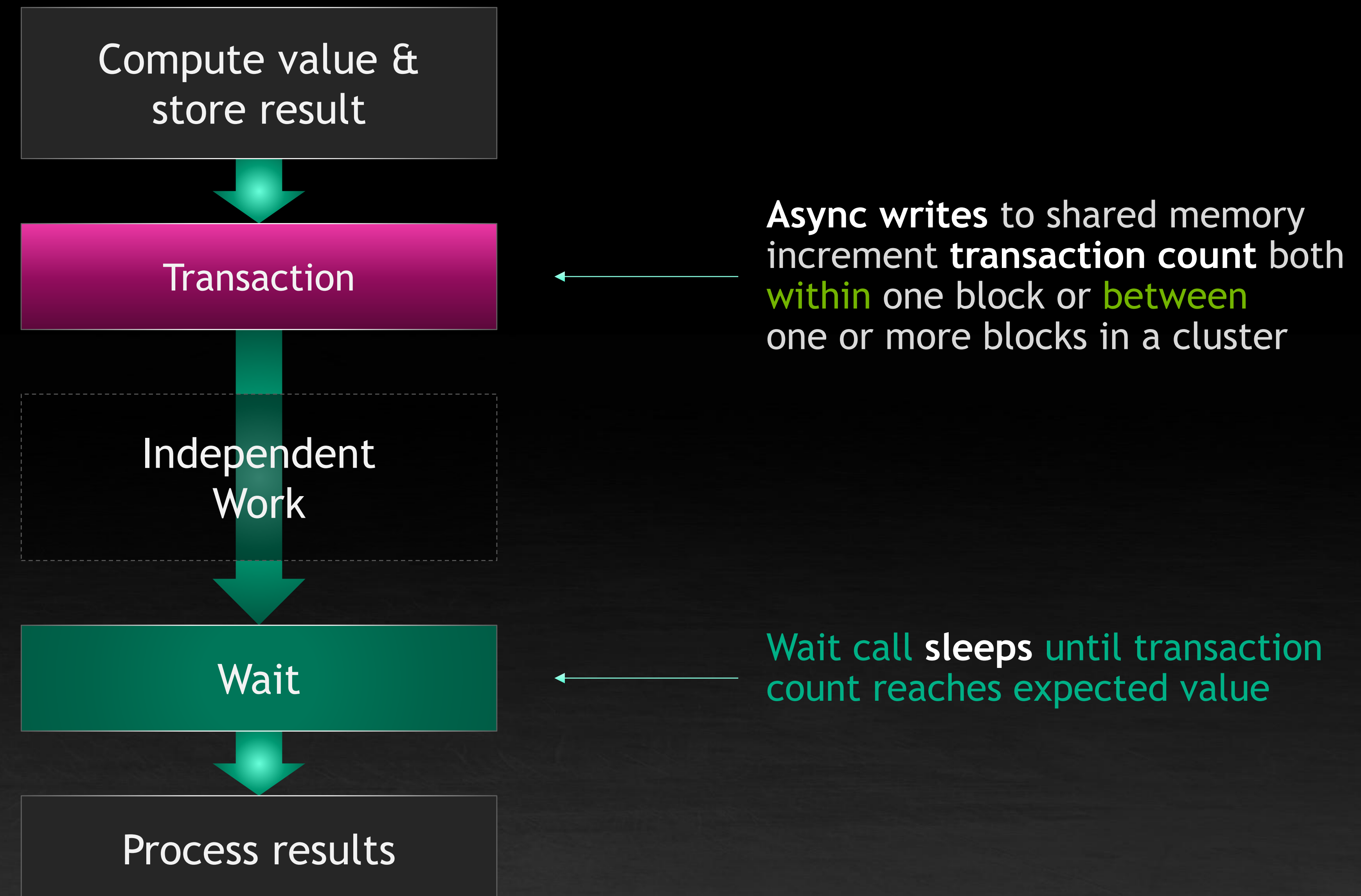

ASYNCHRONOUS BARRIERS

`cuda::barrier`



NEW: TRANSACTION BARRIERS

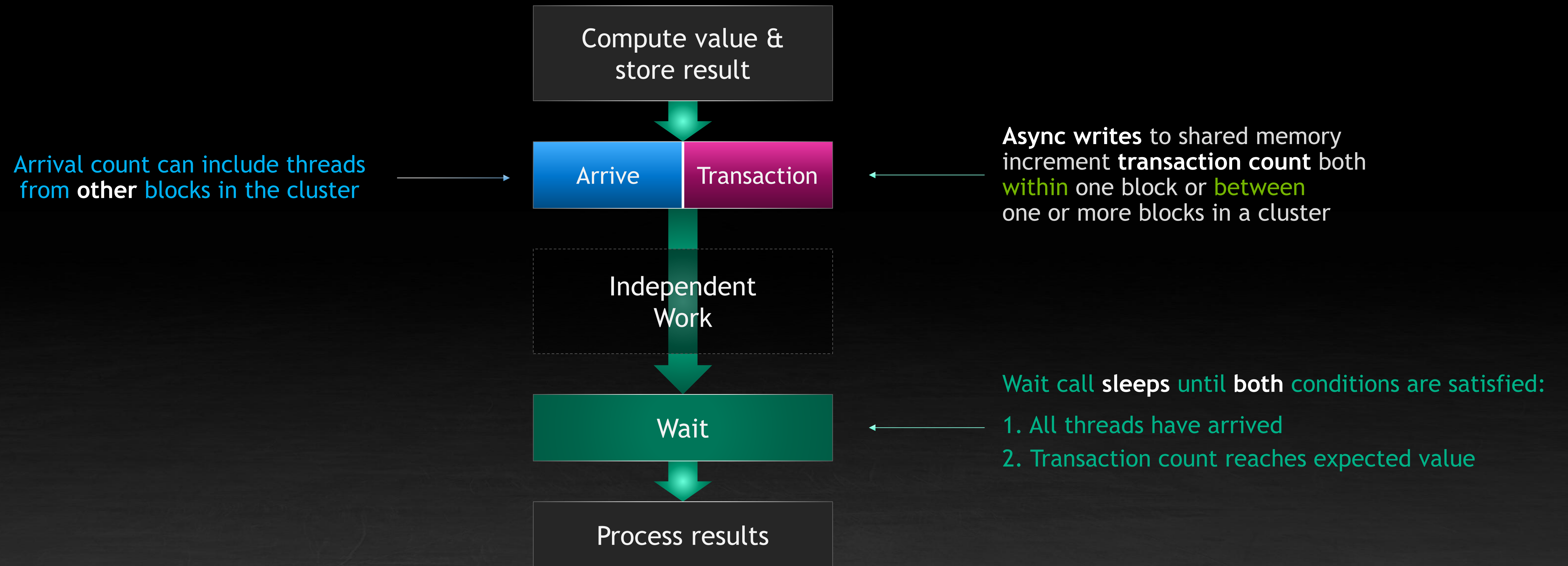
Data can now trigger a barrier



Drop-in enhancement to existing `cuda::barrier`

NEW: ASYNCHRONOUS TRANSACTION BARRIERS

Trigger barrier with both transactions and threads



Drop-in enhancement to existing `cuda::barrier`

ASYNCHRONOUS ONE-SIDED MEMORY COPIES

`cuda::memcpy_async` within & between blocks in a cluster automatically updates transaction count without handshake

Data arrival increments transaction count

One-sided communication within cluster

Accepts async store and atomic operations

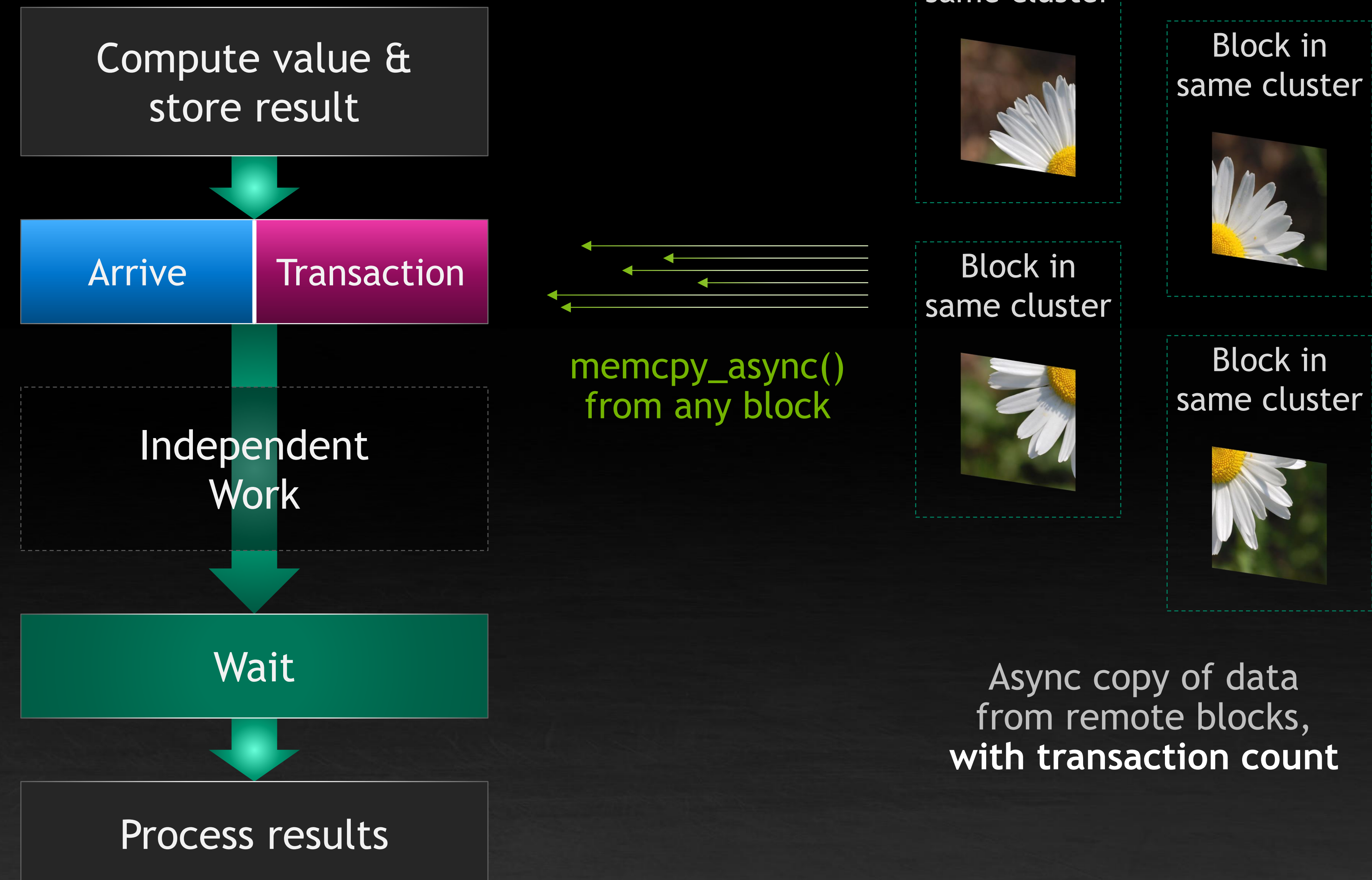
Highly efficient control flow

Self-synchronizing: no memory fence required to update transaction count

7x faster than { data + fence + flag } handshake

Receiver threads **sleep** on wait (instead of busy-waiting) until data arrives

Fast wakeup signal directly from barrier



ASYNCHRONOUS ONE-SIDED COMMUNICATION

No-handshake messaging & data transfer within a block or a cluster

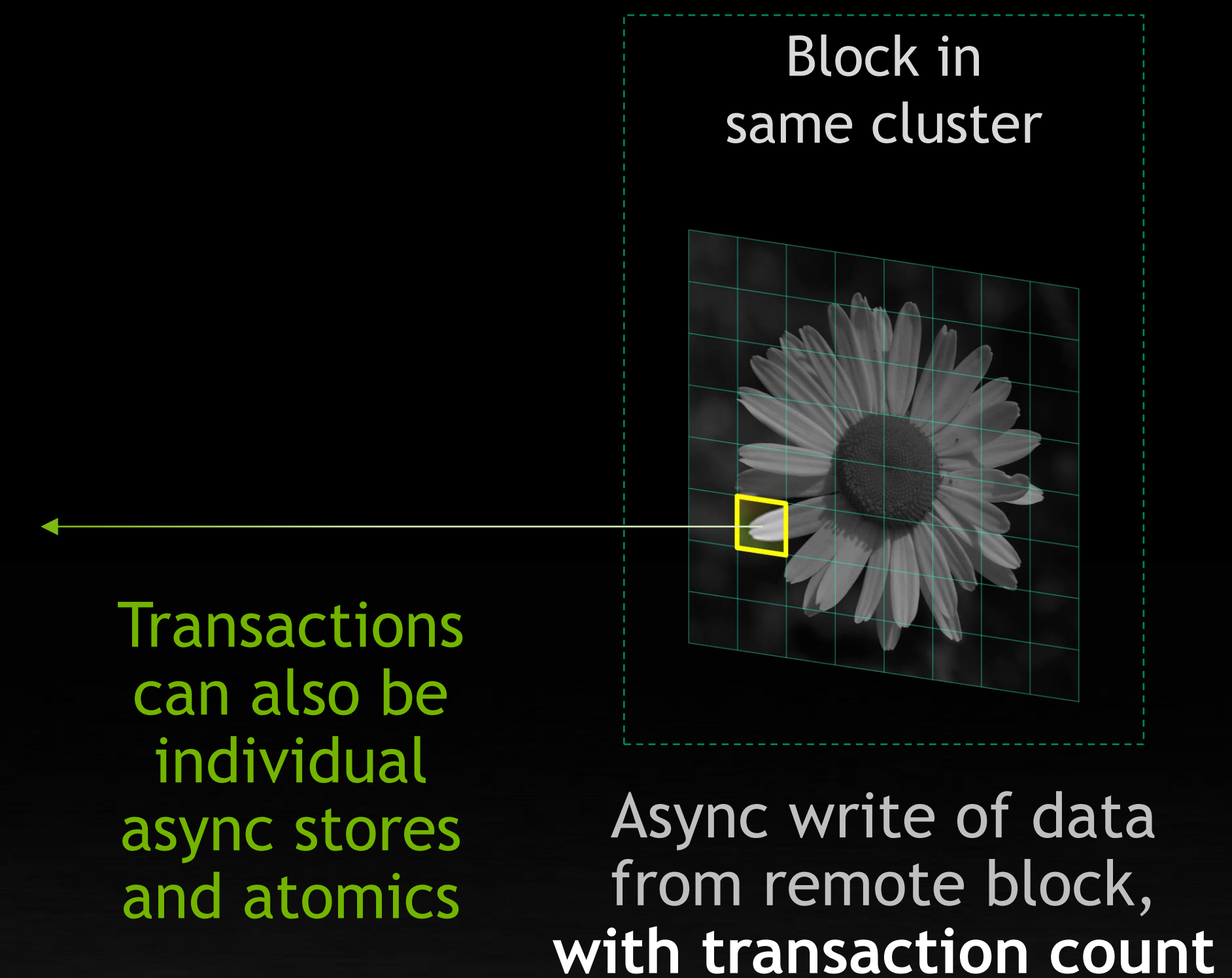
```
cuda::atomic_ref ref;           // Initialization not shown
cuda::barrier admirer;         // Initialization not shown

// Remote store with transaction count
void atomic_store_async(ref, value, barrier);

// Remote atomic reduction operations
void atomic_add_async(ref, value, barrier);
void atomic_min_async(ref, value, barrier);
void atomic_max_async(ref, value, barrier);
void atomic_inc_async(ref, value, barrier);
void atomic_dec_async(ref, value, barrier);
void atomic_and_async(ref, value, barrier);
void atomic_or_async(ref, value, barrier);
void atomic_xor_async(ref, value, barrier);

// Note: CAS is not supported as it is not a reduction operation
```

For these individual async store/atomic operations,
initialise barrier to expected message size before sending



TENSOR MEMORY ACCELERATOR UNIT (TMA) FOR ASYNC DATA MOVEMENT

Example: One-sided halo exchange within a cluster

Tensor Memory Accelerator Unit

Hardware-accelerated **bi-directional** bulk copy

Global $\leftrightarrow$ Shared Memory

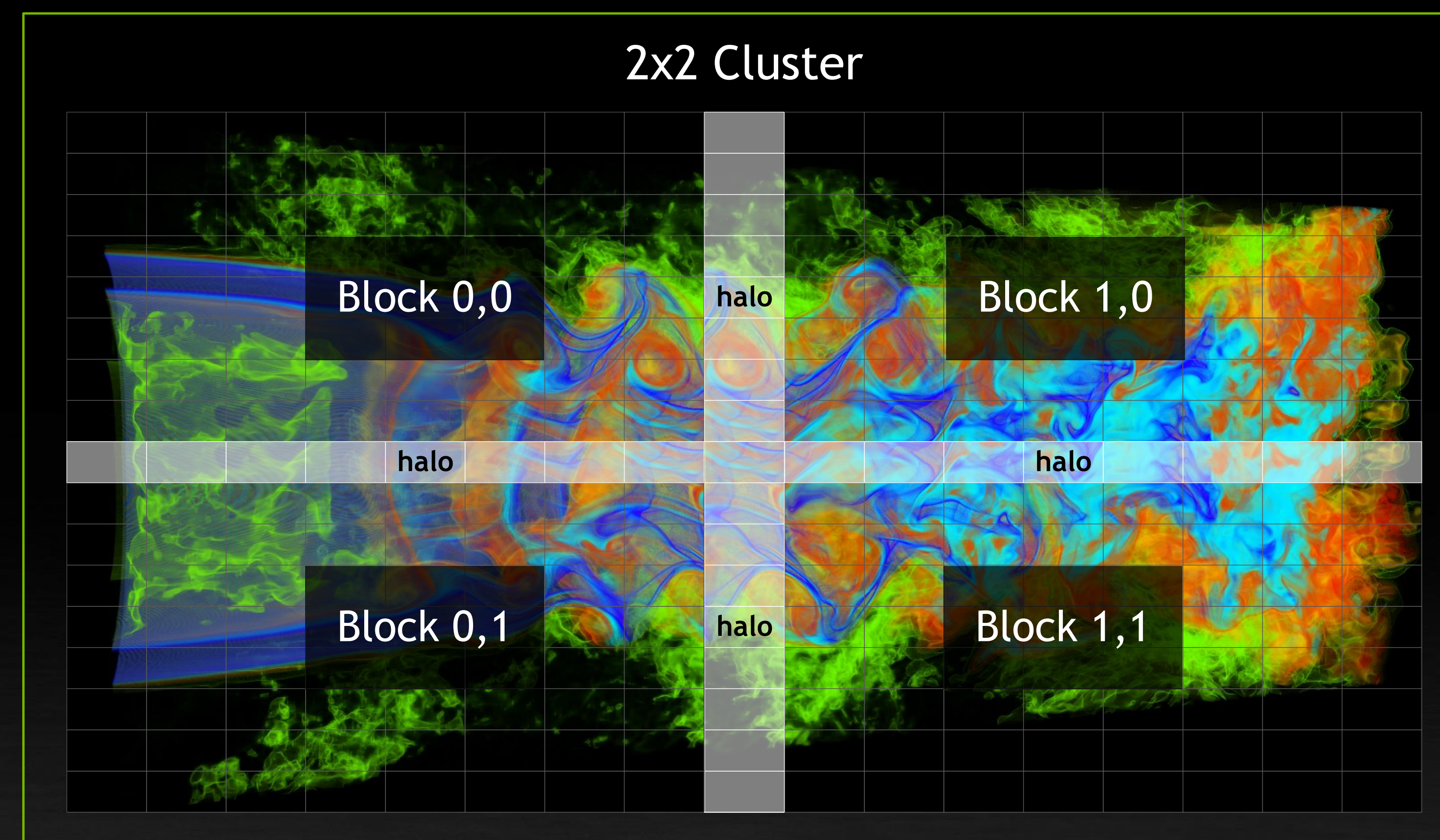
DSMEM $\leftrightarrow$ DSMEM within cluster

Uses **Asynchronous Transaction Barrier** to track completion

Low-Latency One-Sided Data Transfer

Supports one-sided bulk copying within cluster

Supports element-wise reductions when copying to global memory



Self-synchronizing transactions enable 7x faster halo exchange between blocks in the same cluster

TENSOR MEMORY ACCELERATOR UNIT (TMA) FOR ASYNC DATA MOVEMENT

Tensor Memory Accelerator Unit

Hardware-accelerated **bi-directional** bulk copy

Global $\leftrightarrow$ Shared Memory

DSMEM $\leftrightarrow$ DSMEM within cluster

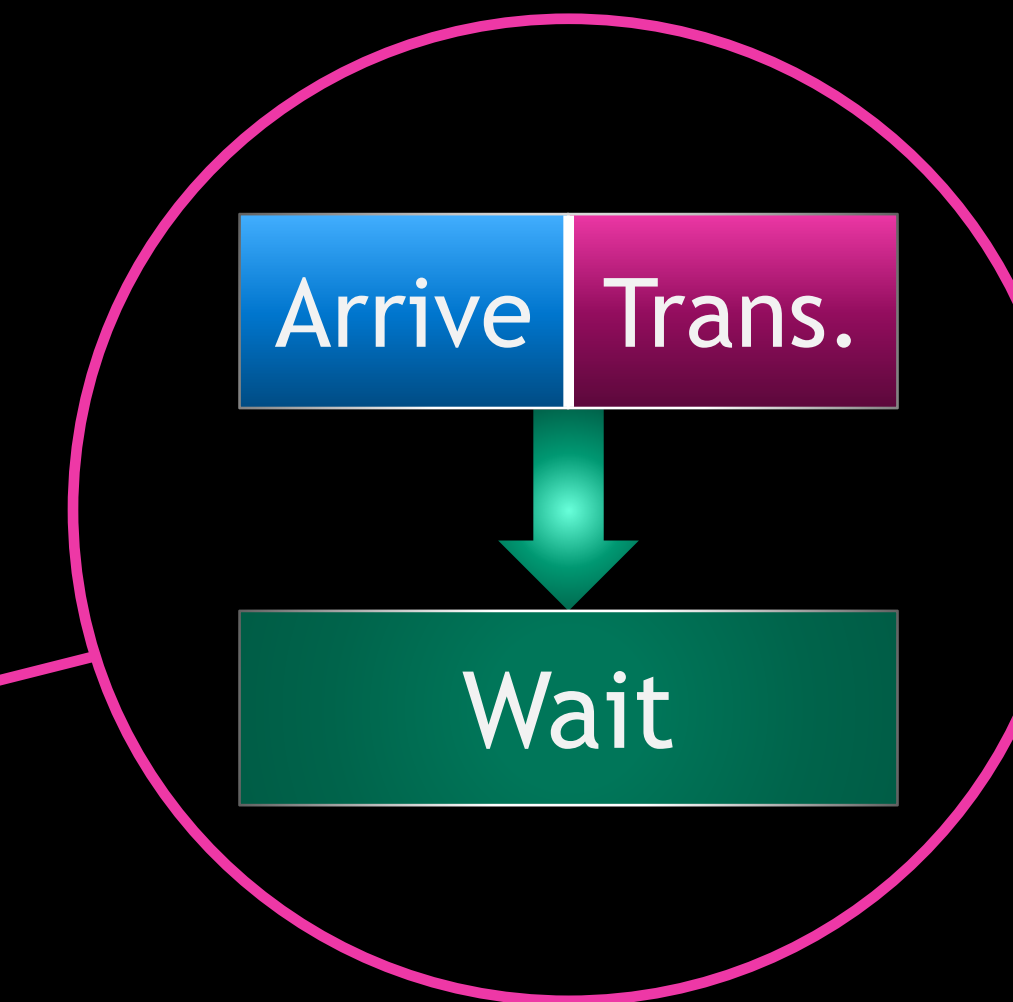
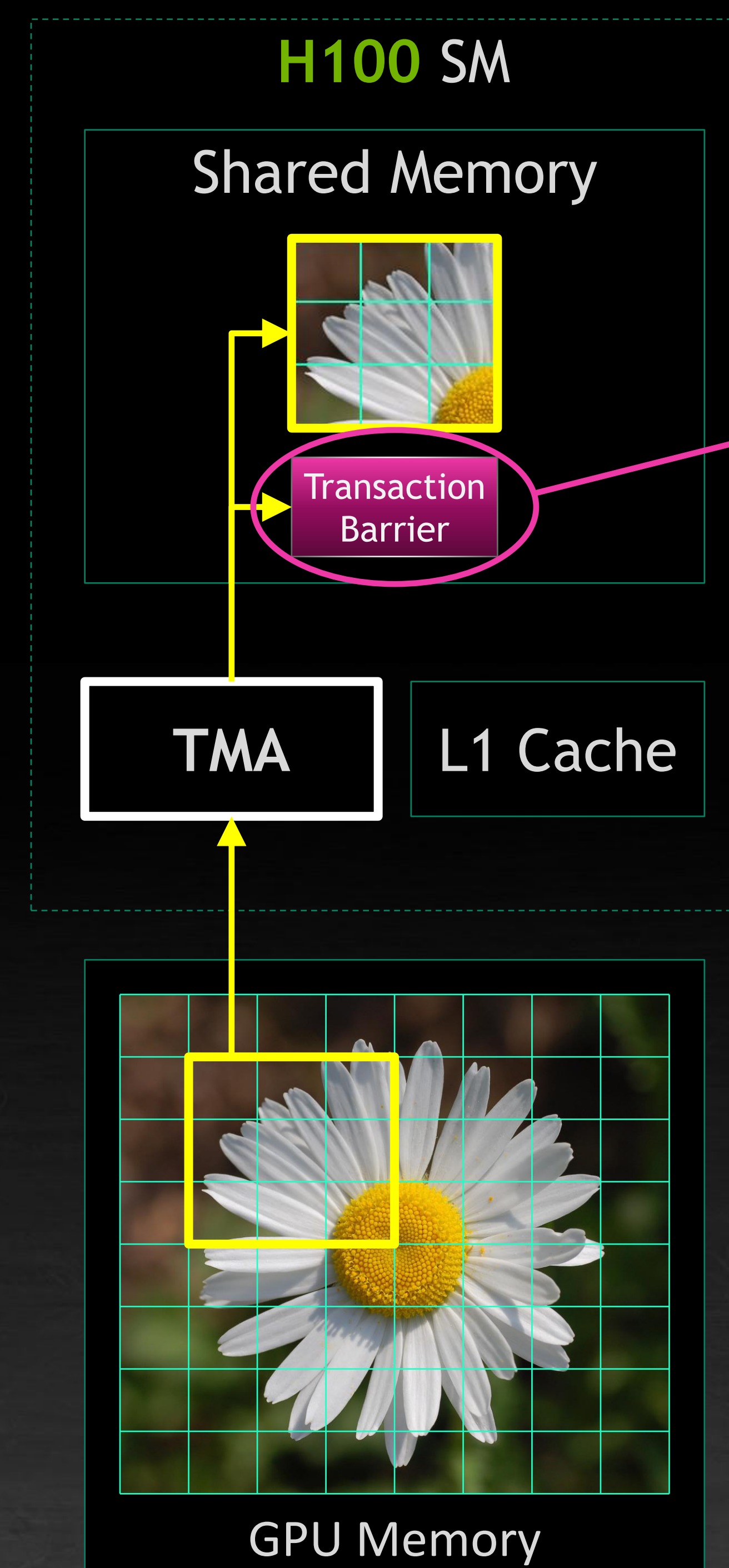
Uses **Asynchronous Transaction Barrier** to track completion

Faster And More Flexible Async Copying

Drop-in enhancement to existing **cuda::memcpy_async()** function

Single thread triggers any-size copy - no need for looping or collective copying

Multiple copy operations can contribute to a single cluster-wide async transaction barrier



New Asynchronous Transaction Barrier

```
__shared__ cuda::barrier barrier; // Pre-initialised
auto block = cg::this_thread_block();

// On H100, memcpy_async automatically issues TMA
// enhanced copy and updates transaction count
if(threadIdx.x == 0) {
    cuda::memcpy_async(block,
                        smem,
                        gmem,
                        block.size() * sizeof(int),
                        barrier);
}
```

Same copy syntax becomes faster and more flexible
New syntax to enable TMA capabilities

HARDWARE ACCELERATED 1D-5D TENSOR MEMORY COPY

Tensor Memory Accelerator Unit

Hardware-accelerated **bi-directional** bulk copy

Global $\leftrightarrow$ Shared Memory

DSMEM $\leftrightarrow$ DSMEM within cluster

Uses **Asynchronous Transaction Barrier** to track completion

Multi-Dimensional Tensor Copying

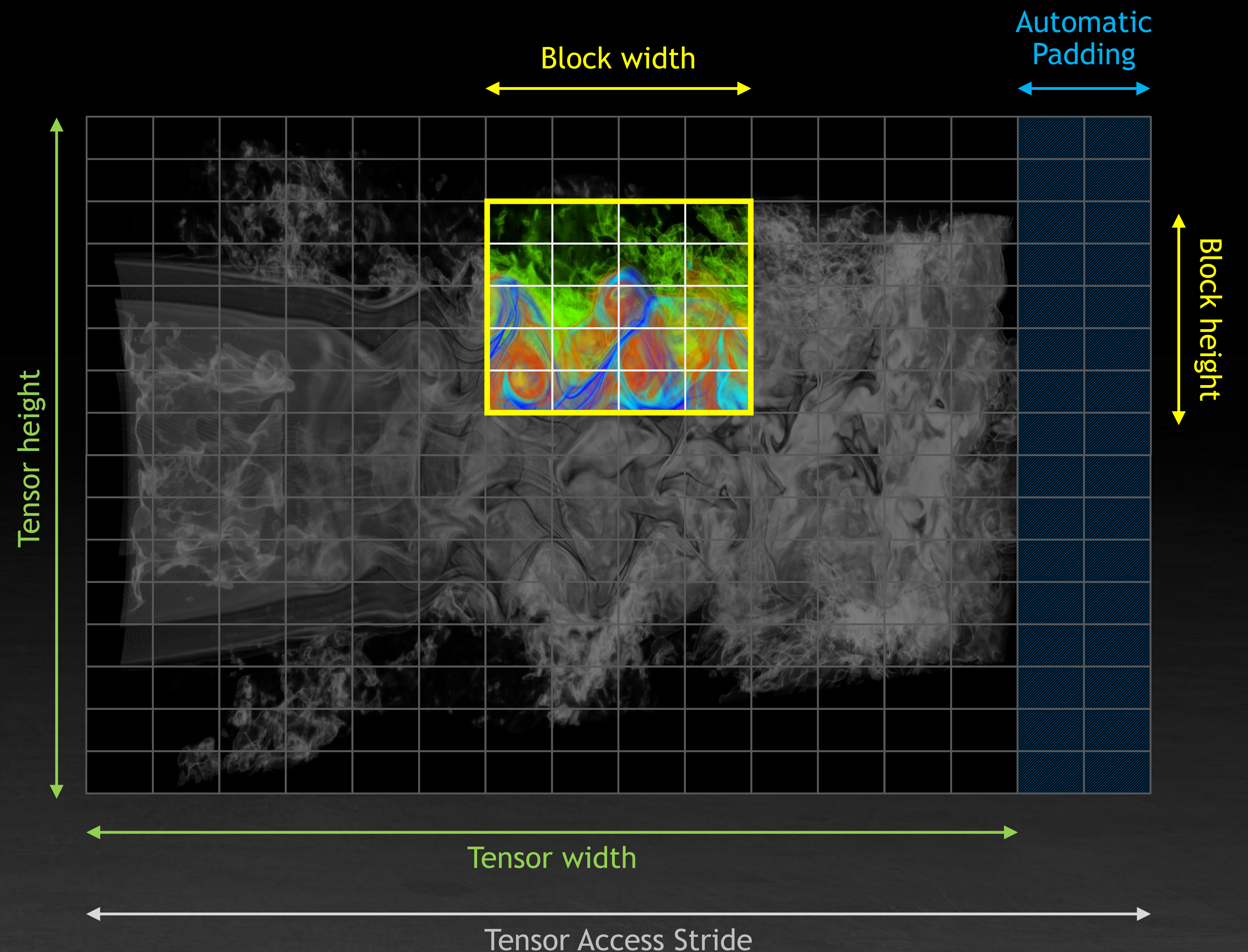
Automatic stride & address generation up to tensors of rank 5

Boundary padding for out-of-bounds accesses

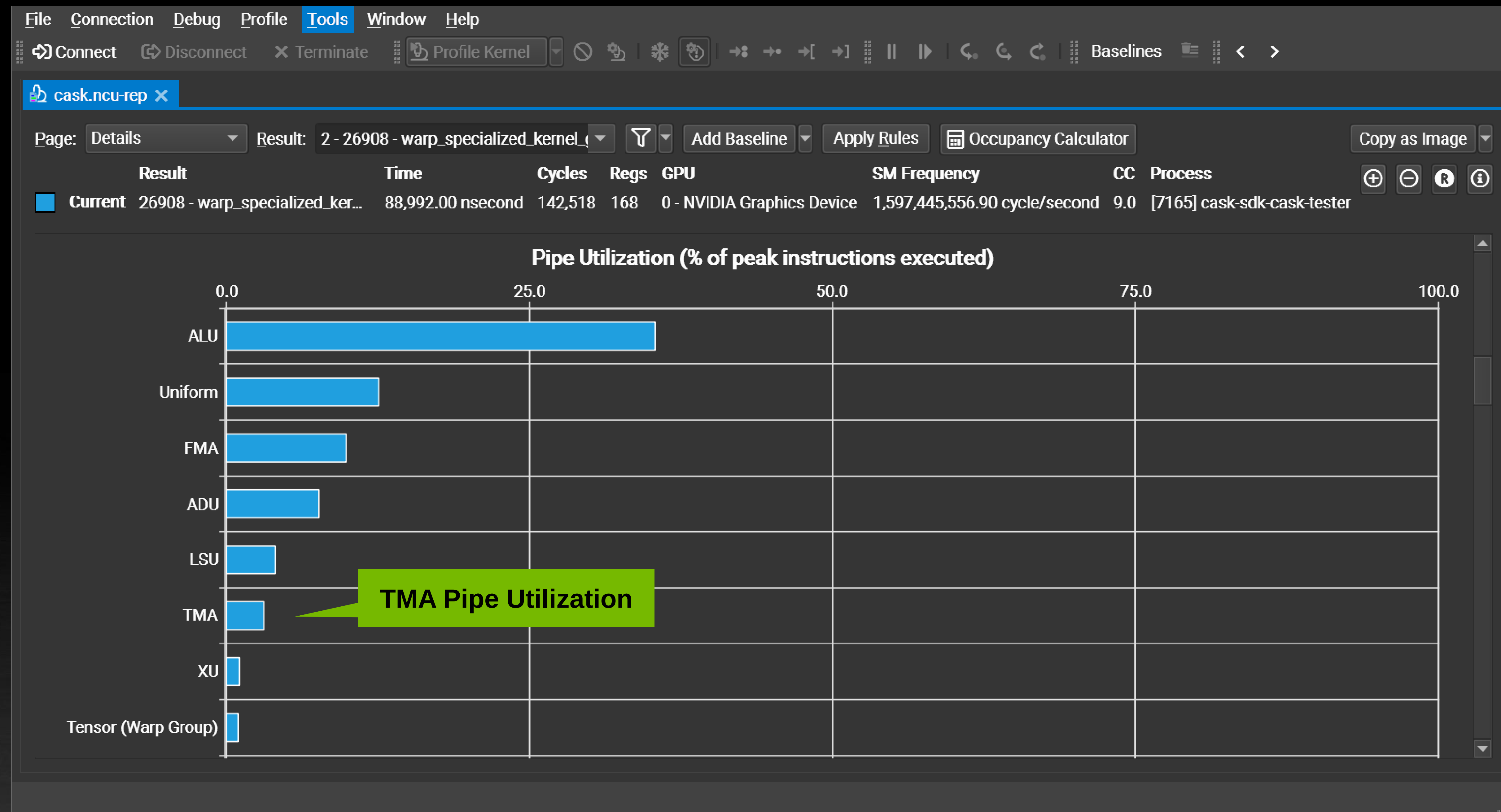
Fire-and-forget from a single thread - everything handled by TMA

No iteration or bounds-checking code required

The TMA can copy sub-regions of a multi-dimensional tensor

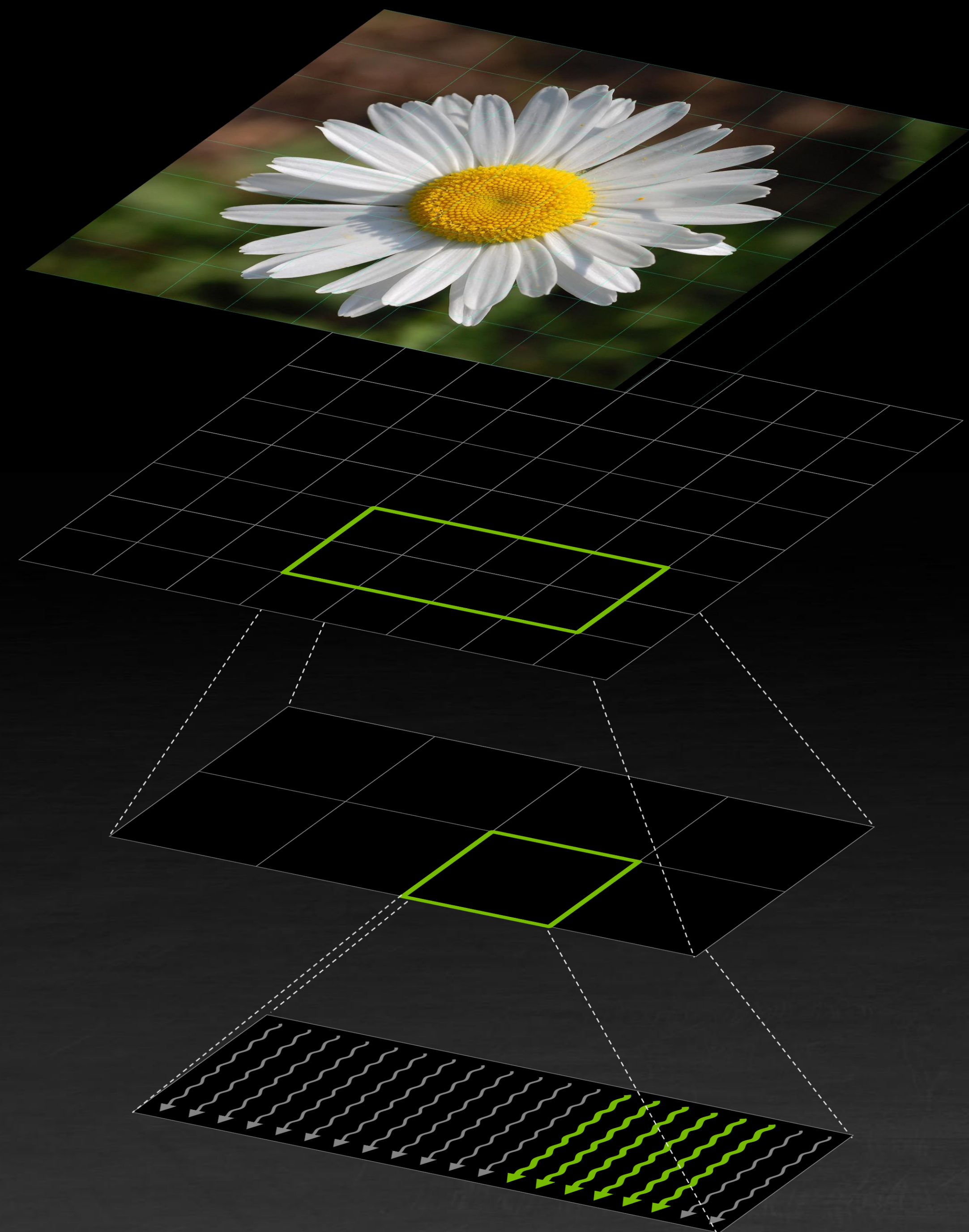


TENSOR MEMORY ACCELERATOR UTILIZATION IN NSIGHT COMPUTE



COOPERATIVE GROUPS: PROGRAMMING TO THE NATURAL EXECUTION HIERARCHY

Cooperative Groups is an **explicit** model for expressing cooperation & synchronization among threads



Grid
of work

```
cooperative_groups::grid_group grid = this_grid();
```

Cluster
of Blocks

```
cooperative_groups::cluster_group cluster = this_cluster();
```

Block
of Threads

```
cooperative_groups::thread_block block = this_thread_block();
```

Warp
of Threads

```
cooperative_groups::thread_block_tile<32> warp = tiled_partition<32>(block);
```


COLLECTIVE OPERATIONS AT EVERY LEVEL OF HIERARCHY

Synchronization

Grid must be launched cooperatively in order to use grid-scope collectives

```
cooperative_groups::grid_group grid = this_grid();
```

```
cooperative_groups::cluster_group cluster = this_cluster();
```

Synchronization

Reduction

Prefix-sum

```
cooperative_groups::thread_block block = this_thread_block();
```

Warp & smaller groups can use: shfl, any, all, ballot, match

```
cooperative_groups::thread_block_tile<32> warp = tiled_partition<32>(block);
```


COLLECTIVE OPERATIONS INCLUDING FOR MULTI-WARP GROUPS

Supported thread_block_tile sizes of 64, 128, 256, 512

Synchronization

```
cooperative_groups::grid_group grid = this_grid();
```

```
cooperative_groups::cluster_group cluster = this_cluster();
```

Synchronization

Reduction

Prefix-sum

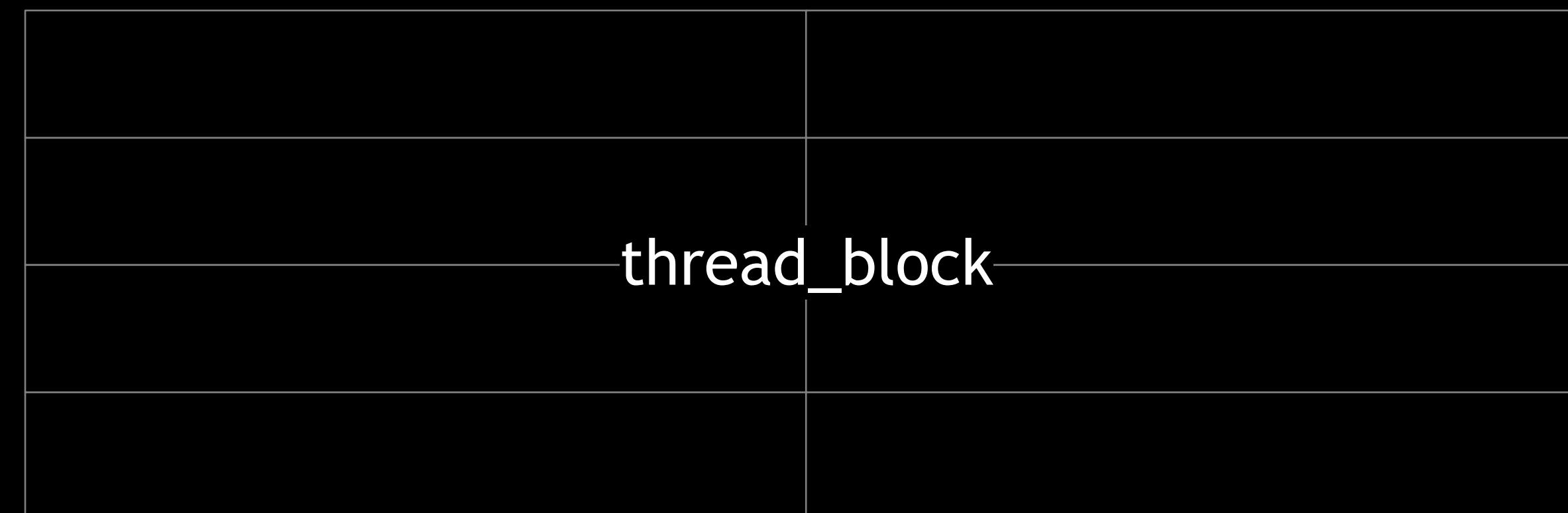
```
cooperative_groups::thread_block block = this_thread_block();
```

```
cooperative_groups::thread_block_tile<128> quadwarp = tiled_partition<128>(block);
```

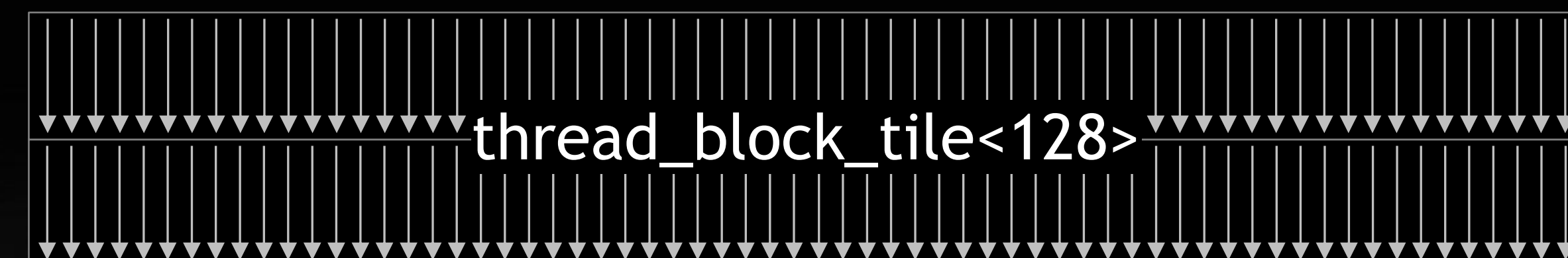
```
cooperative_groups::thread_block_tile<32> warp = tiled_partition<32>(quadwarp);
```


TYPE SAFETY ENABLES COMPOSABLE LIBRARIES OF PARALLEL FUNCTIONS

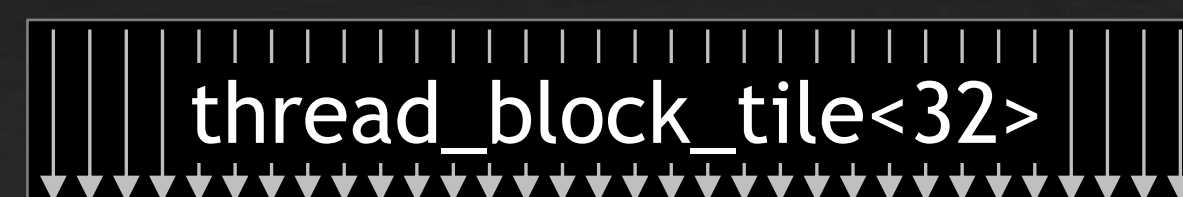
Allows being explicit about how many threads a function wants - e.g. Radix-128 FFT can enforce 128 threads



Base thread block of 256 threads (8 warps)



Group of 128 threads (4 warps)



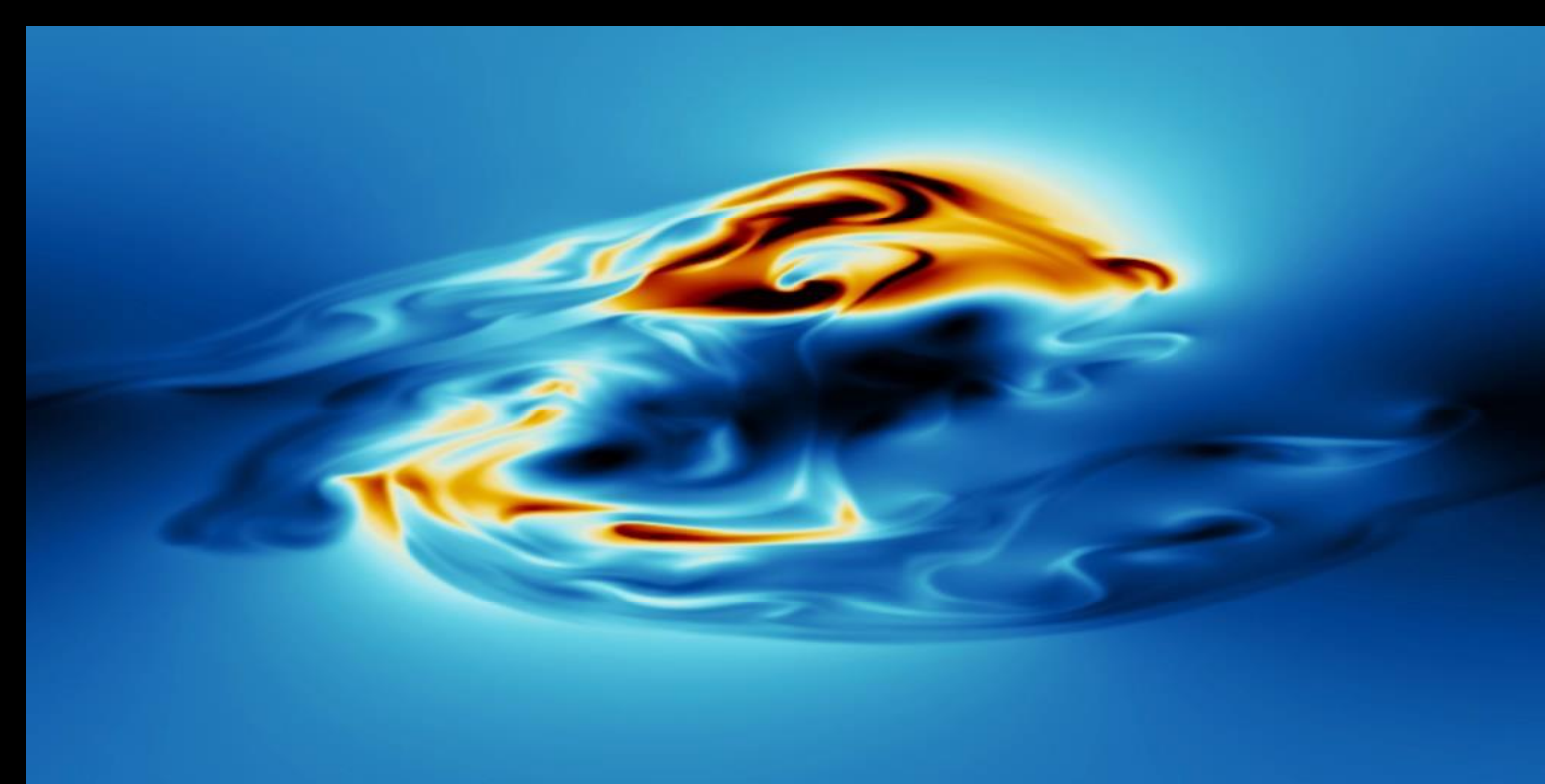
Group of 32 threads (1 warp)

```
using namespace cooperative_groups;
__global__ void kernel(float *in, float *out, ...) {
    // Subdivide my block into 128-thread tiles for the FFT call
    thread_block_tile<128> tile128 = tiled_partition<128>(this_thread_block());
    fft128(tile128, in, out, ...);
}
```

```
// A radix-128 FFT function which *must* be called with 128 threads
__device__ void fft128(const thread_block_tile<128> &group,
                      float *in, float *out, ...) {
    do_fft(in, out, ...);
    group.sync();      // Sync all 128 threads in the group
}
```

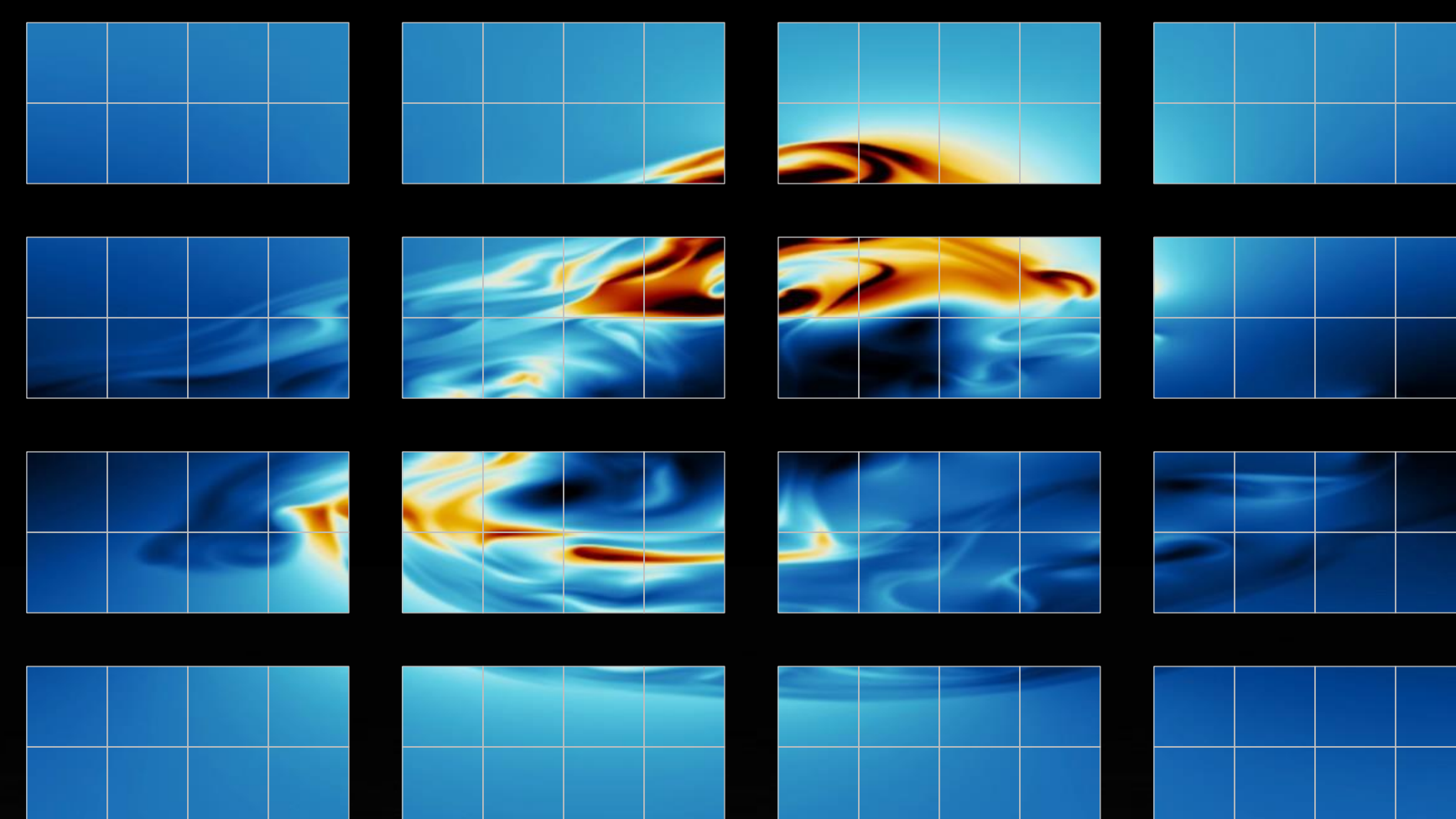
```
// Functions can require an explicit thread count via the type system
__device__ int sum(const thread_block_tile<32> &group,
                  float *in, float *out, ...) {
    // Collective operations are common to all block and sub-block groups
    return group.reduce(in[group.thread_rank()]);
}
```


EXAMPLE: HIERARCHICAL EXECUTION, DATA EXCHANGE AND SYNCHRONIZATION



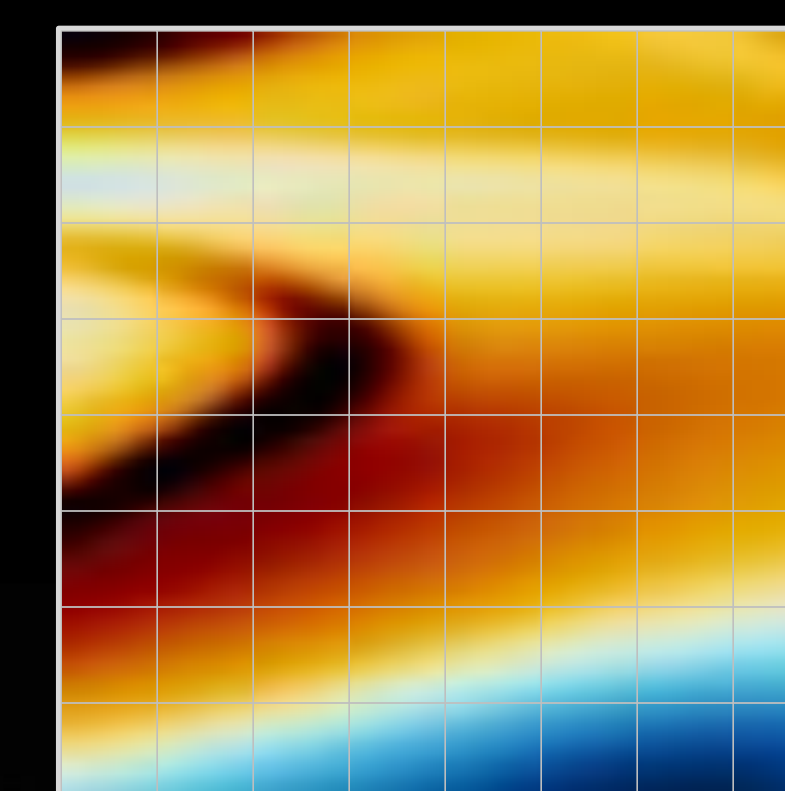
1

Launch cooperative grid
of 4x4 clusters, each
with 4x2 blocks



2

Use cooperative cluster indexing
to pull data from neighbours

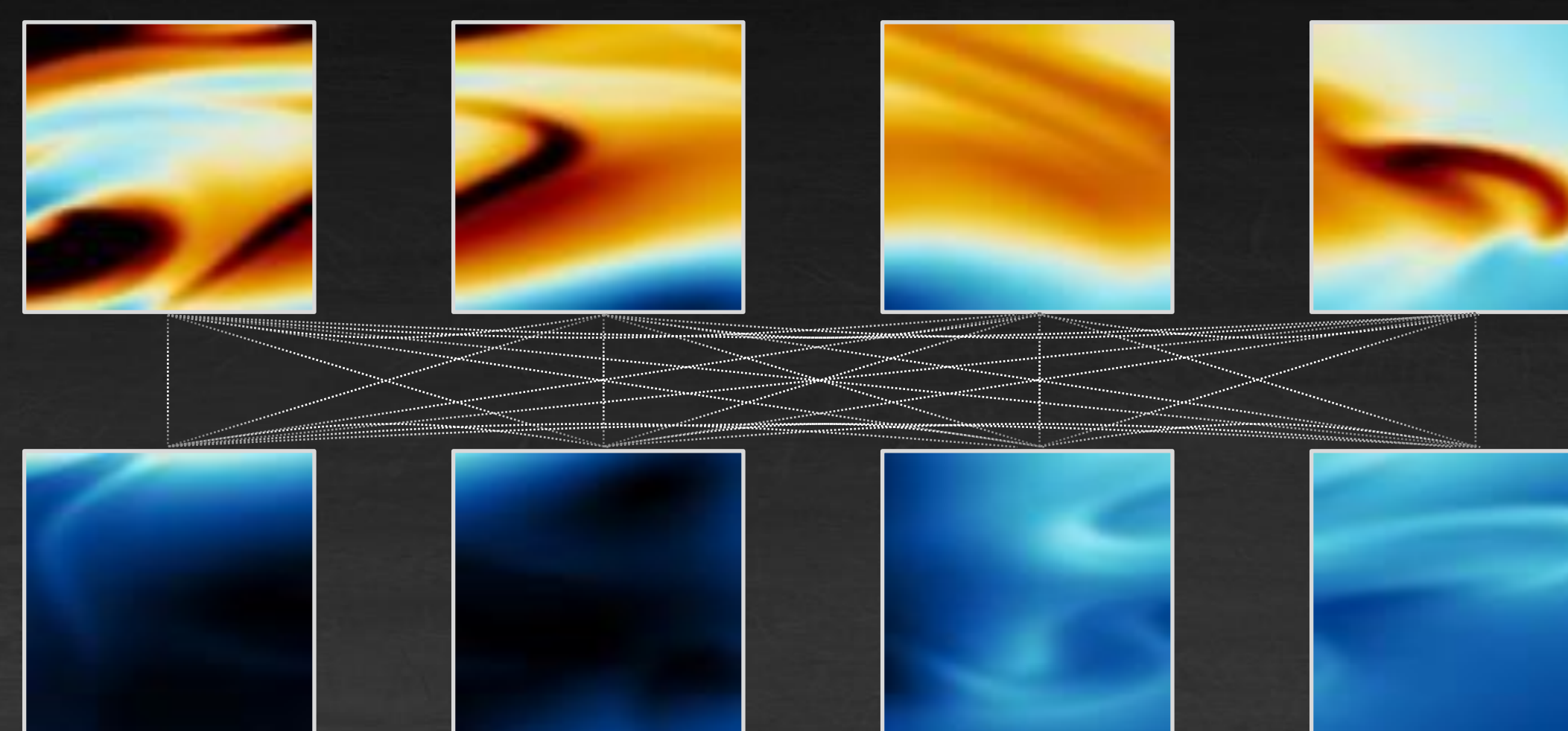


3

Solve locally
within each
block

5

Synchronize with
grid and solve at
grid level; go to 2



4

Synchronize with cluster and
solve at cluster level

Use distributed shared memory for fast inter-block communication.

EXAMPLE: LONGSTAFF SCHWARTZ PRICING MODEL

Producer/Consumer task parallelism

What is Longstaff Schwartz?

Pricing technique for Quantitative Finance

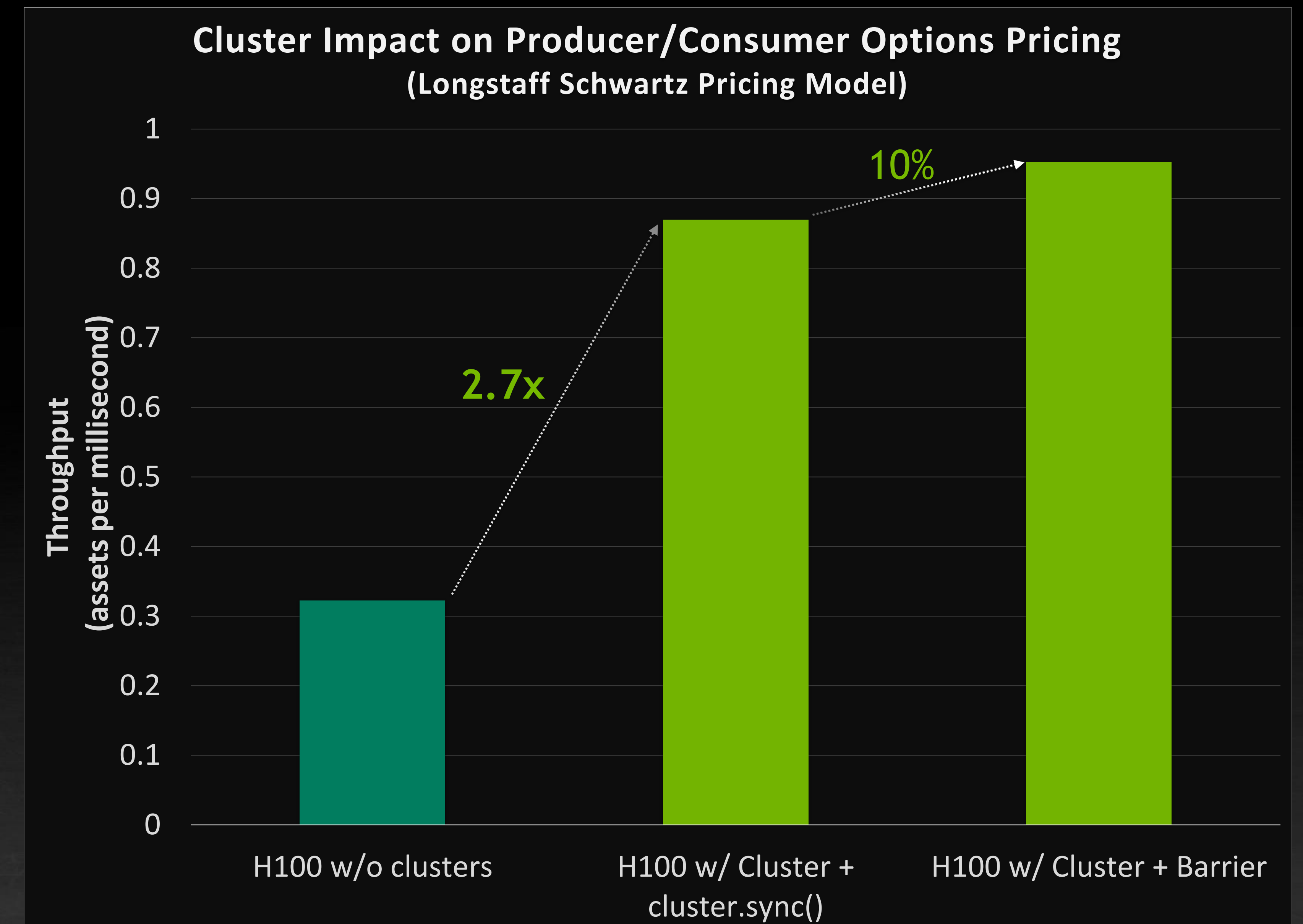
Monte-Carlo simulation with backward iteration to determine financial option values

Analysis of Example

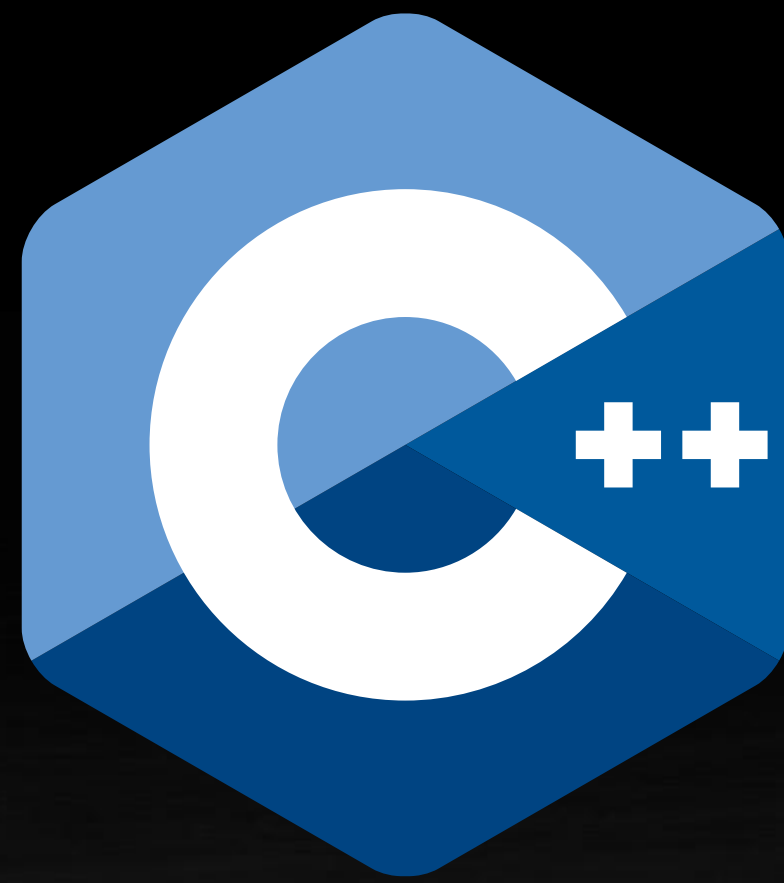
Larger capacity of **Distributed shared memory** enables new approach to programming efficient kernels

Thread Block level **Producer Consumer** models can be now **implemented in shared memory** with fast synchronization

Hardware accelerated cluster synchronization is easy to use and efficient



NVCC COMPILER: UPDATED C++ SUPPORT



C++20 supported in CUDA 12.0

C++20 supported as preview in host & device code

Enable with: `nvcc -std=c++20`

Supported host compilers

GCC 12.x, Clang 10, nvc++ 20.7, Visual Studio (2019 & 2022)

Still pending

Modules not supported

Coroutines not supported in device code

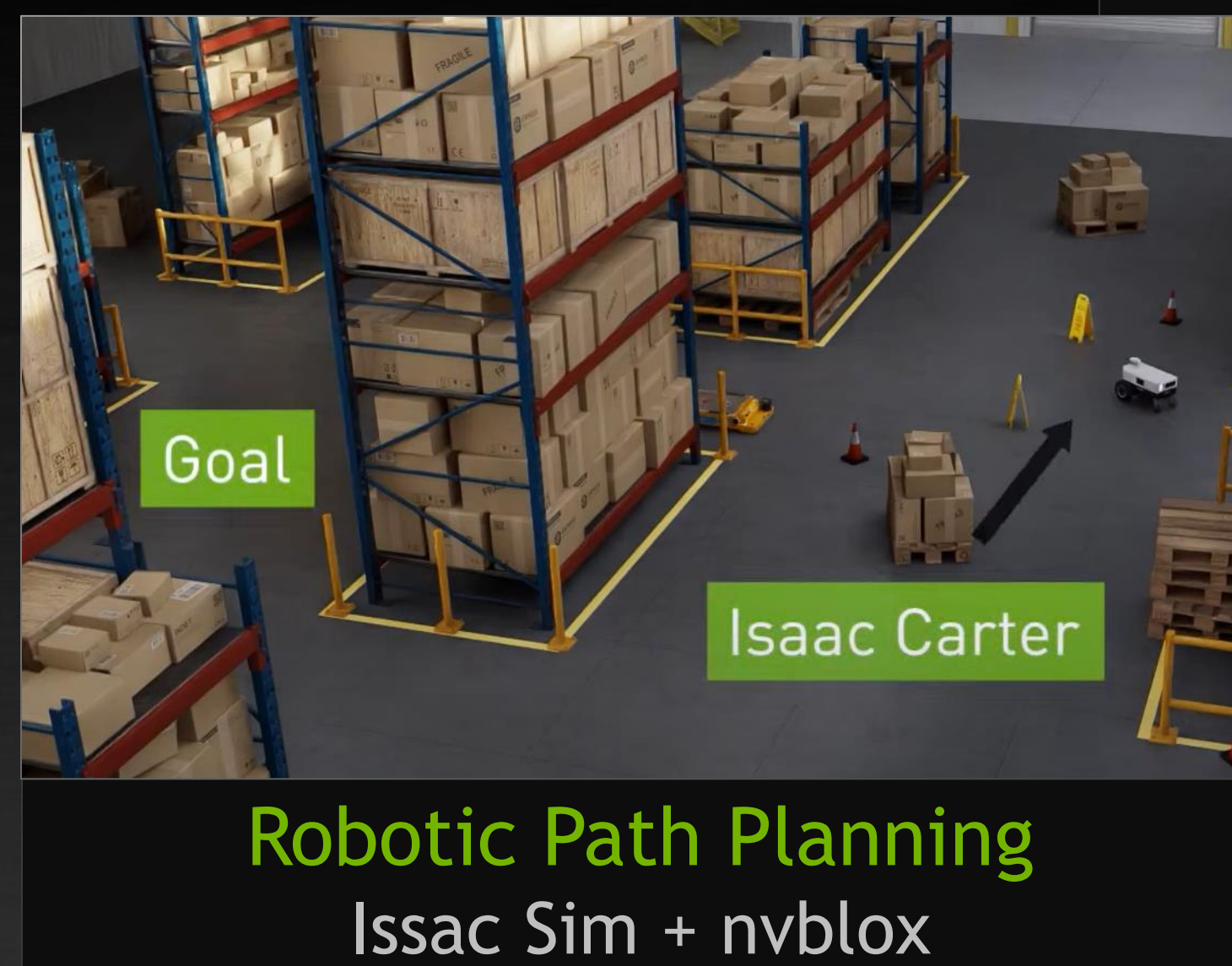
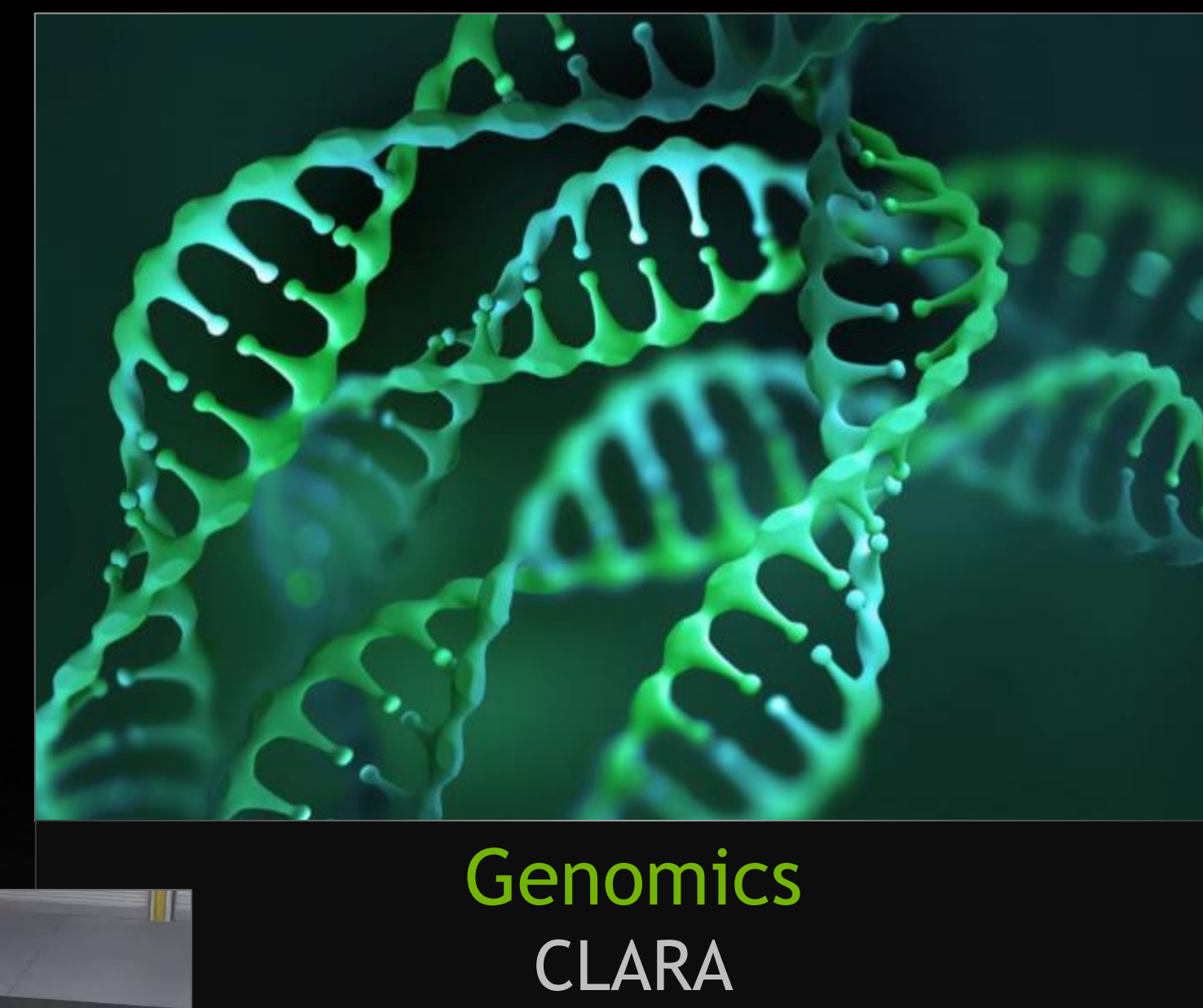
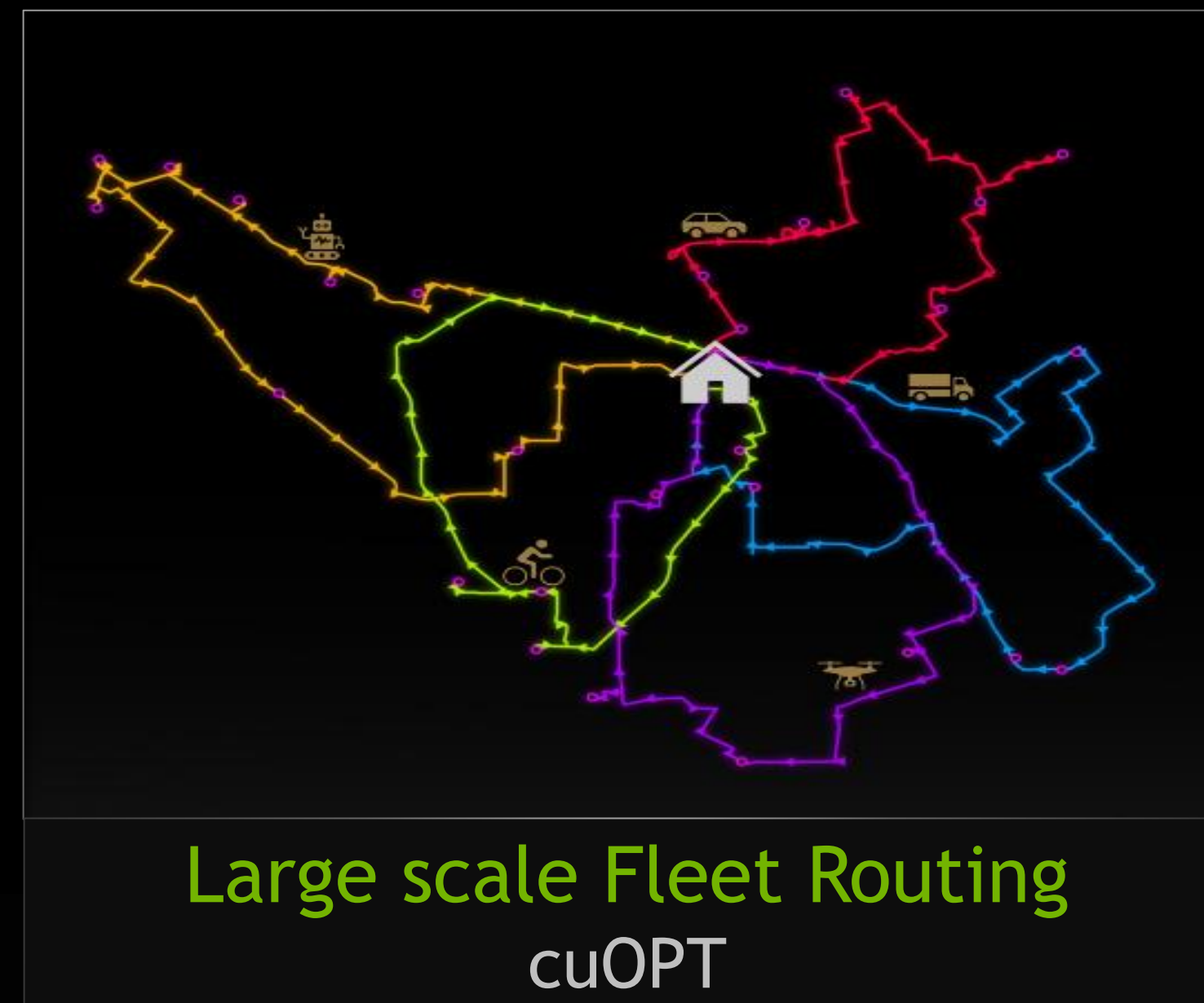
3-way comparator in device code requires flag: `--expt-relaxedconstexpr`

*(support for C++20 depends on host compiler standard compliance)

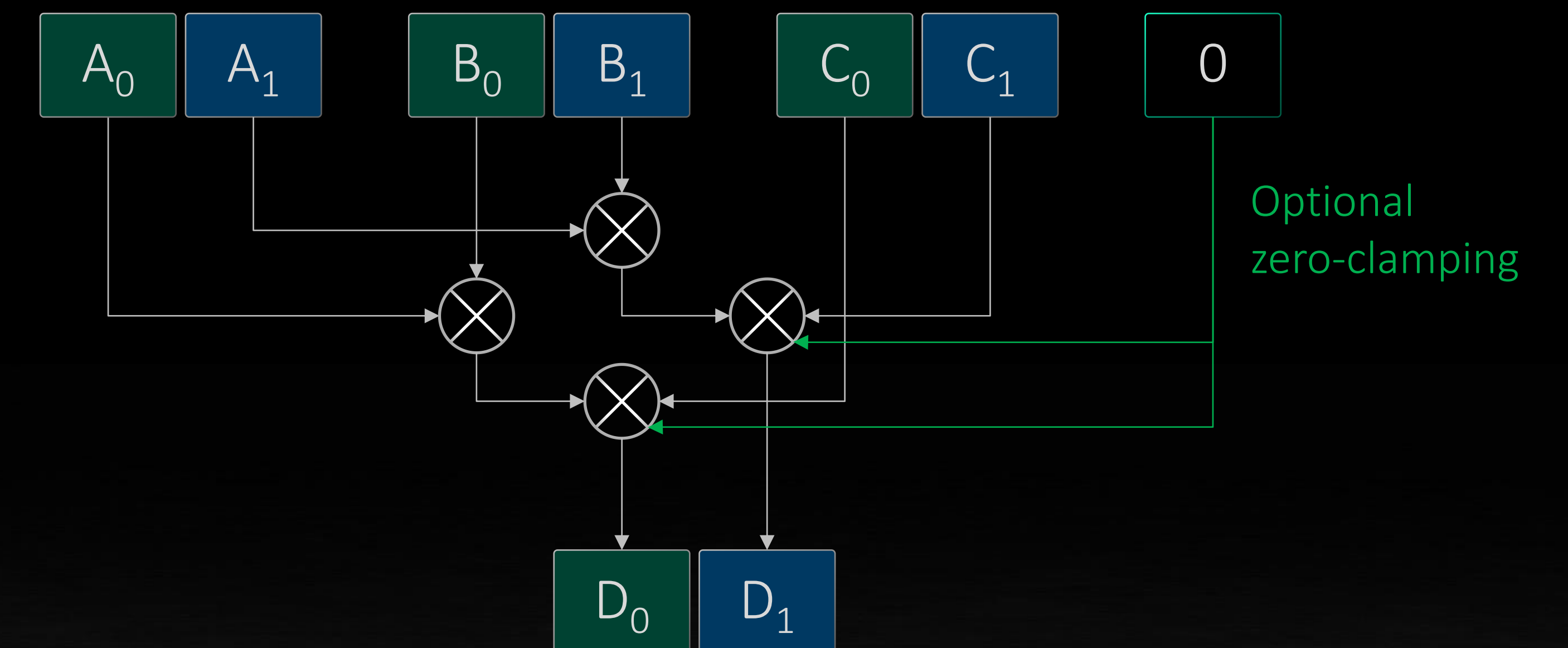
Default dialect for run-time compilation (NVRTC) is now C++17 (was C++03)

DYNAMIC PROGRAMMING

Recursive optimization of overlapping sub-problems



A, B, C, D datatypes are int32 / int16



DPX composite functions

$$D = \text{MAX}(A+B, C, 0)$$

$$D = \text{MAX}(A, B, C, 0)$$

EXAMPLE: GENOME SEQUENCE MATCHING

Smith-Waterman Algorithm

Basic cell computation

	G	T
G	6	4
A	4	3

DPX functions accelerate
building scoring matrix

Max of (0, one of below)

- +x (3) from diagonal element on match
- x (-3) from diagonal element on mismatch
- y (-2) from vertical element mismatch (insertion)
- y (-2) from horizontal element mismatch (deletion)

		T	G	T	T	A	C	G	G
	0	0	0	0	0	0	0	0	0
G	0	0	3	1	0	0	0	3	3
G	0	0	3	1	0	0	0	3	6
T	0	3	1	6	4	2	0	1	
T	0	3	1				5		
G	0	1							
A	0								
C	0								
T	0								
A	0								

Wavefront

EXAMPLE: GENOME SEQUENCE MATCHING

Smith-Waterman Algorithm

Basic cell computation

	G	T
G	6	4
A	4	3

DPX functions accelerate
building scoring matrix

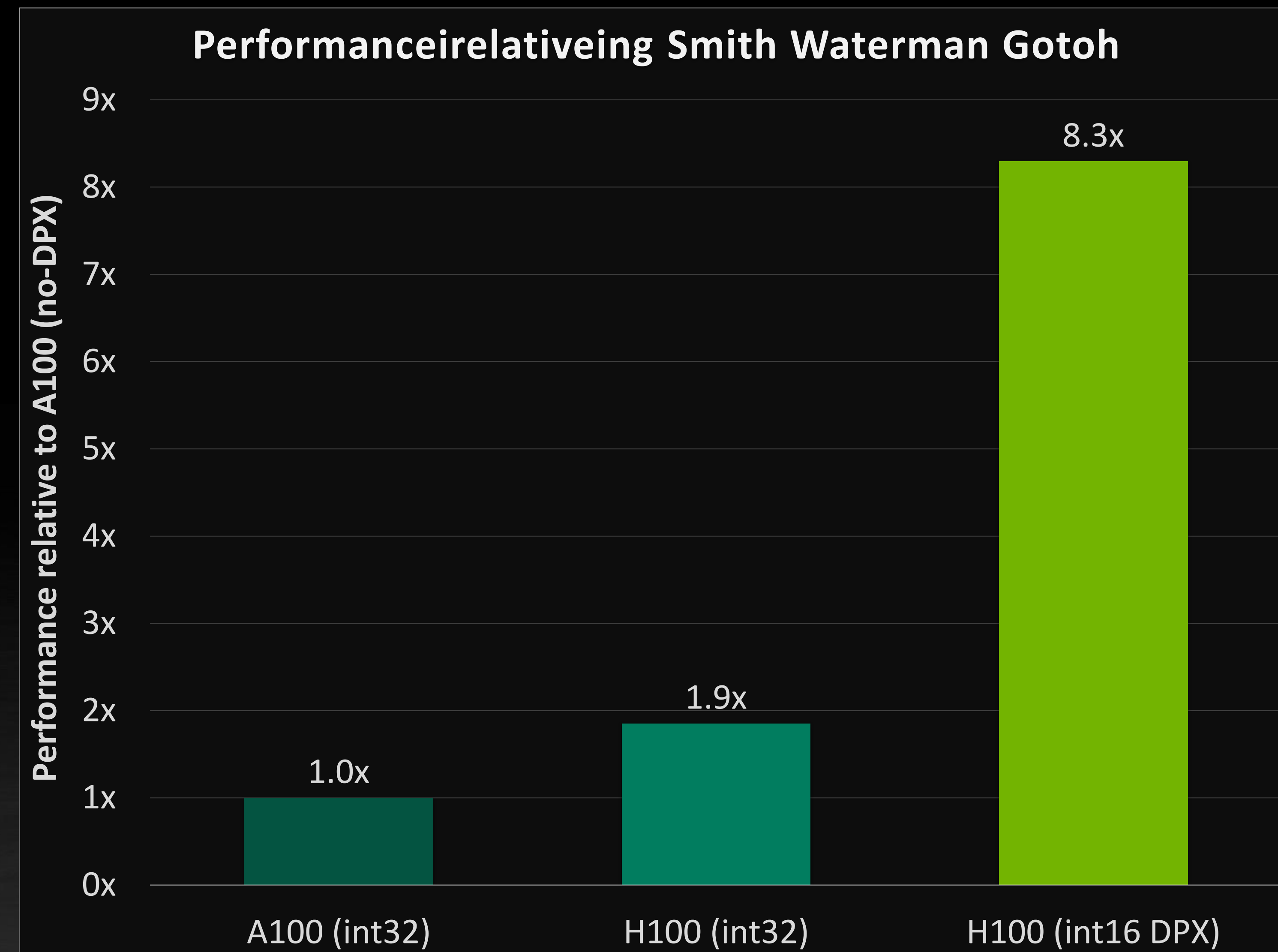
Max of (0, one of below)

- +x (3) from diagonal element on match
- x (-3) from diagonal element on mismatch
- y (-2) from vertical element mismatch (insertion)
- y (-2) from horizontal element mismatch (deletion)

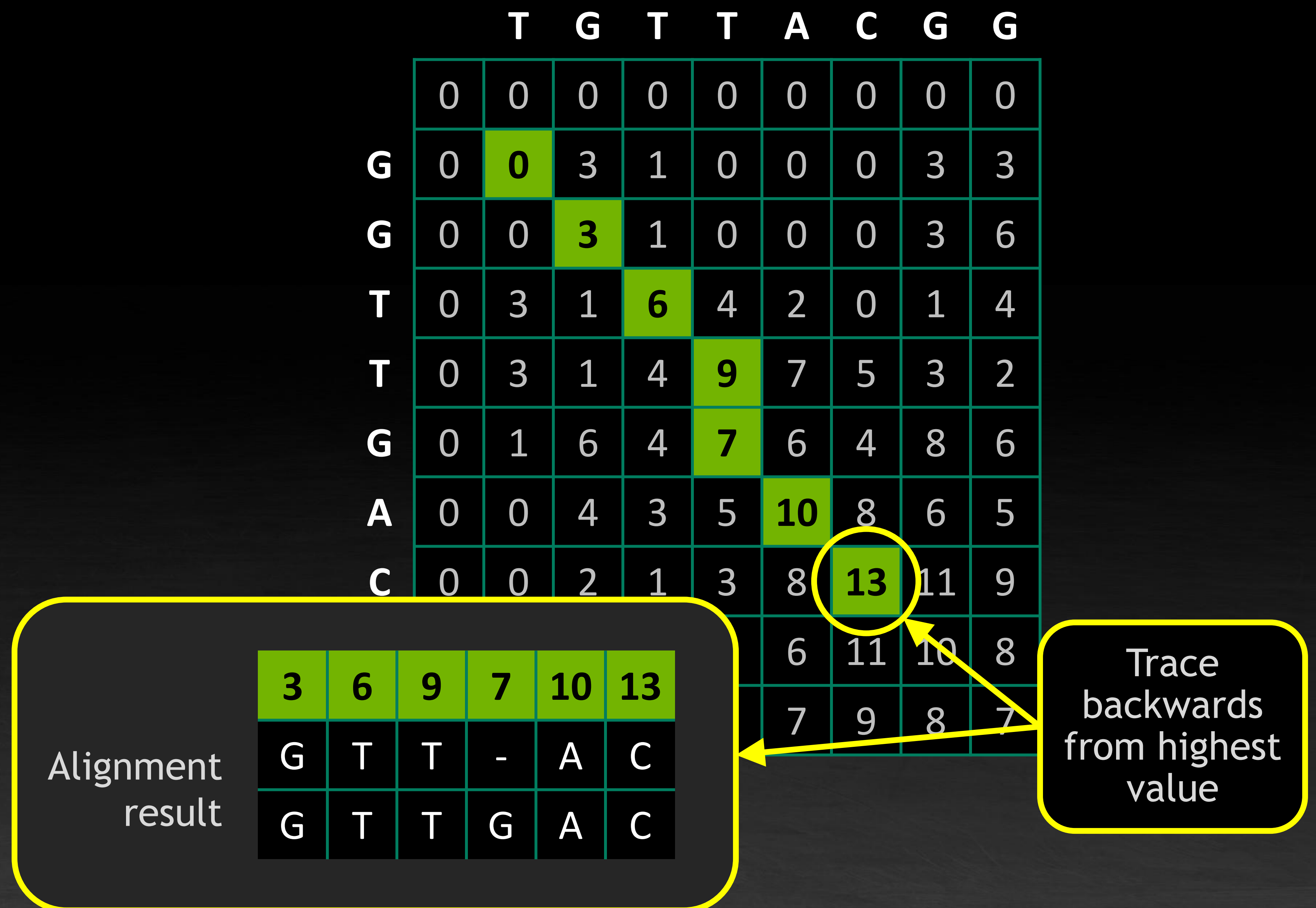
		T	G	T	T	A	C	G	G
	0	0	0	0	0	0	0	0	0
G	0	0	3	1	0	0	0	3	3
G	0	0	3	1	0	0	0	3	6
T	0	3	1	6	4	2	0	1	4
T	0	3	1	4	9	7	5	3	2
G	0	1	6	4	7	6	4	8	6
A	0	0	4	3	5	10	8	6	5
C	0	0	2	1	3	8	13	11	9
T	0	3	1	5	4	6	11	10	8
A	0	1	0	3	2	7	9	8	7

EXAMPLE: GENOME SEQUENCE MATCHING

Smith-Waterman Algorithm



Affine gap alignment (score calculation), weights used from BWA.
Data: HG002 (NA24385) pair-end protocol using Illumina Sequencers.



CUDA DYNAMIC PARALLELISM HELLO WORLD

hello.cu

CPU portion

```
void main() {  
    hello<<< 1, 1 >>>();  
    cudaDeviceSynchronize();  
}
```

GPU portion

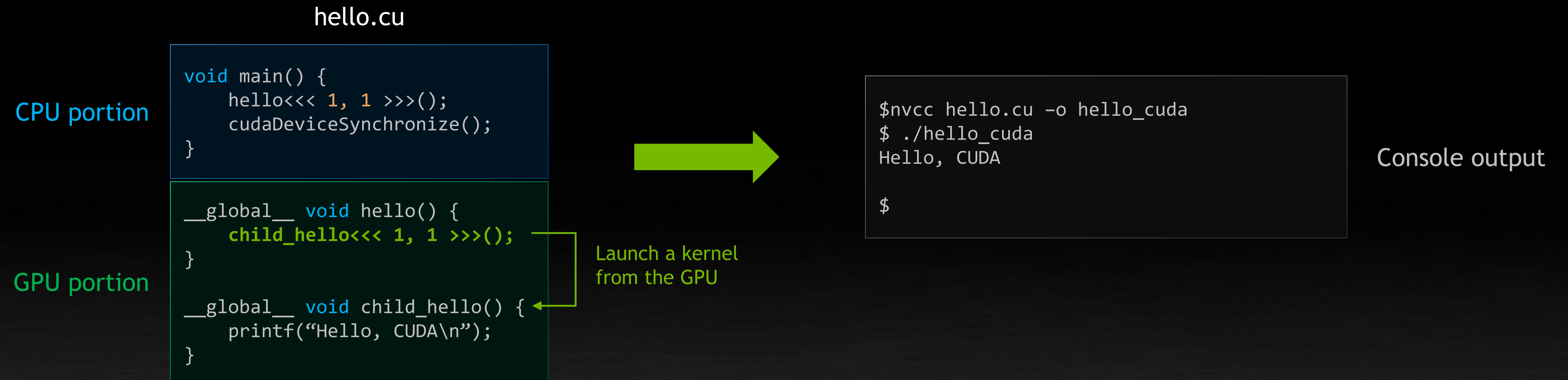
```
__global__ void hello() {  
    printf("Hello, CUDA\n");  
}
```



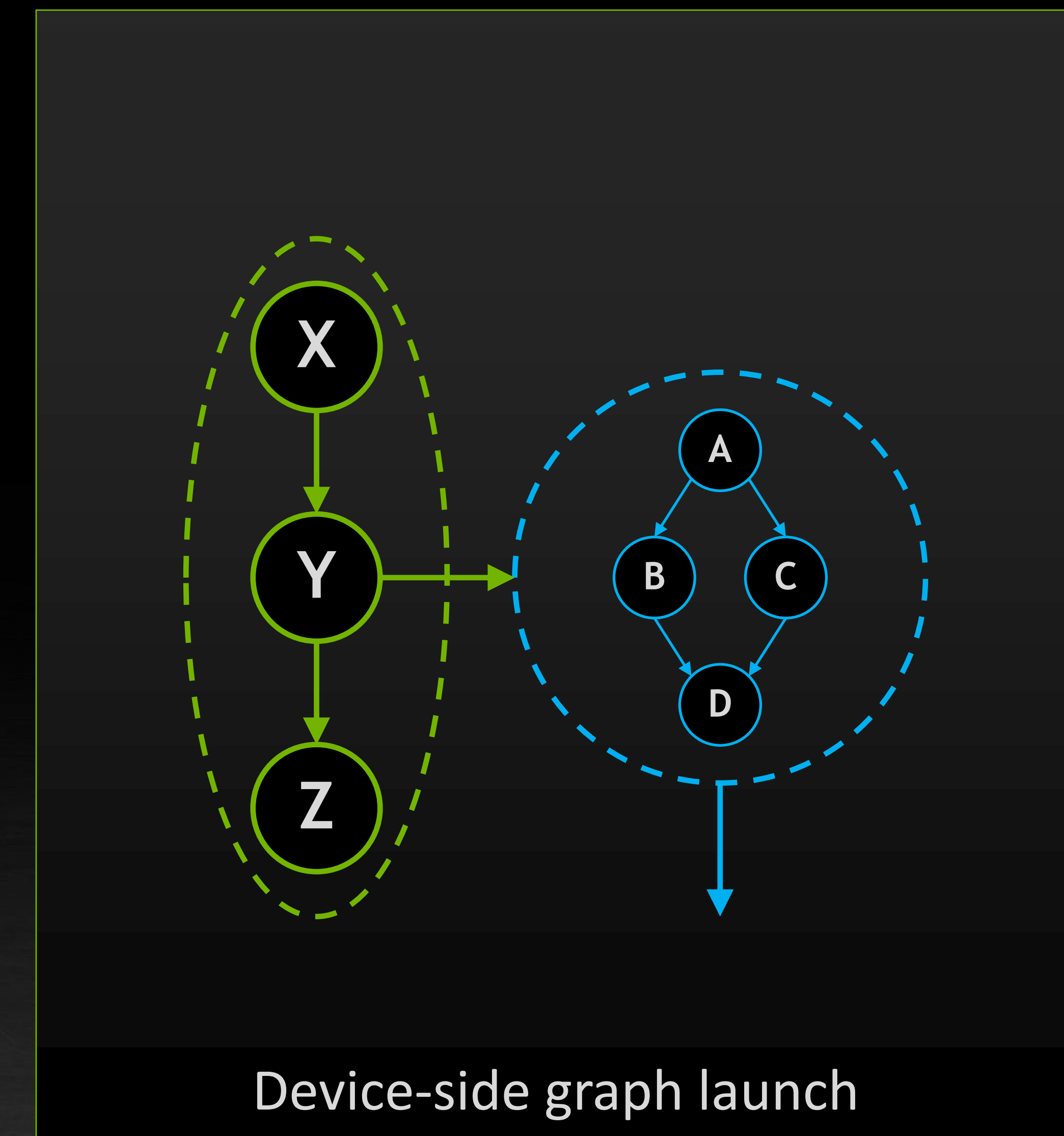
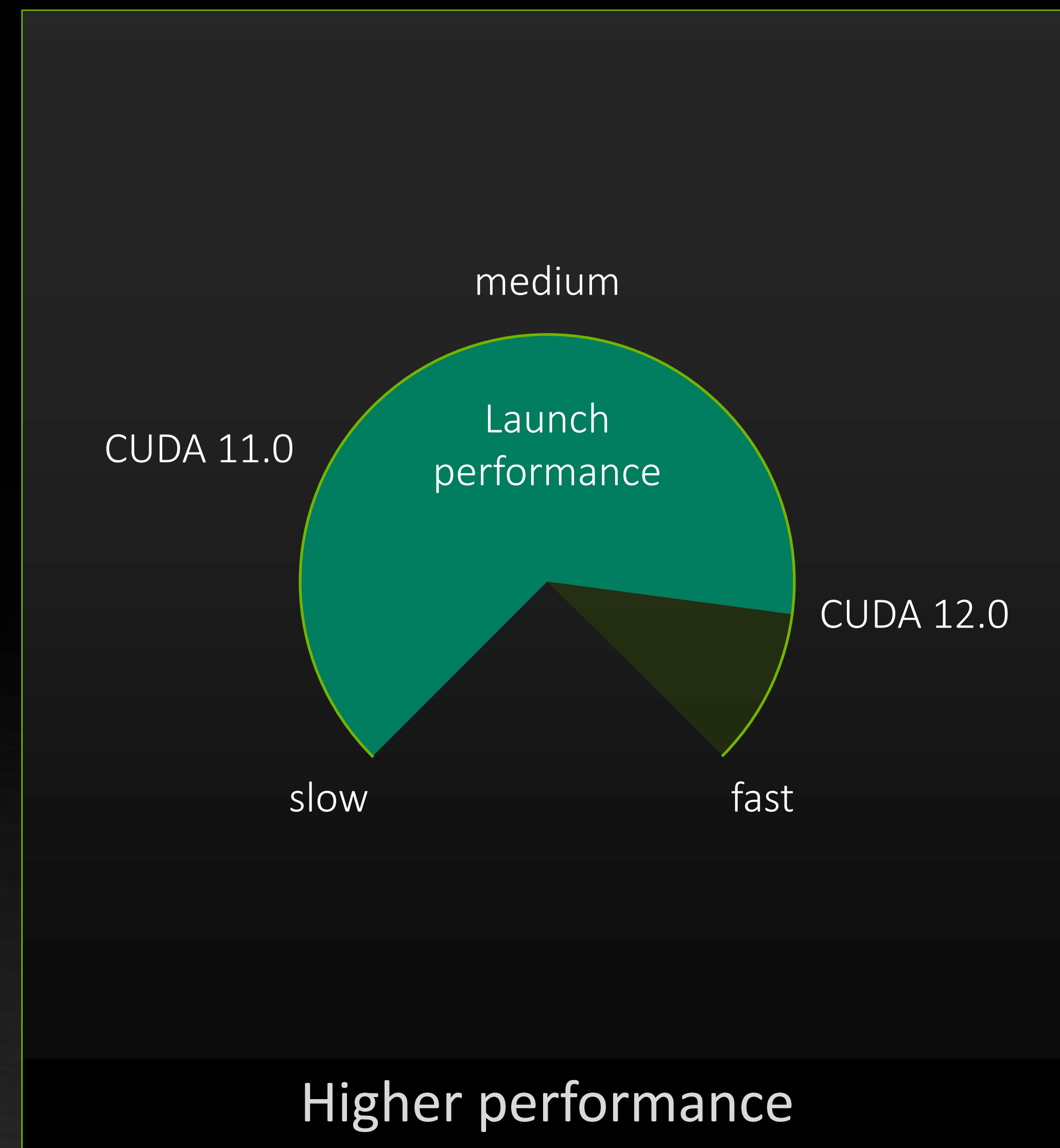
```
$nvcc hello.cu -o hello_cuda  
$ ./hello_cuda  
Hello, CUDA  
  
$
```

Console output

CUDA DYNAMIC PARALLELISM HELLO WORLD



ENHANCING DYNAMIC PARALLELISM

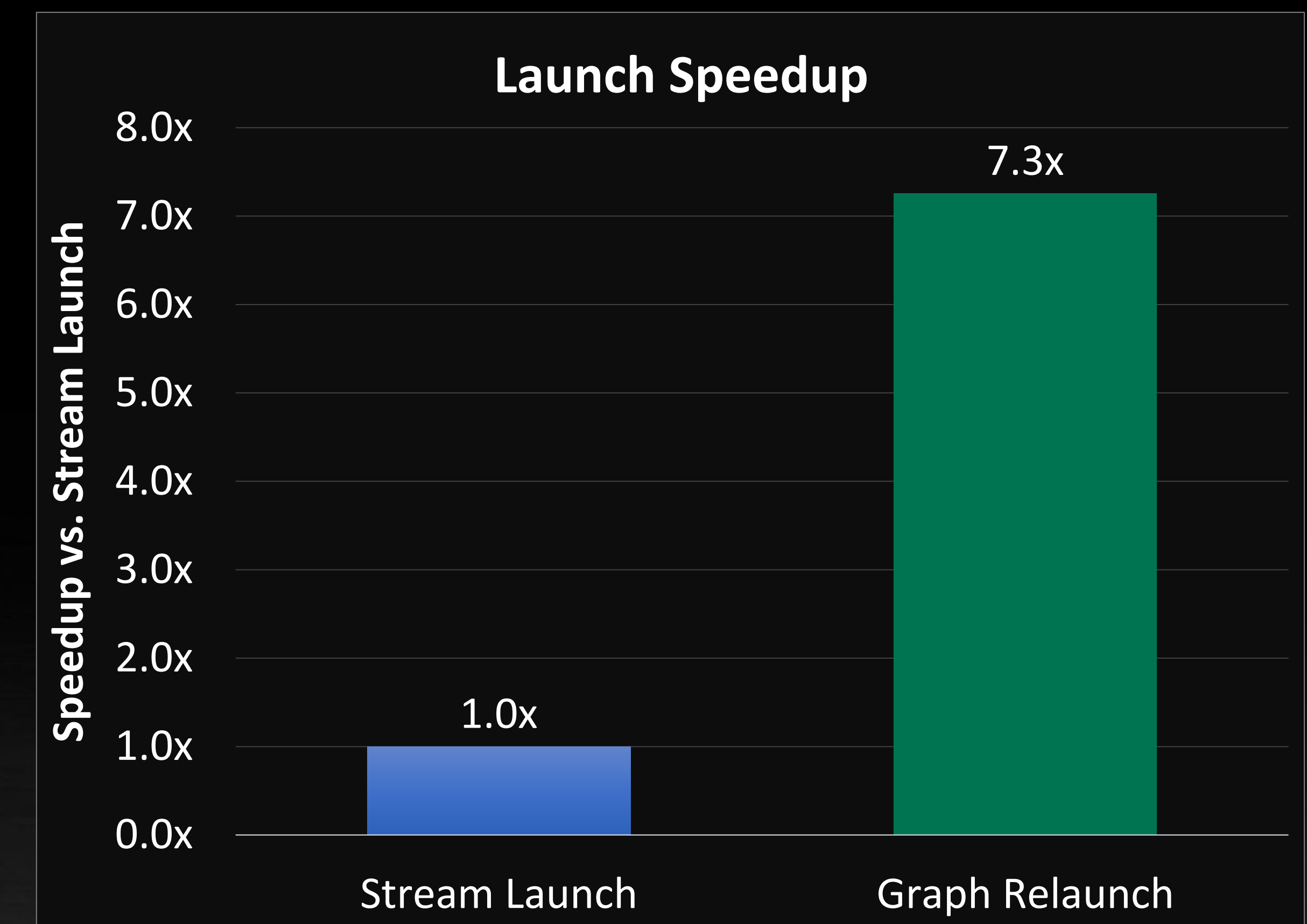
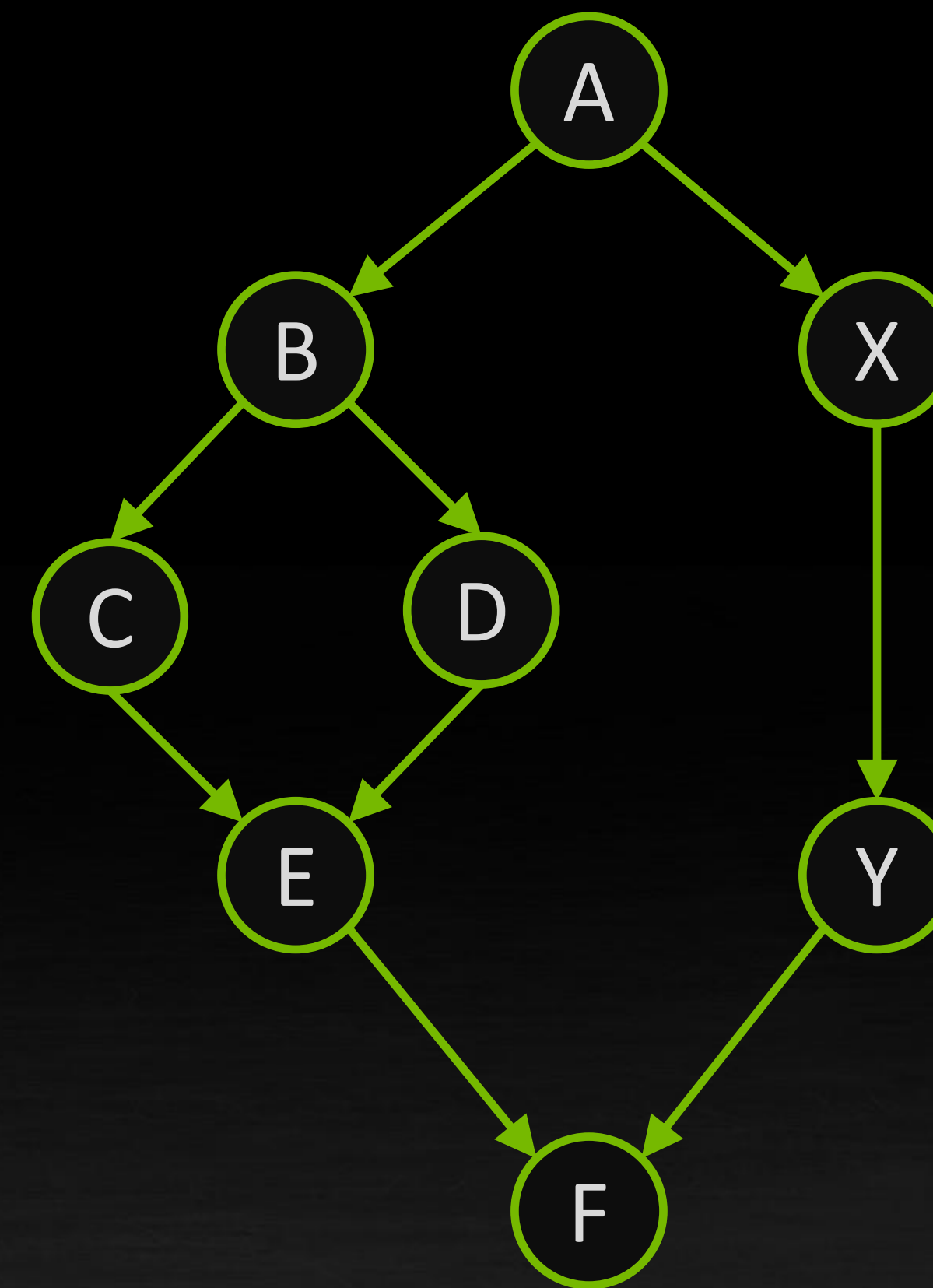


TASK GRAPH BASICS

Graphs can be generated once and then launched repeatedly

```
// Build graph ABCDEFXY once
cudaGraphCreate(&graph);
...
cudaGraphInstantiate(graph);

// Launch rapidly many times
for(int i=0; i<1000; i++) {
    launch_graph(graph);
}
```



Launch of an already-created graph is **7-8x faster** than launching the same kernels into a stream

IMPROVED CUDA GRAPHS TRACING IN NSIGHT COMPUTE

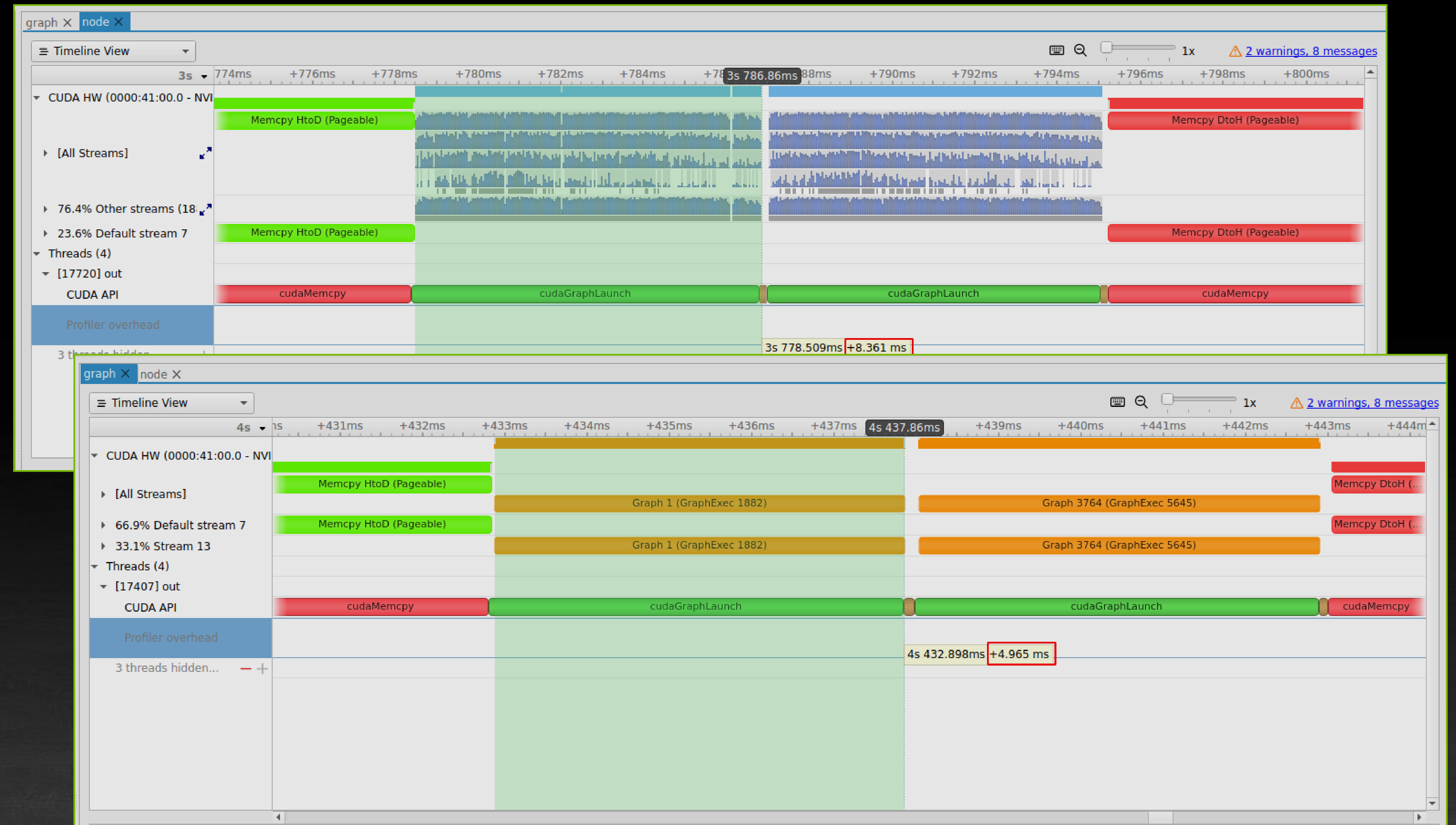
--cuda-graph-trace=graph|node

Identify CUDA Graphs on timeline

Locate underperforming individual graph instances

Reduced overhead for complex graph structures

Ability to profile in more detail if specific graphs are identified as bottleneck



DYNAMIC PARALLEL TASK GRAPHS

cdp_graphs.cu

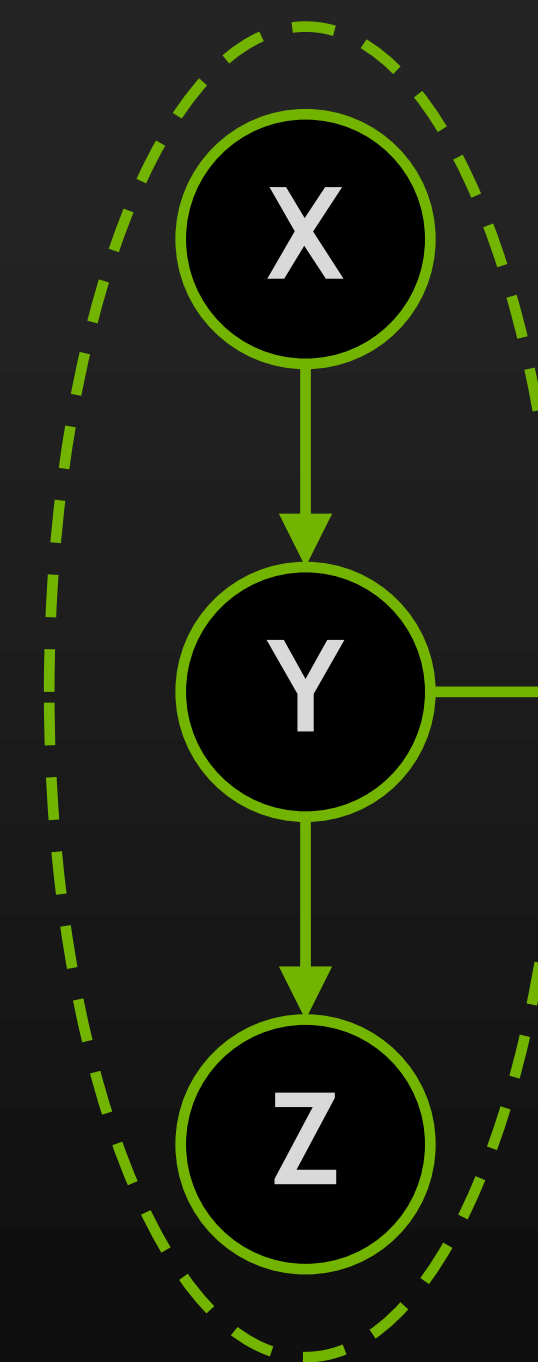
CPU portion

```
void main() {  
    cudaGraphCreate(&G1);  
    // Build graph G1 = XYZ  
    cudaGraphInstantiate(G1);  
  
    cudaGraphCreate(&G2);  
    // Build graph G2 = ABCD  
    cudaGraphInstantiate(G2, DeviceLaunch);  
  
    cudaGraphLaunch(G1, ...);  
}
```

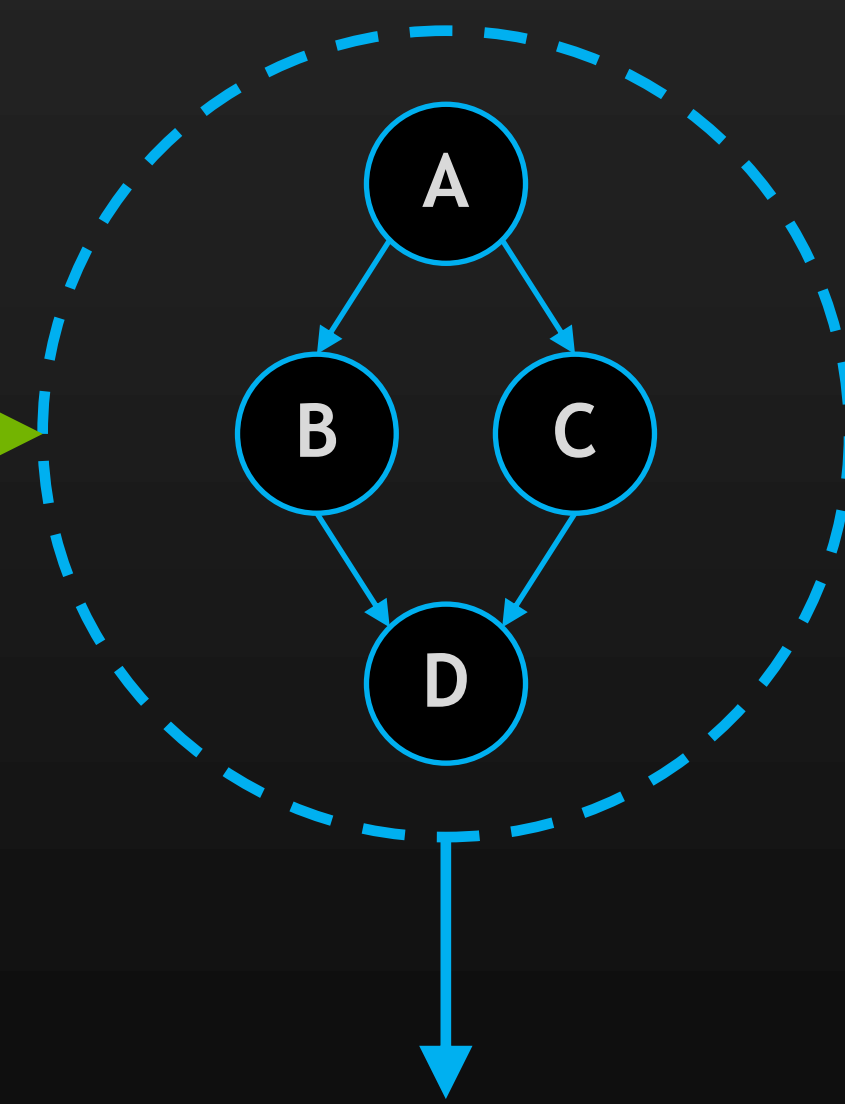
GPU portion

```
__global__ void Y(cudaDeviceGraph_t G2) {  
    cudaGraphLaunch(G2, ...);  
}
```

Graph G1

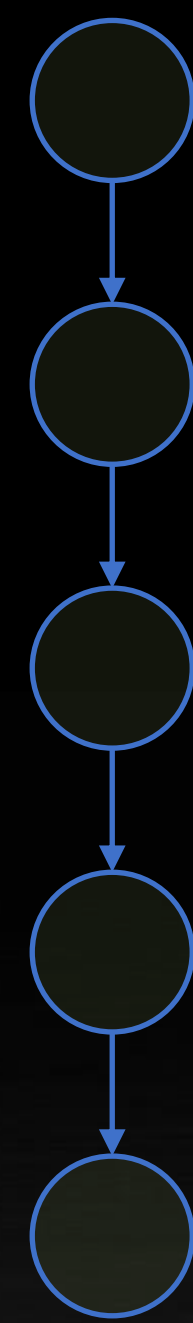


Graph G2

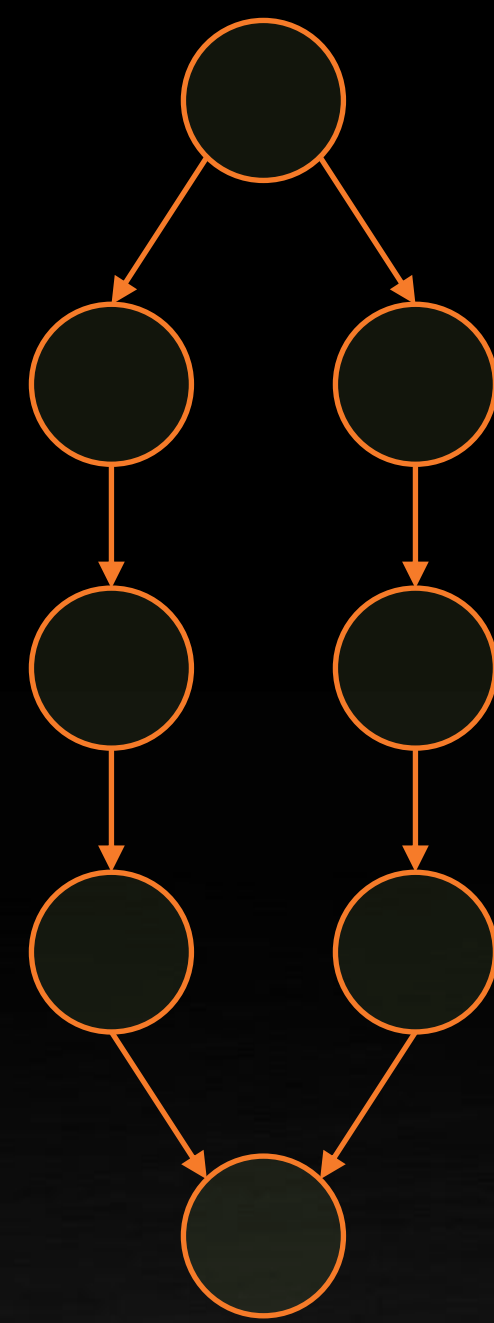


Device-side graph launch

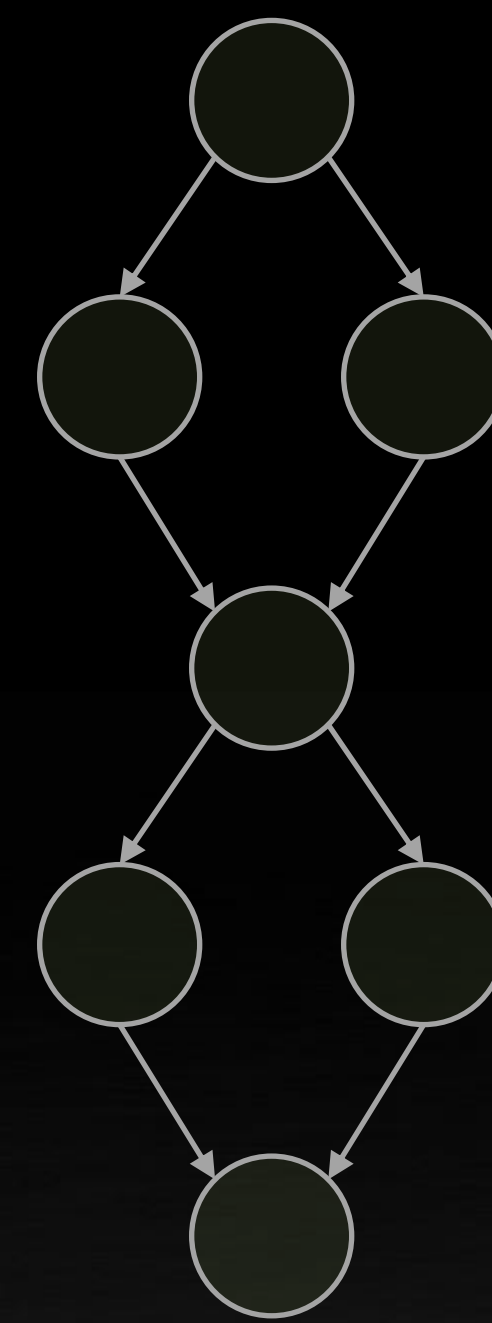
EXAMPLE: RUN-TIME DYNAMIC WORK SCHEDULING



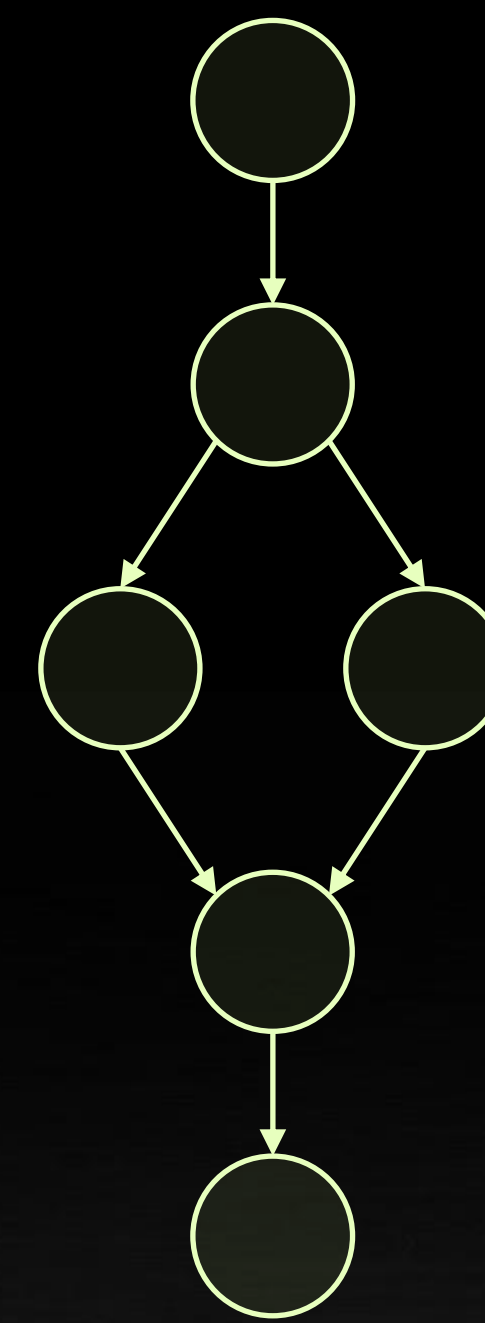
G1



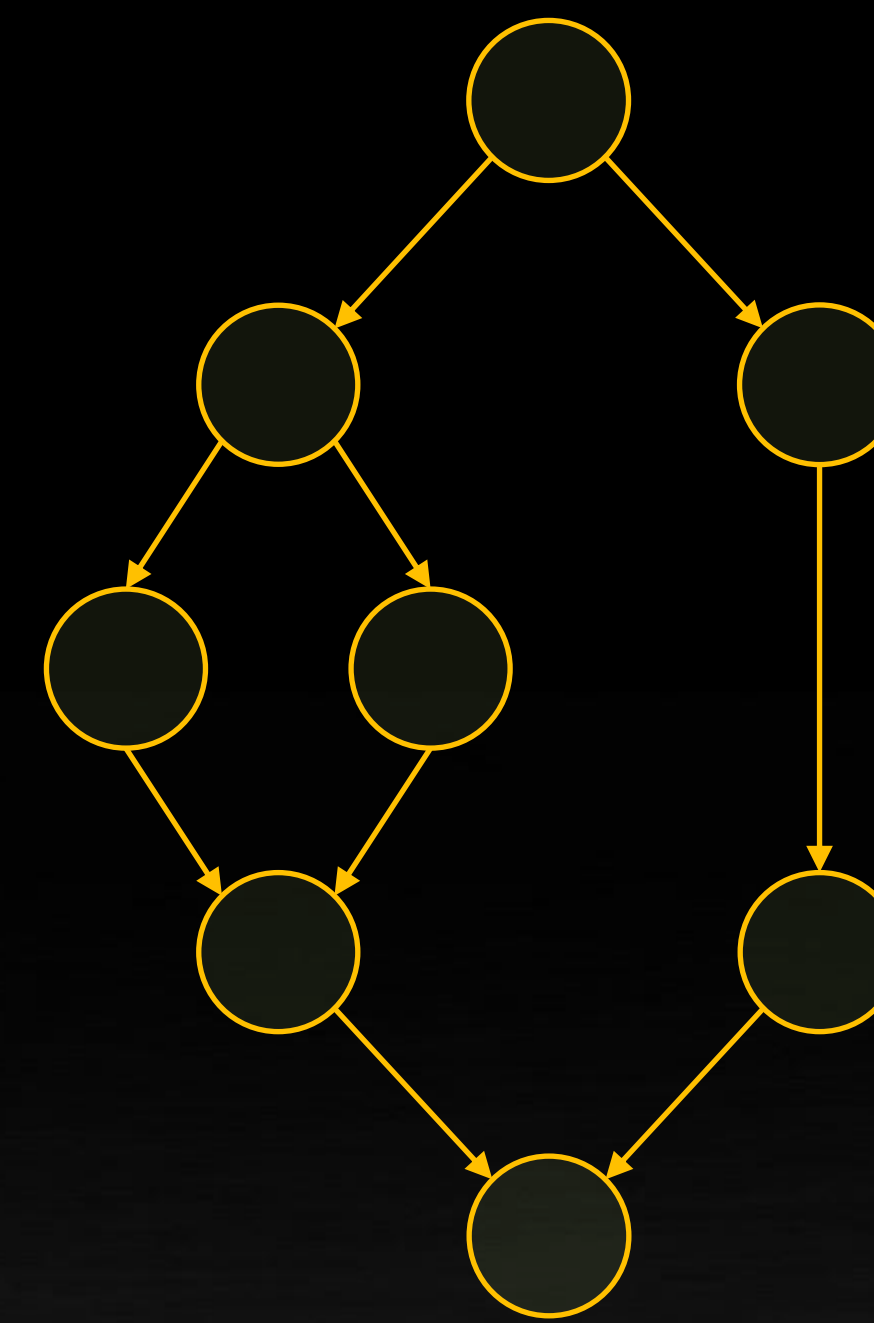
G2



G3



G4

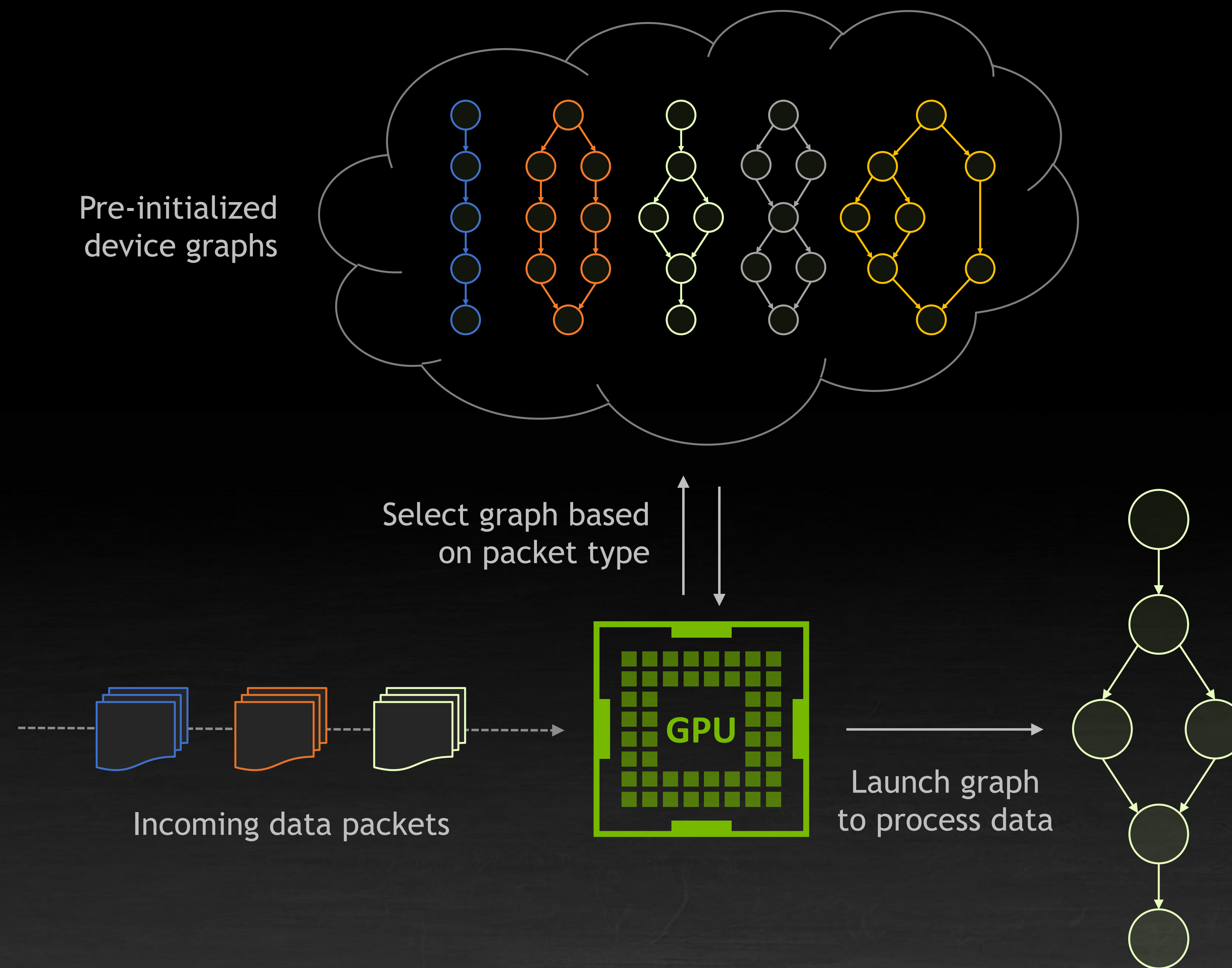


G5

```
void init() {  
    cudaGraphCreate(G1);  
    ...           // Set up graph G1  
  
    cudaGraphCreate(G2);  
    ...           // Set up graph G2  
  
    cudaGraphCreate(G3);  
    ...           // Set up graph G3  
  
    cudaGraphCreate(G4);  
    ...           // Set up graph G4  
  
    cudaGraphCreate(G5);  
    ...           // Set up graph G5  
}
```

Create multiple graphs in host code
during program init

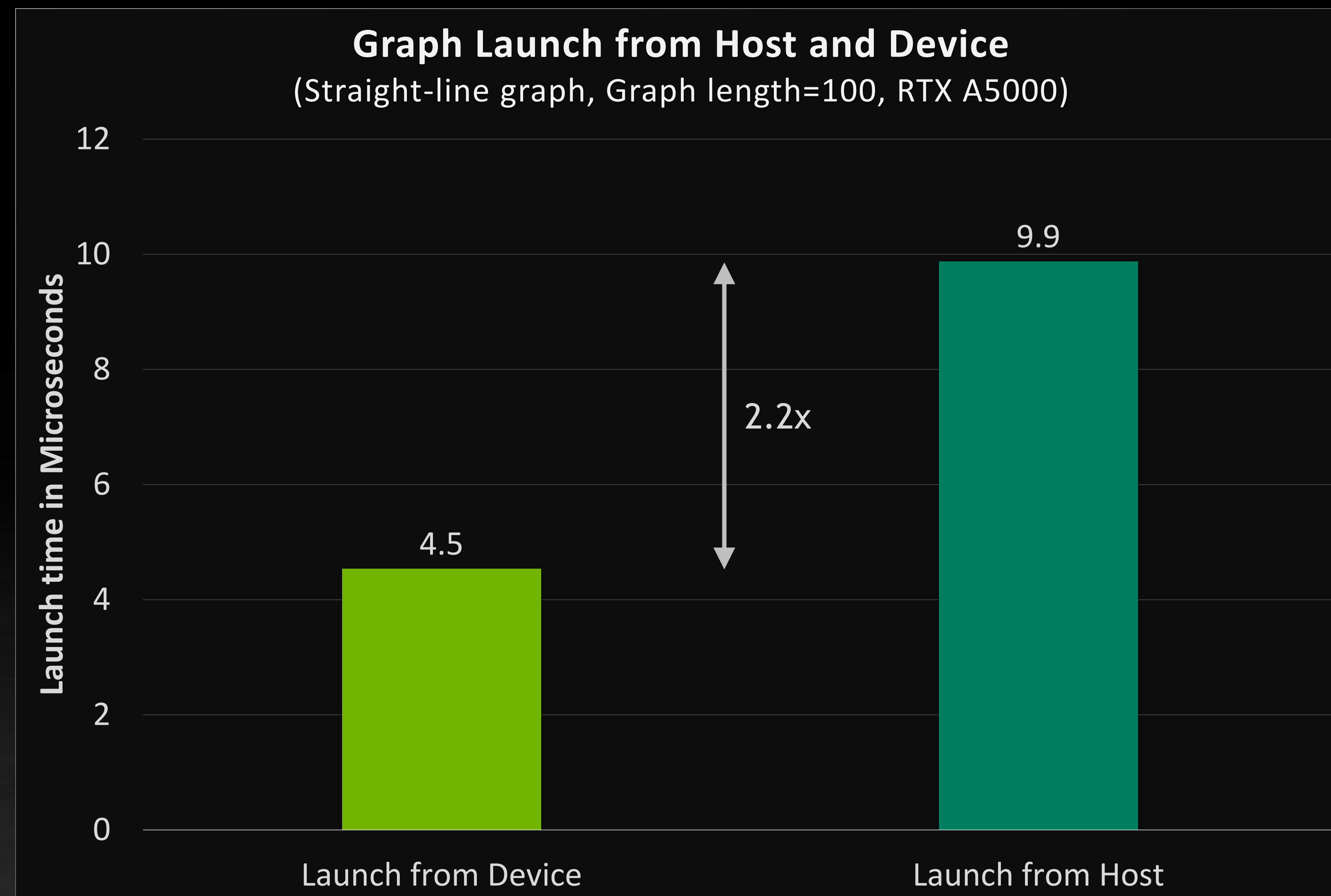
EXAMPLE: RUN-TIME DYNAMIC WORK SCHEDULING



```
__global__ void scheduler(...) {  
    Packet data = receivePacket(...);  
  
    switch(data.type) {  
        case 1:  
            cudaGraphLaunch(G1, ...);  
            break;  
        case 2:  
            cudaGraphLaunch(G2, ...);  
            break;  
        case 3:  
            cudaGraphLaunch(G3, ...);  
            break;  
        case 4:  
            cudaGraphLaunch(G4, ...);  
            break;  
        case 5:  
            cudaGraphLaunch(G5, ...);  
            break;  
    }  
  
    // Re-launch the scheduler to run after processing  
    cudaGraphLaunch(scheduler, ...);  
}
```

Scheduler kernel executing on device

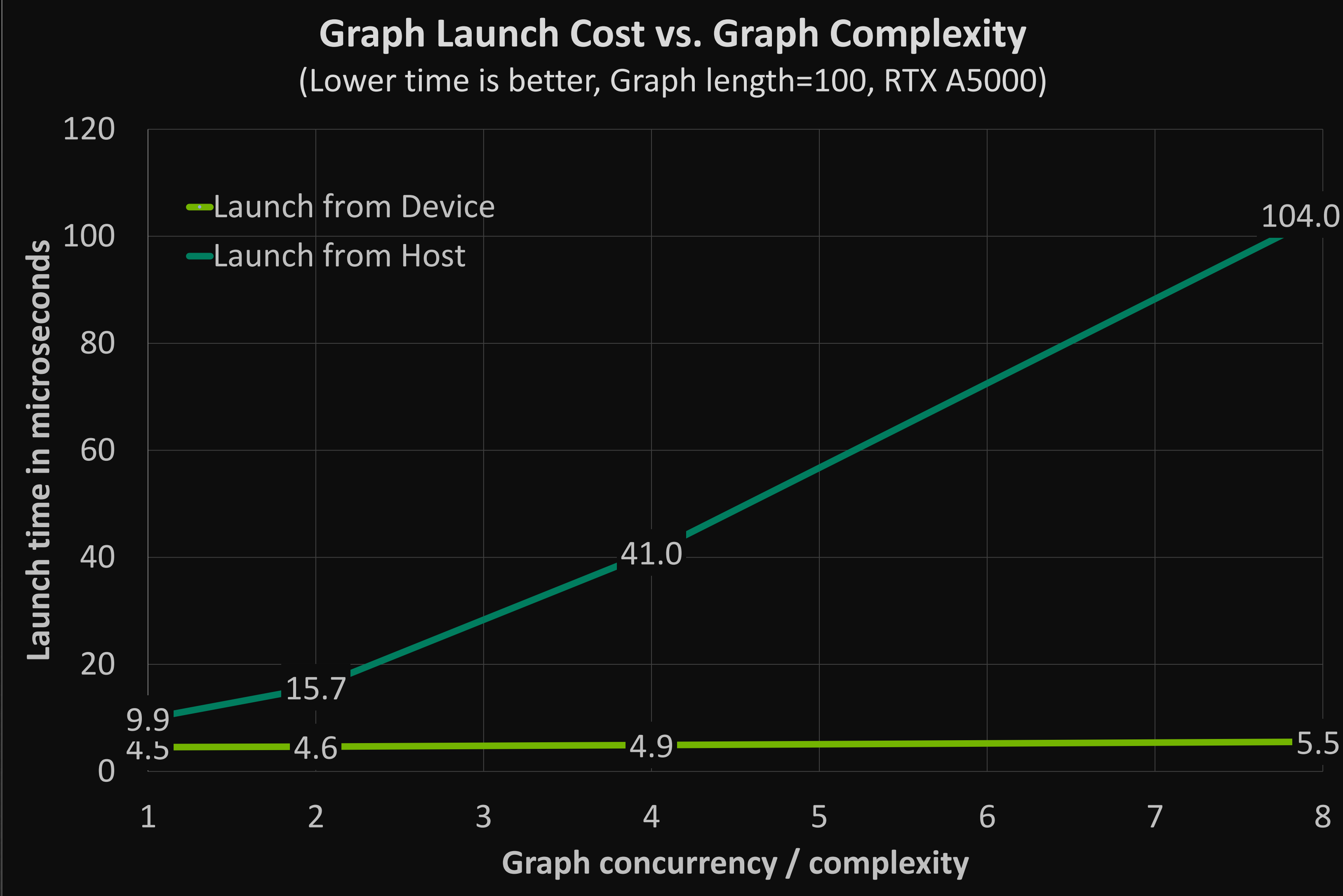
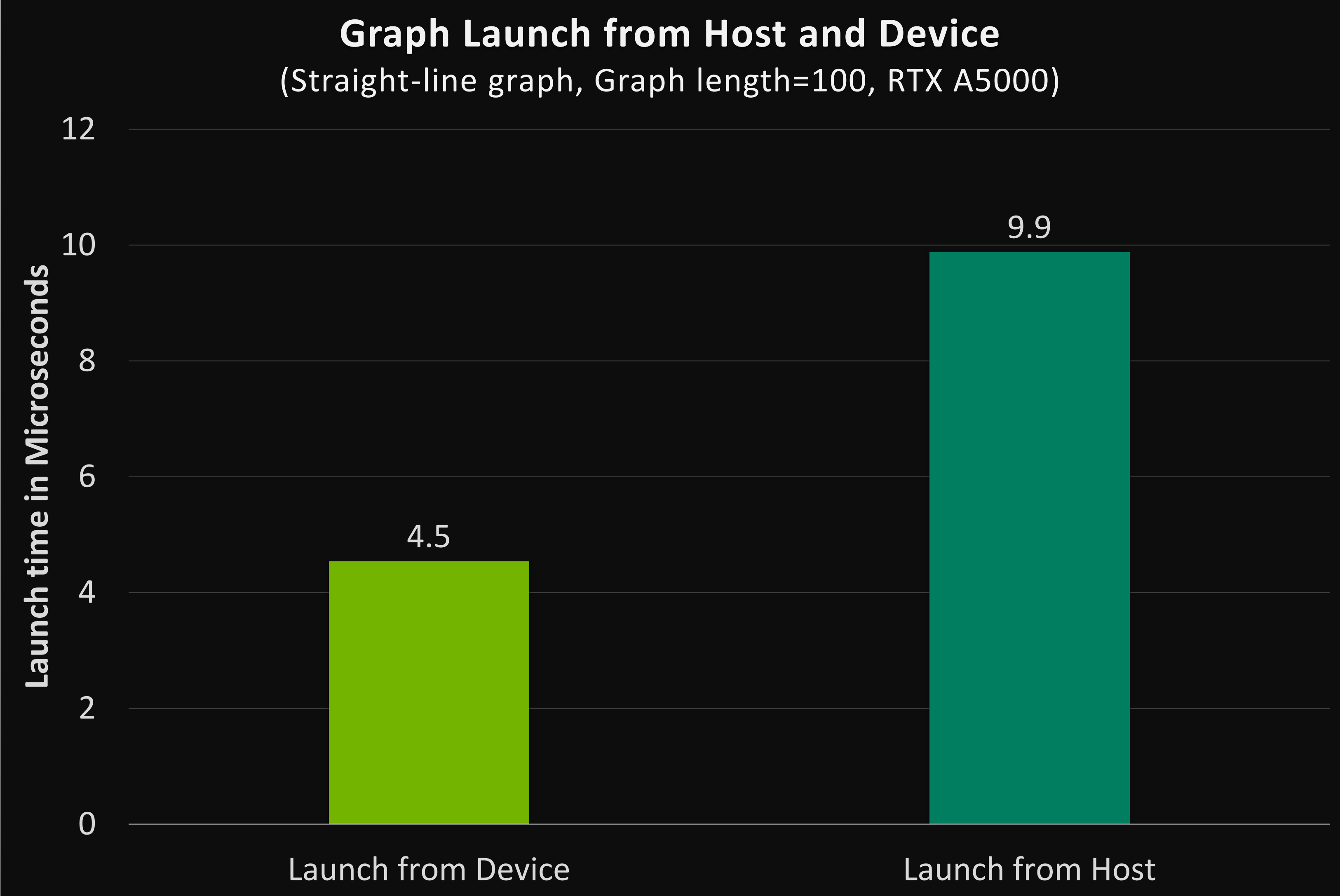
THE DEVICE-LAUNCH ADVANTAGE



```
__global__ void scheduler(...) {  
    Packet data = receivePacket(...);  
  
    switch(data.type) {  
        case 1:  
            cudaGraphLaunch(G1, ...);  
            break;  
        case 2:  
            cudaGraphLaunch(G2, ...);  
            break;  
        case 3:  
            cudaGraphLaunch(G3, ...);  
            break;  
        case 4:  
            cudaGraphLaunch(G4, ...);  
            break;  
        case 5:  
            cudaGraphLaunch(G5, ...);  
            break;  
    }  
  
    // Re-launch the scheduler to run after processing  
    cudaGraphLaunch(scheduler, ...);  
}
```

Scheduler kernel executing on device

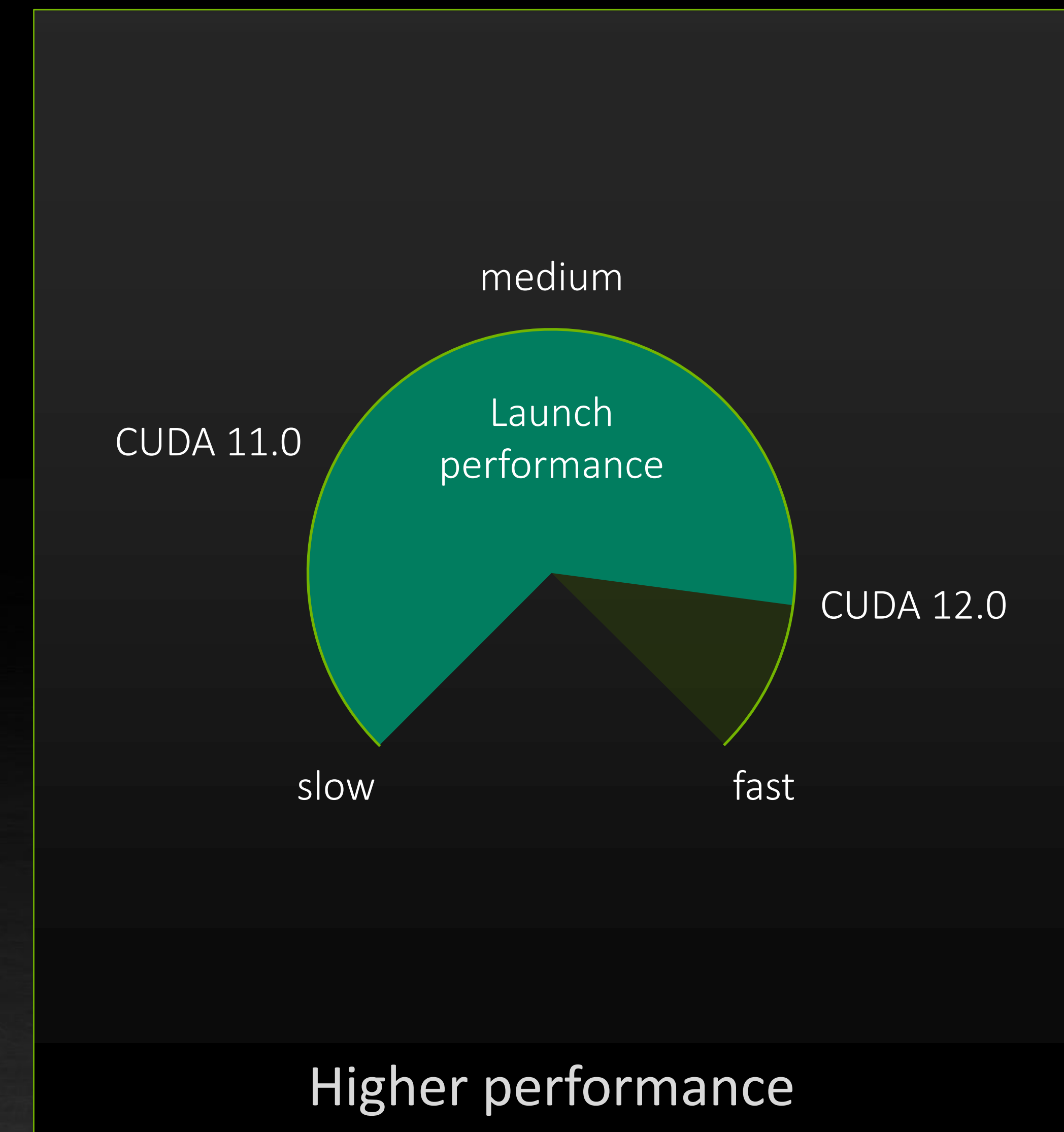
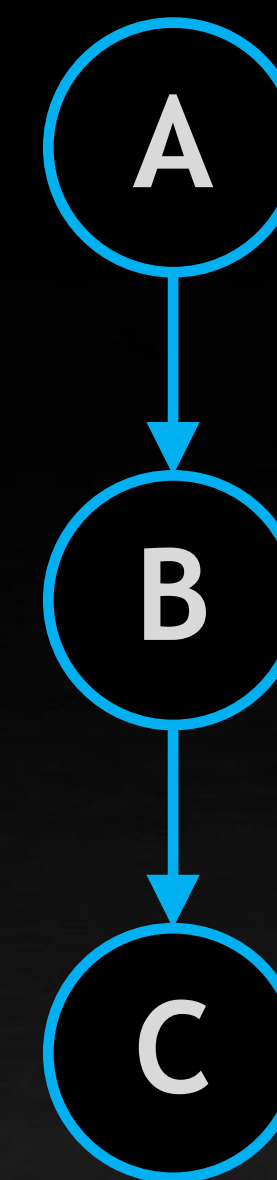
THE DEVICE-LAUNCH ADVANTAGE



DYNAMIC PARALLELISM PROGRAMMING MODEL: ENCAPSULATION

```
void main() {  
    cudaStream_t cpu_stream;  
    cudaStreamCreate(&cpu_stream);  
  
    A <<< ..., cpu_stream >>>();  
    B <<< ..., cpu_stream >>>();  
    C <<< ..., cpu_stream >>>();  
  
    cudaStreamSynchronize(cpu_stream);  
}
```

cpu_stream



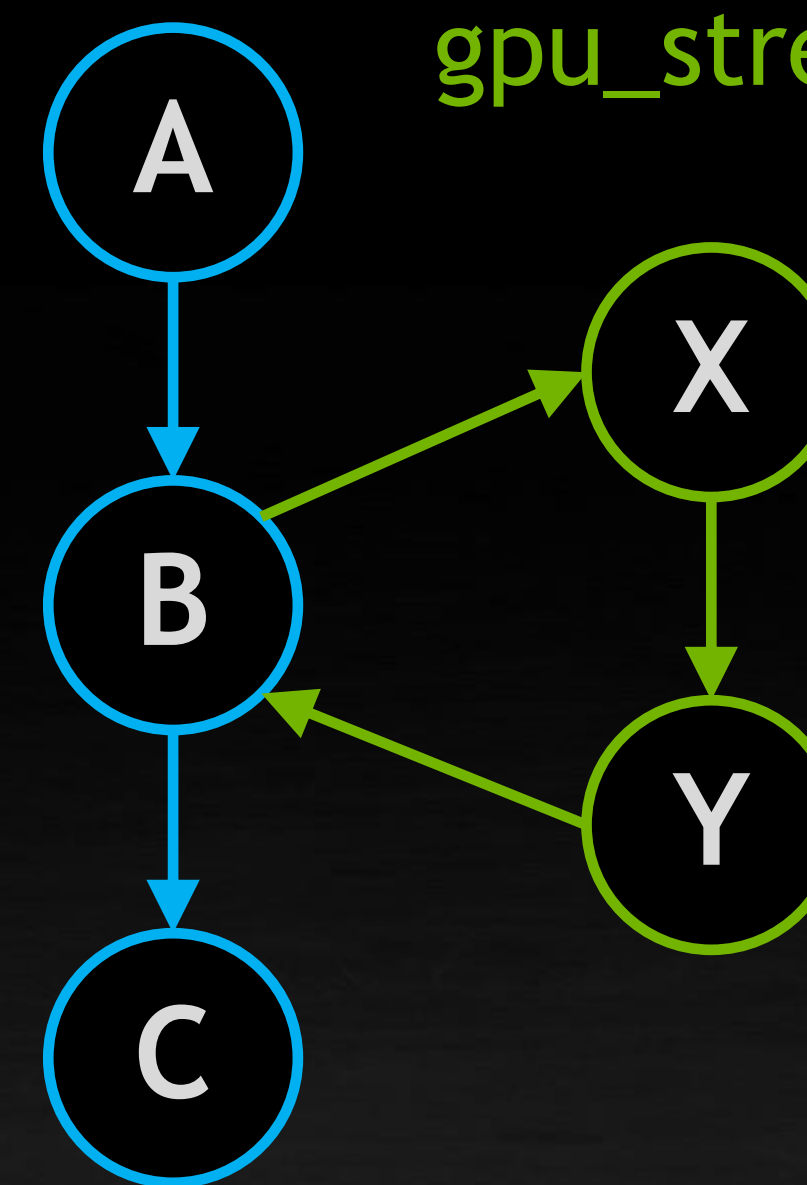
DYNAMIC PARALLELISM PROGRAMMING MODEL: ENCAPSULATION

```
void main() {  
    cudaStream_t cpu_stream;  
    cudaStreamCreate(&cpu_stream);  
  
    A <<< ..., cpu_stream >>>();  
    B <<< ..., cpu_stream >>>();  
    C <<< ..., cpu_stream >>>();  
  
    cudaStreamSynchronize(cpu_stream);  
}
```

```
__global__ void B() {  
    cudaStream_t gpu_stream;  
    cudaStreamCreateWithFlags(&gpu_stream,  
                             cudaStreamNonBlocking);  
  
    X <<< ..., gpu_stream >>>();  
    Y <<< ..., gpu_stream >>>();  
  
    cudaDeviceSynchronize();  
    do_something();  
}
```

cpu_stream

gpu_stream

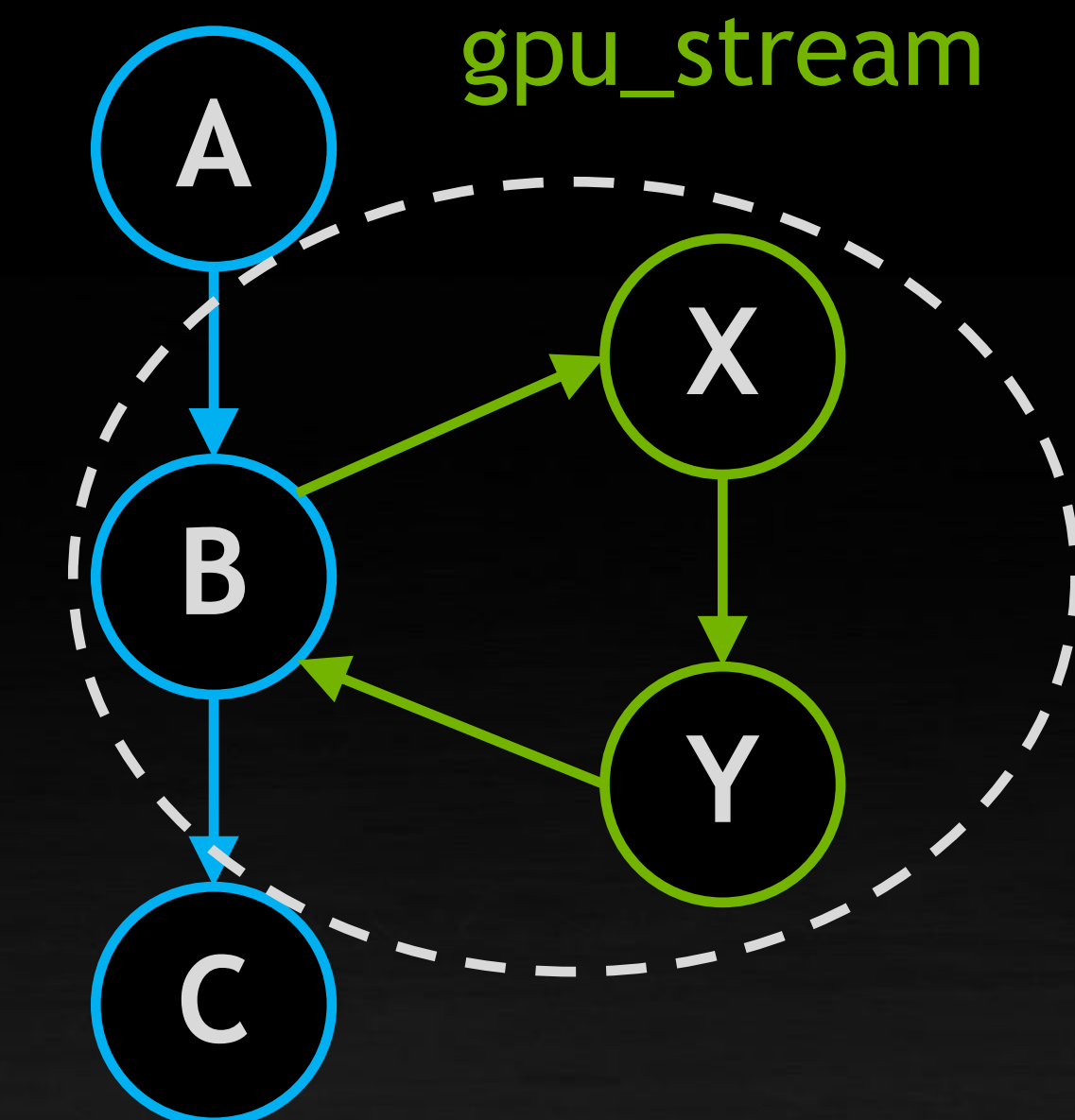


DYNAMIC PARALLELISM PROGRAMMING MODEL: ENCAPSULATION

```
void main() {  
    cudaStream_t cpu_stream;  
    cudaStreamCreate(&cpu_stream);  
  
    A <<< ..., cpu_stream >>>();  
    B <<< ..., cpu_stream >>>();  
    C <<< ..., cpu_stream >>>();  
  
    cudaStreamSynchronize(cpu_stream);  
}
```

```
__global__ void B() {  
    cudaStream_t gpu_stream;  
    cudaStreamCreateWithFlags(&gpu_stream,  
                             cudaStreamNonBlocking);  
  
    X <<< ..., gpu_stream >>>();  
    Y <<< ..., gpu_stream >>>();  
  
    cudaDeviceSynchronize();  
    do_something();  
}
```

cpu_stream



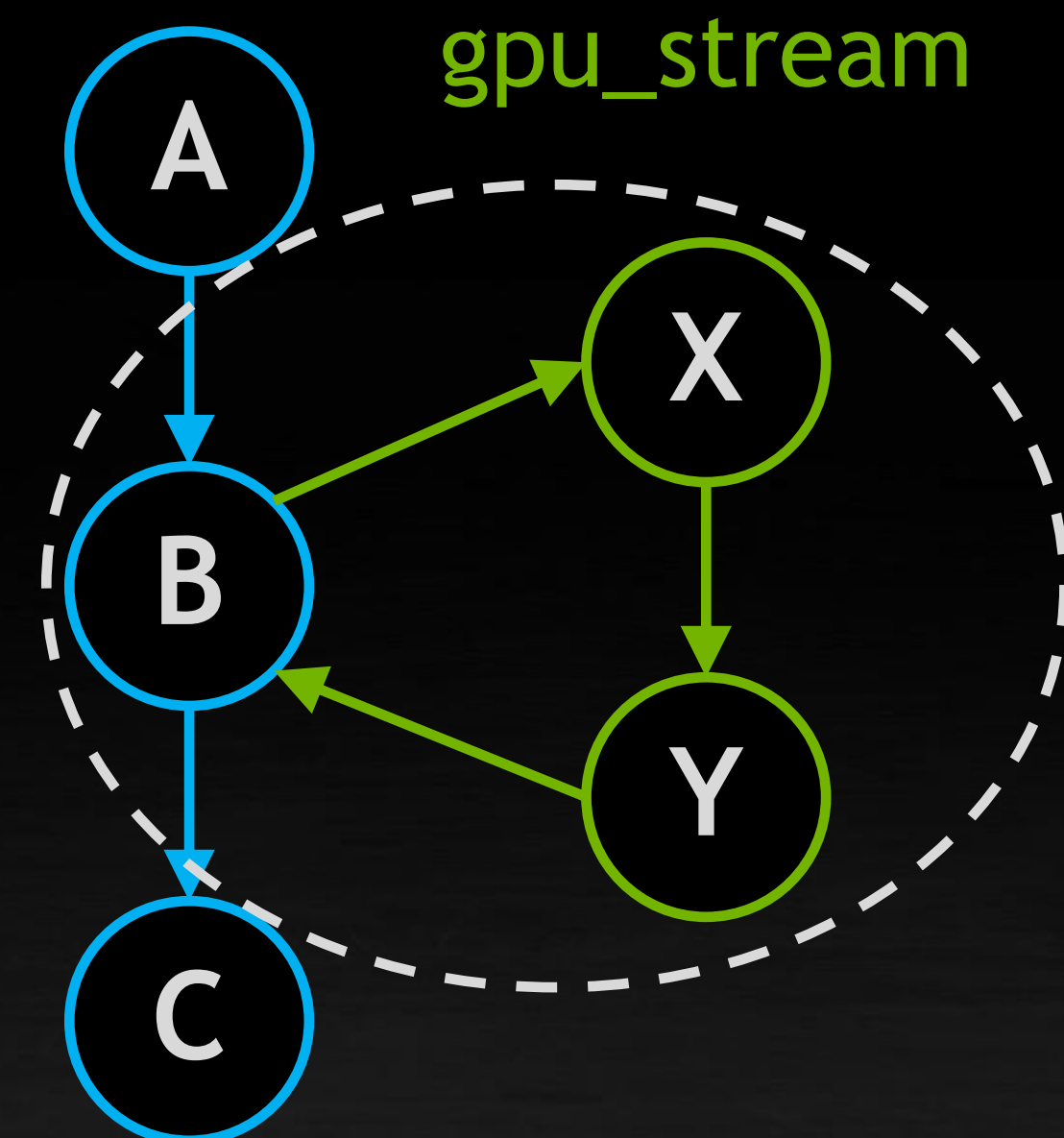
Encapsulation boundary
All launches from B just look
like part of B from the outside

DYNAMIC PARALLELISM PROGRAMMING MODEL: ENCAPSULATION

```
void main() {  
    cudaStream_t cpu_stream;  
    cudaStreamCreate(&cpu_stream);  
  
    A <<< ..., cpu_stream >>>();  
    B <<< ..., cpu_stream >>>();  
    C <<< ..., cpu_stream >>>();  
  
    cudaStreamSynchronize(cpu_stream);  
}
```

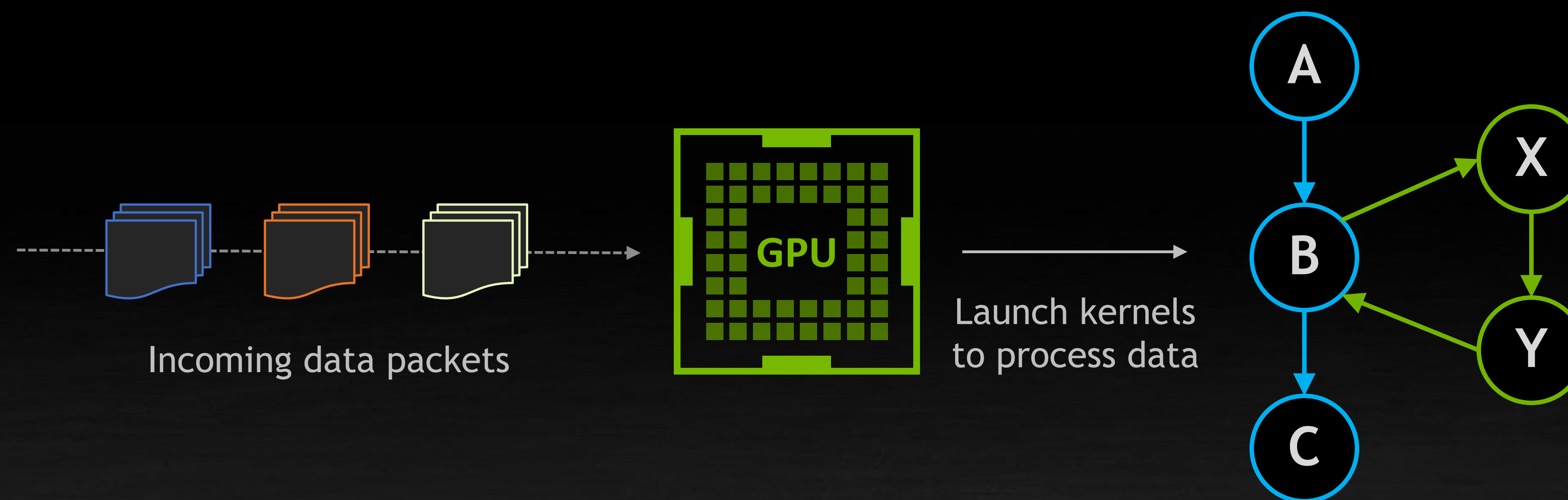
```
__global__ void B() {  
    cudaStream_t gpu_stream;  
    cudaStreamCreateWithFlags(&gpu_stream,  
                             cudaStreamNonBlocking);  
  
    X <<< ..., gpu_stream >>>();  
    Y <<< ..., gpu_stream >>>();  
  
    cudaDeviceSynchronize();  
    do_something();  
}
```

cpu_stream

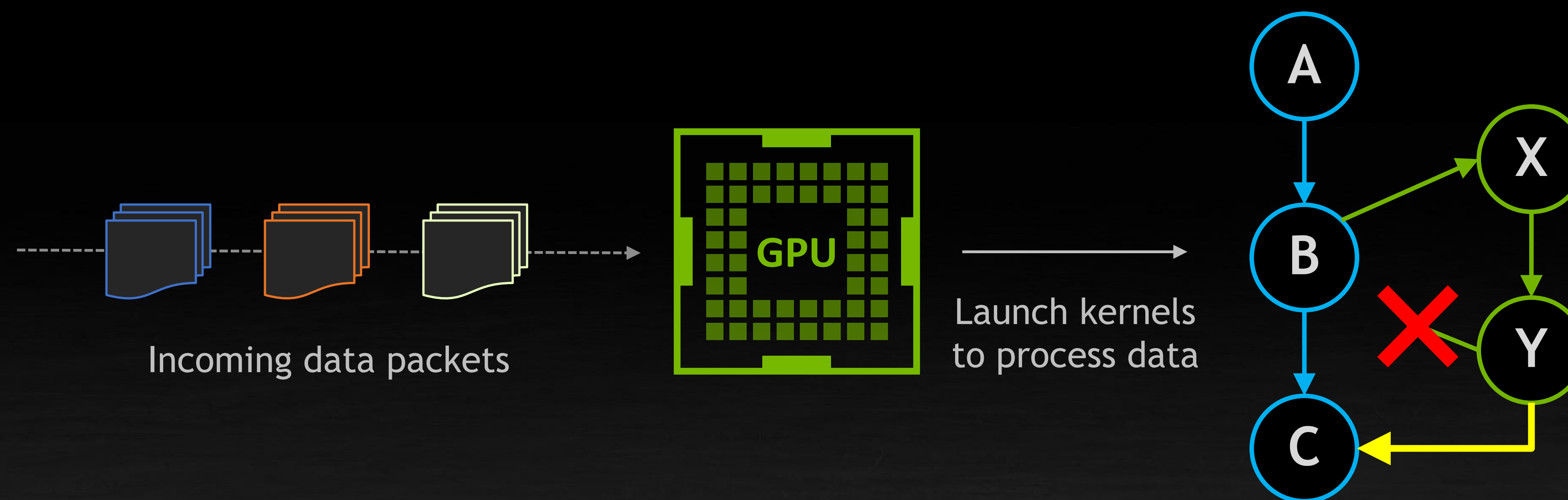


Encapsulation boundary
All launches from B just look
like part of B from the outside

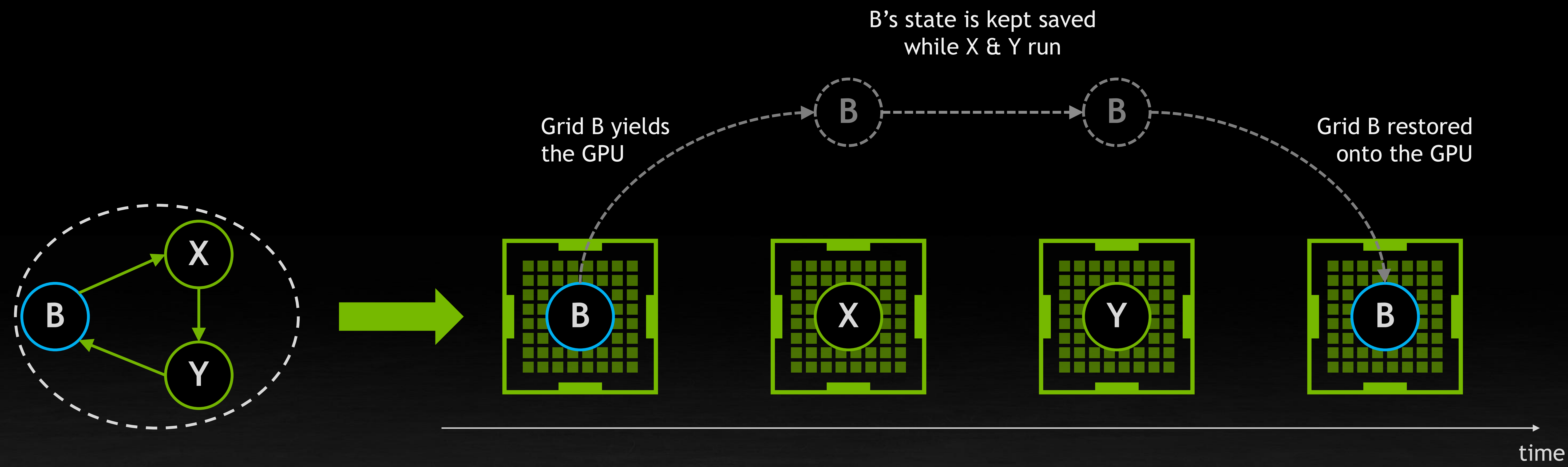
RUN-TIME SCHEDULING VS. FORK/JOIN PARALLELISM



RUN-TIME SCHEDULING VS. FORK/JOIN PARALLELISM

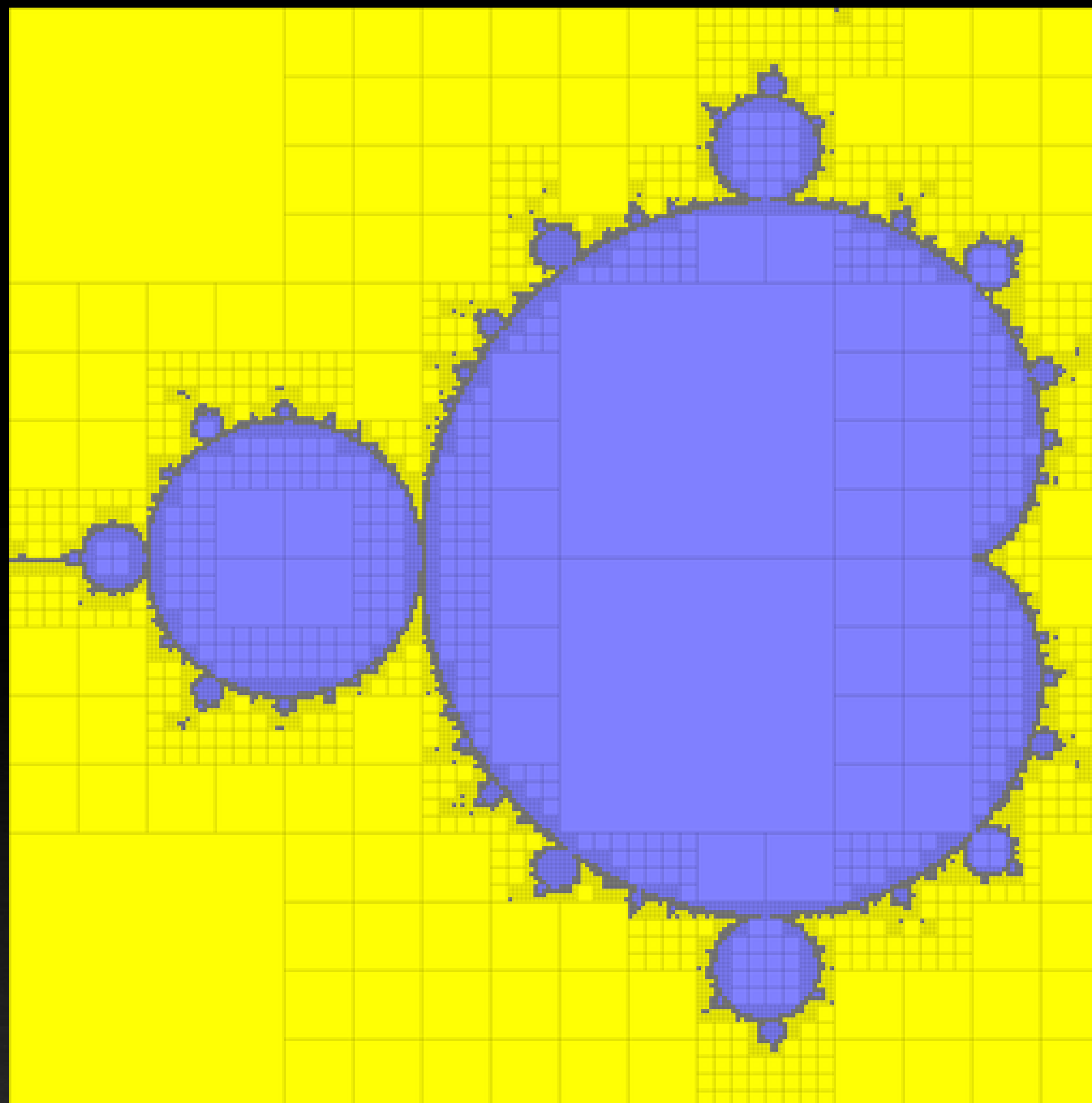


FORK/JOIN PARALLELISM TIMELINE



OPTIMIZING DYNAMIC PARALLELISM

Sometimes less is more



Adaptive parallel Mandelbrot → 14% end-to-end speedup

Mariani-Silver Algorithm on 16384x16384 grid, NVIDIA A10G / GA102

NAMED STREAMS

```
__global__ void B() {  
    cudaStream_t gpu_stream;  
    cudaStreamCreateWithFlags(&gpu_stream, cudaStreamPerThread);  
  
    X <<< ..., gpu_stream >>>();  
    Y <<< ..., gpu_stream >>>();  
}
```

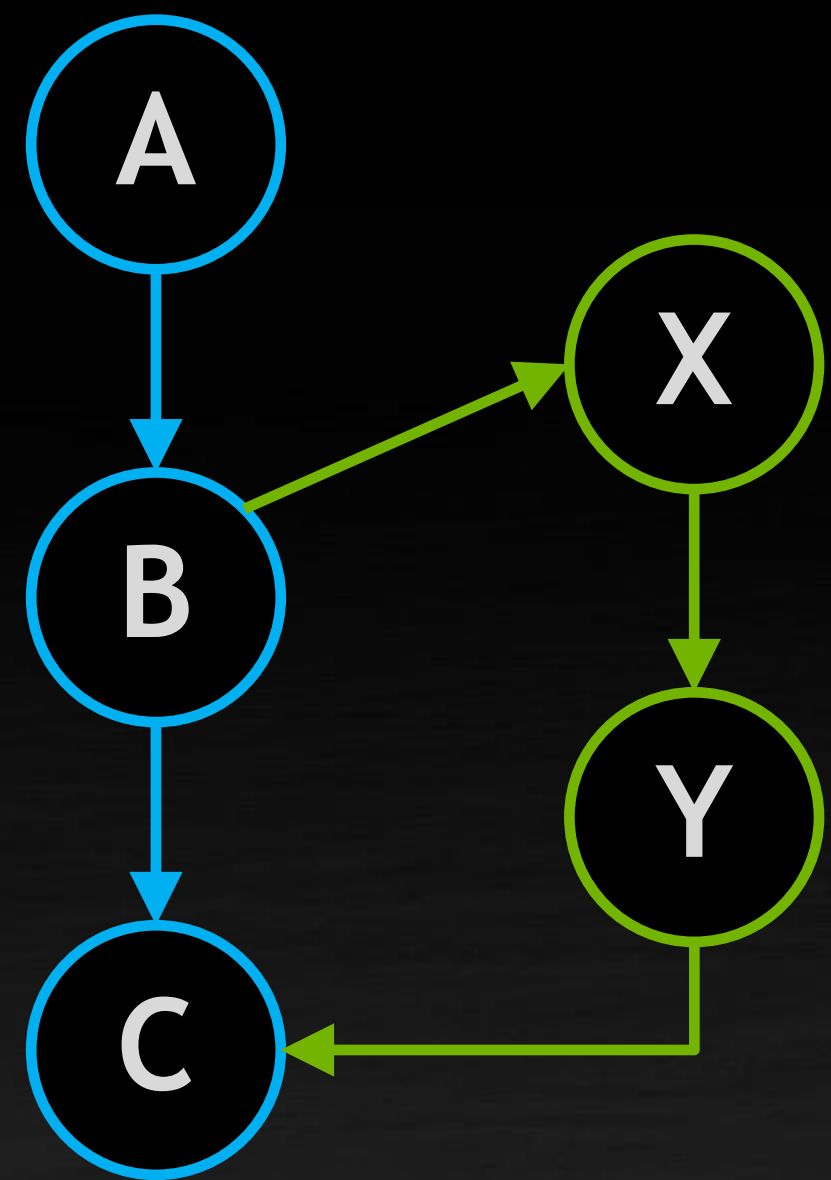
Previous example code using
generic stream creation

```
__global__ void B() {  
    X <<< ..., cudaStreamPerThread >>>();  
    Y <<< ..., cudaStreamPerThread >>>();  
}
```

Similar code using
“named” stream

OPTIMIZING COMMON LAUNCH PATTERNS

A higher-performance, enhanced programming model using “named streams”



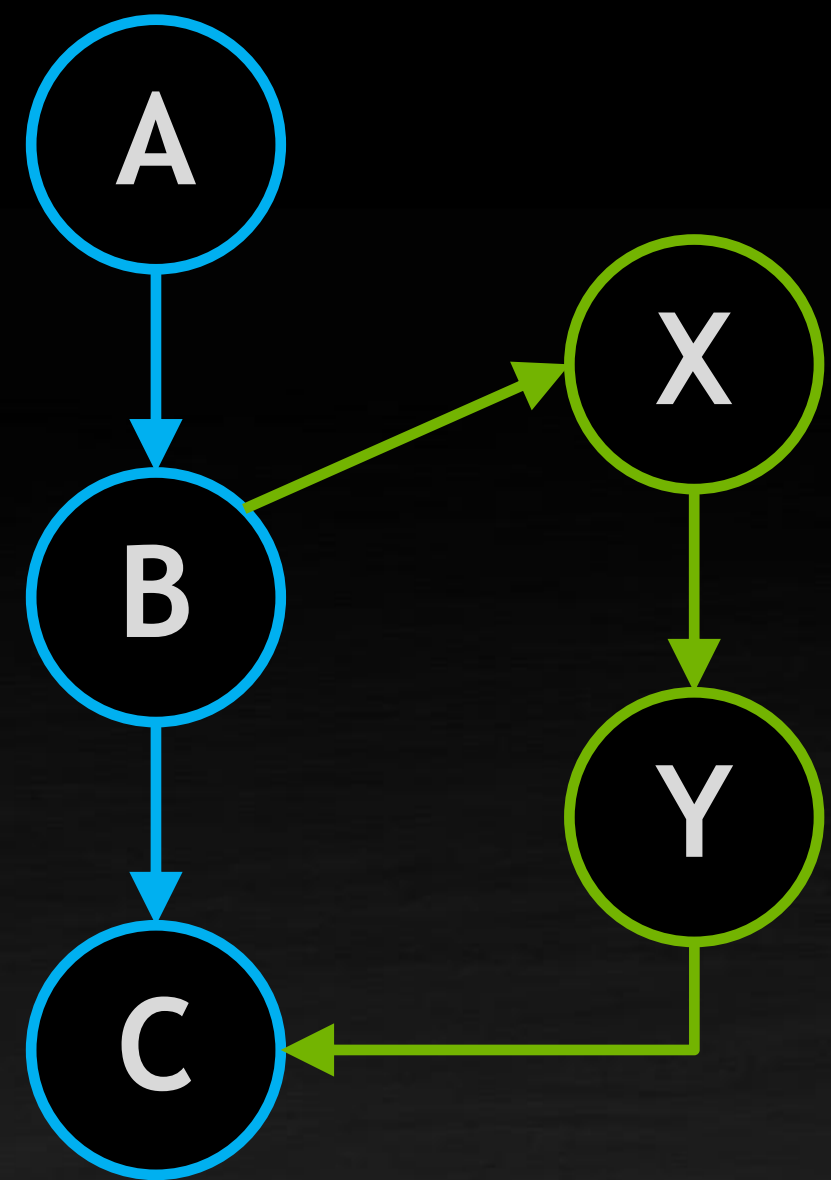
Per-Thread stream

X & Y execute **sequentially**,
similar to existing stream launch

```
__global__ void B() {  
    X <<< ..., cudaStreamPerThread >>>();  
    Y <<< ..., cudaStreamPerThread >>>();  
}
```

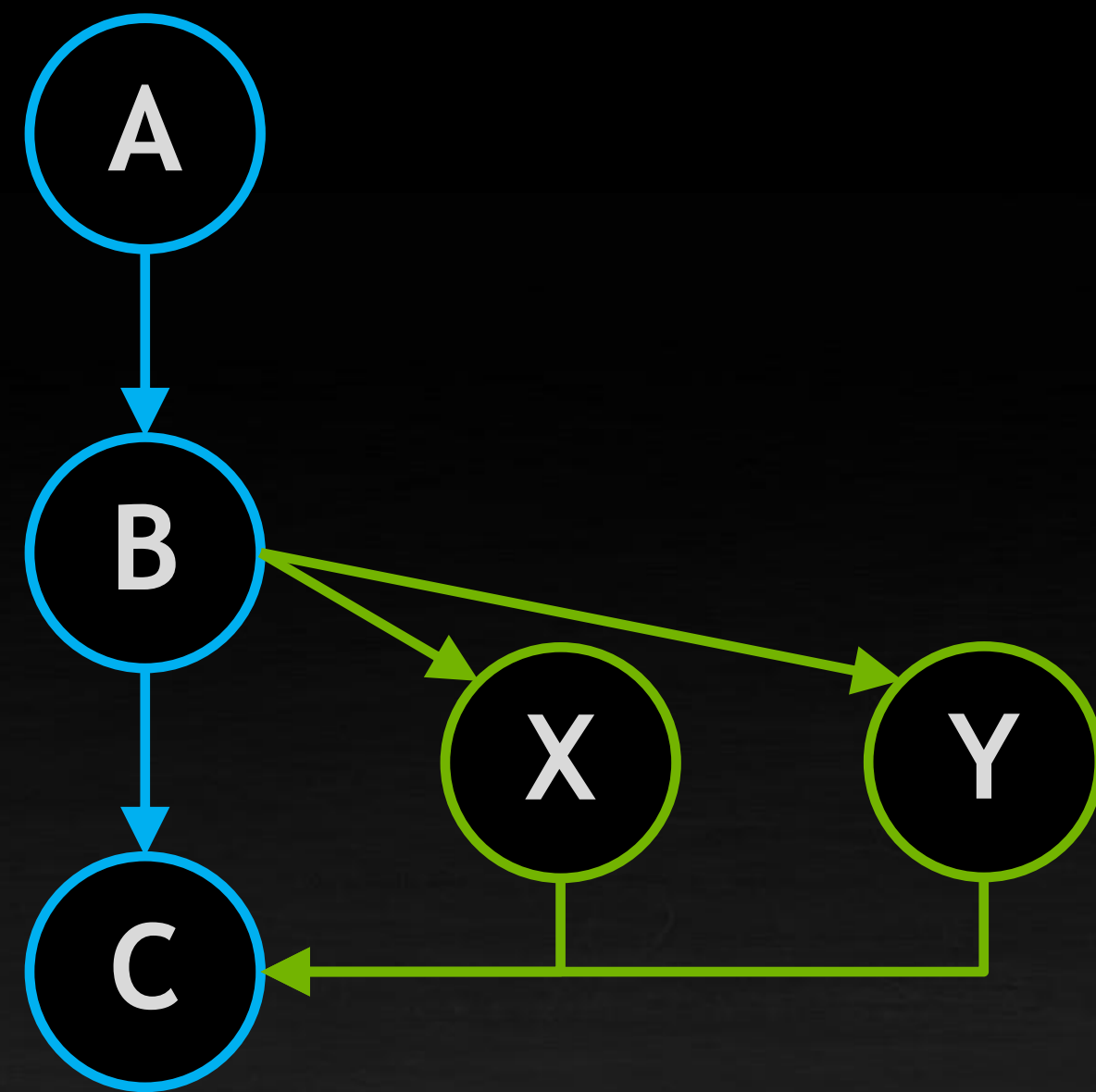

THREE NEW TYPES OF DEVICE-SIDE LAUNCH

A higher-performance, enhanced programming model using “named streams”



Per-Thread stream

X & Y execute **sequentially**,
similar to existing stream launch



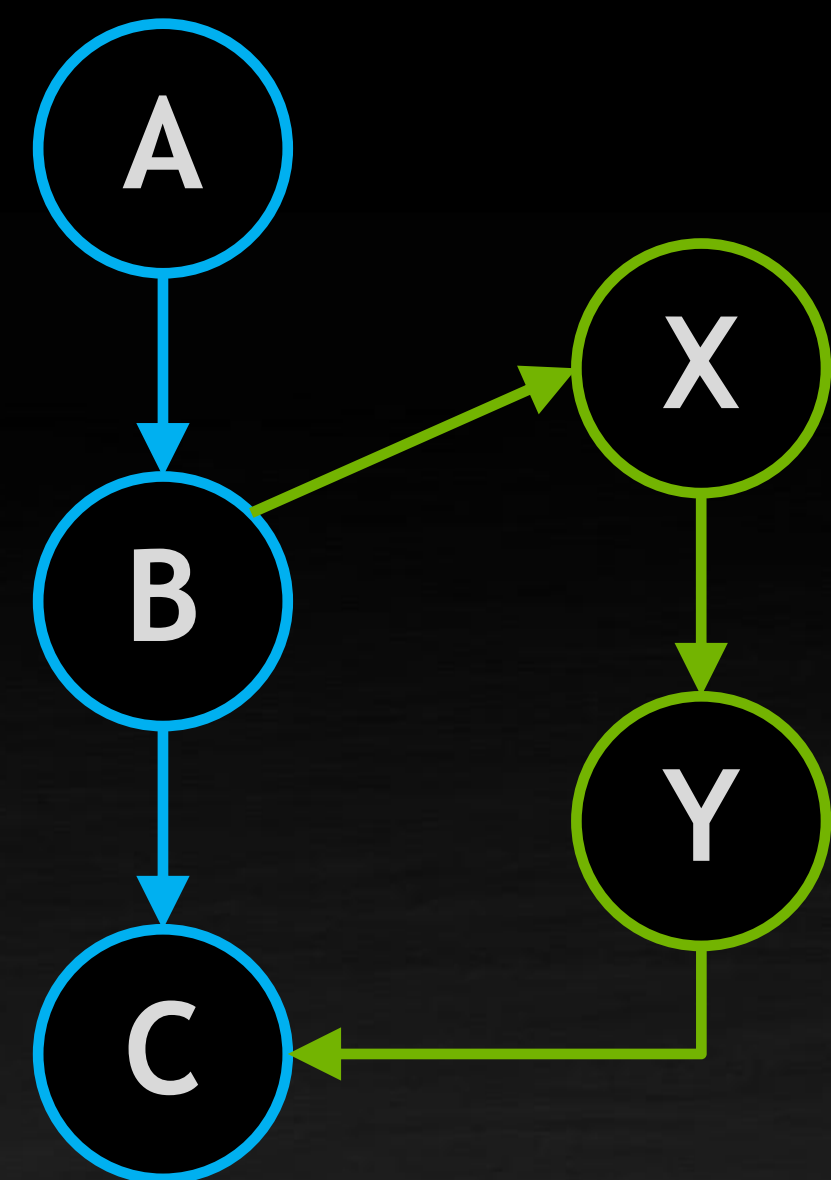
Fire-and-forget

X & Y execute **independently**
as if launched in separate streams

```
__global__ void B() {  
    X <<< ..., cudaStreamFireAndForget >>>();  
    Y <<< ..., cudaStreamFireAndForget >>>();  
}
```

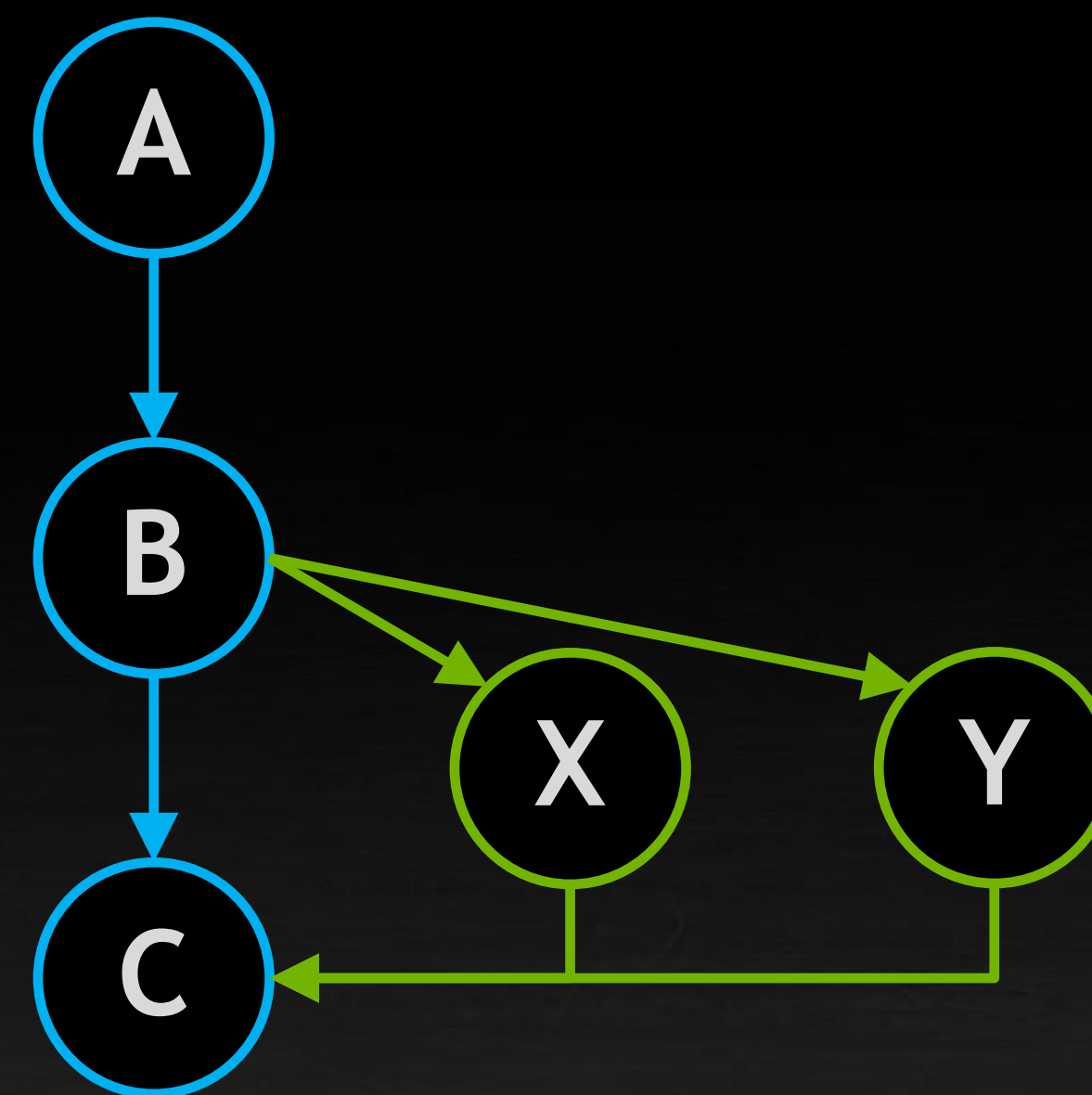

THREE NEW TYPES OF DEVICE-SIDE LAUNCH

A higher-performance, enhanced programming model using “named streams”



Per-Thread stream

X & Y execute **sequentially**,
similar to existing stream launch



Fire-and-forget

X & Y execute **independently**
as if launched in separate streams



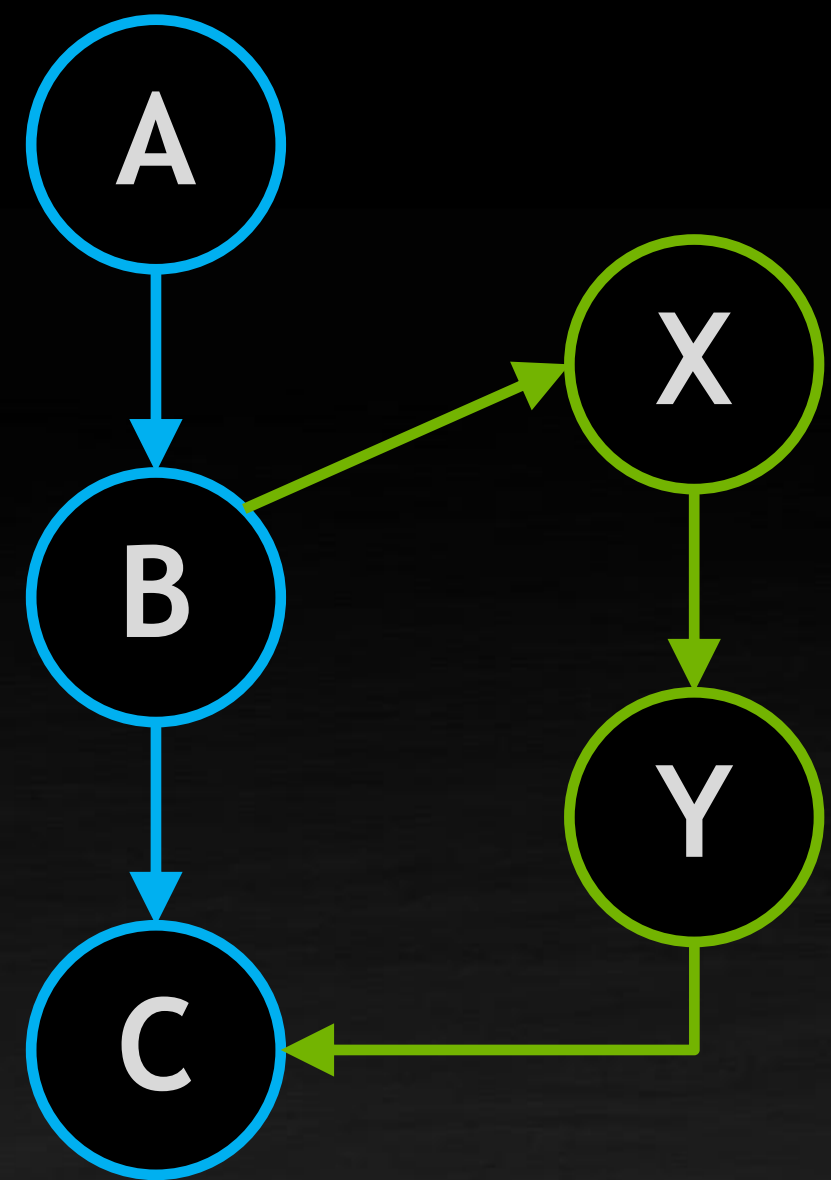
Tail launch

X & Y execute sequentially
after parent kernel completes

```
__global__ void B() {  
    X <<< ..., cudaStreamTailLaunch >>>();  
    Y <<< ..., cudaStreamTailLaunch >>>();  
}
```

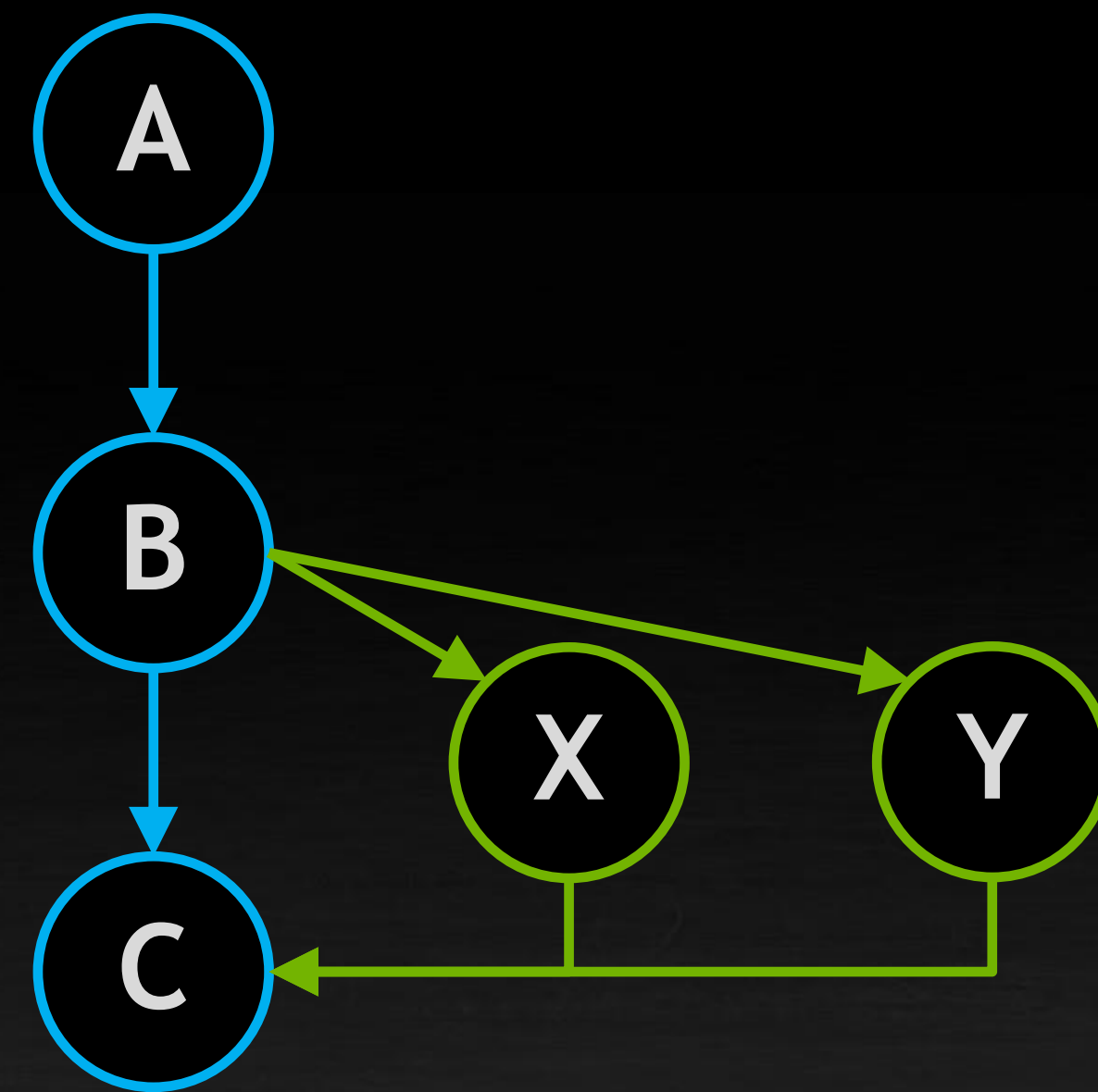

THREE NEW TYPES OF DEVICE-SIDE LAUNCH

A higher-performance, enhanced programming model using “named streams”



Per-Thread stream

X & Y execute **sequentially**,
similar to existing stream launch



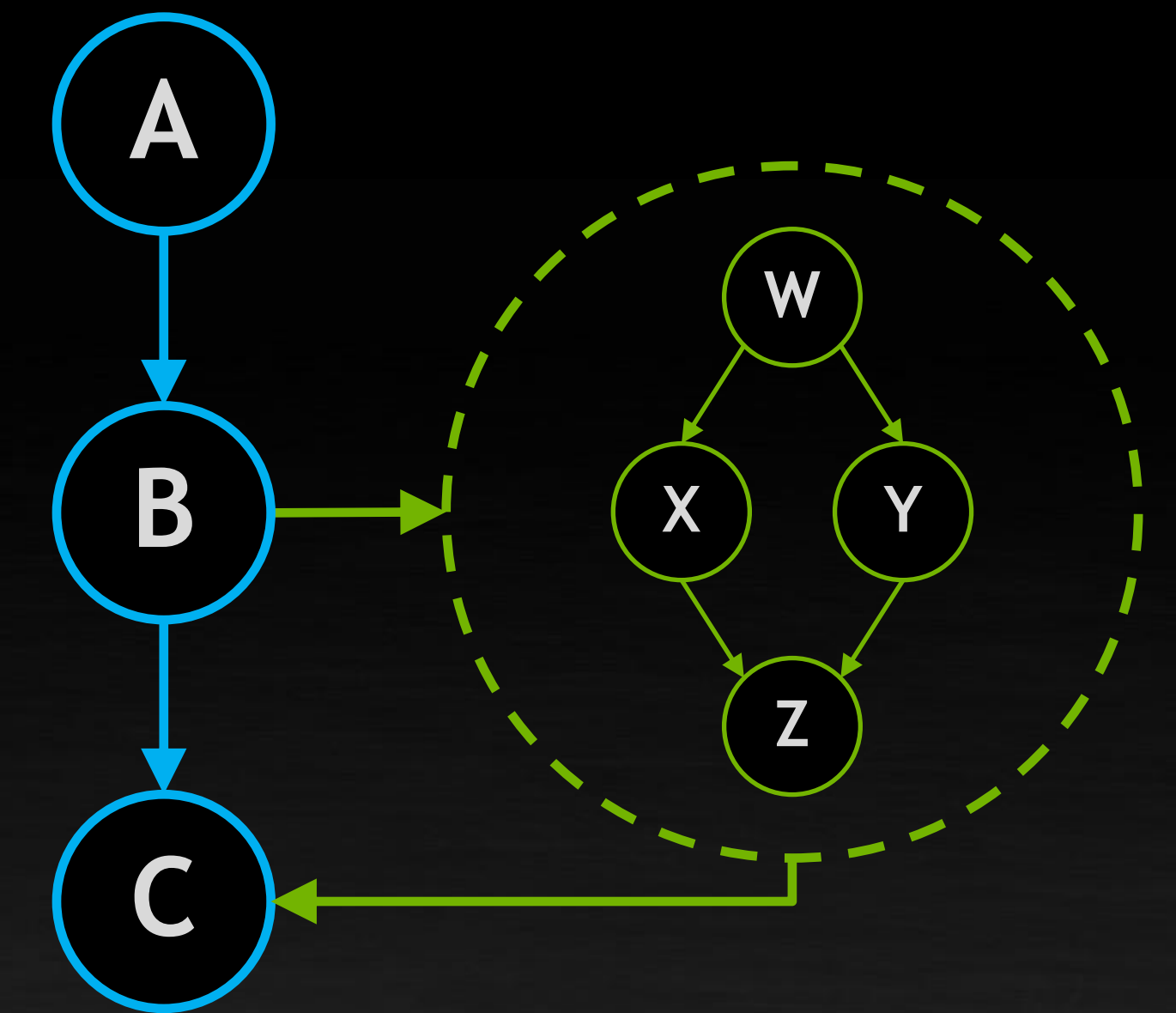
Fire-and-forget

X & Y execute **independently**
as if launched in separate streams



Tail launch

X & Y execute sequentially
after parent kernel completes

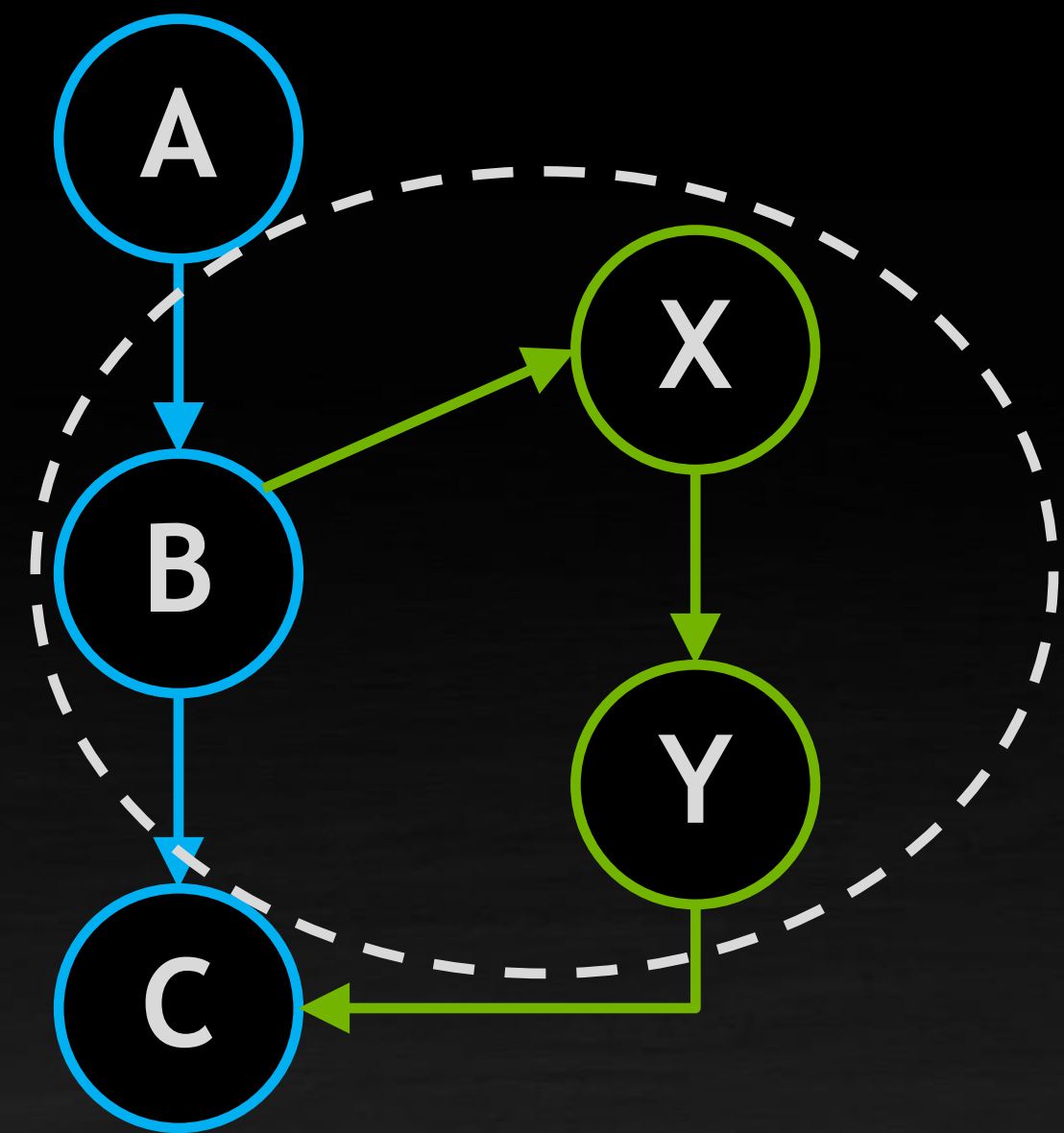


Graph launch

Can now launch whole graphs
from a GPU kernel

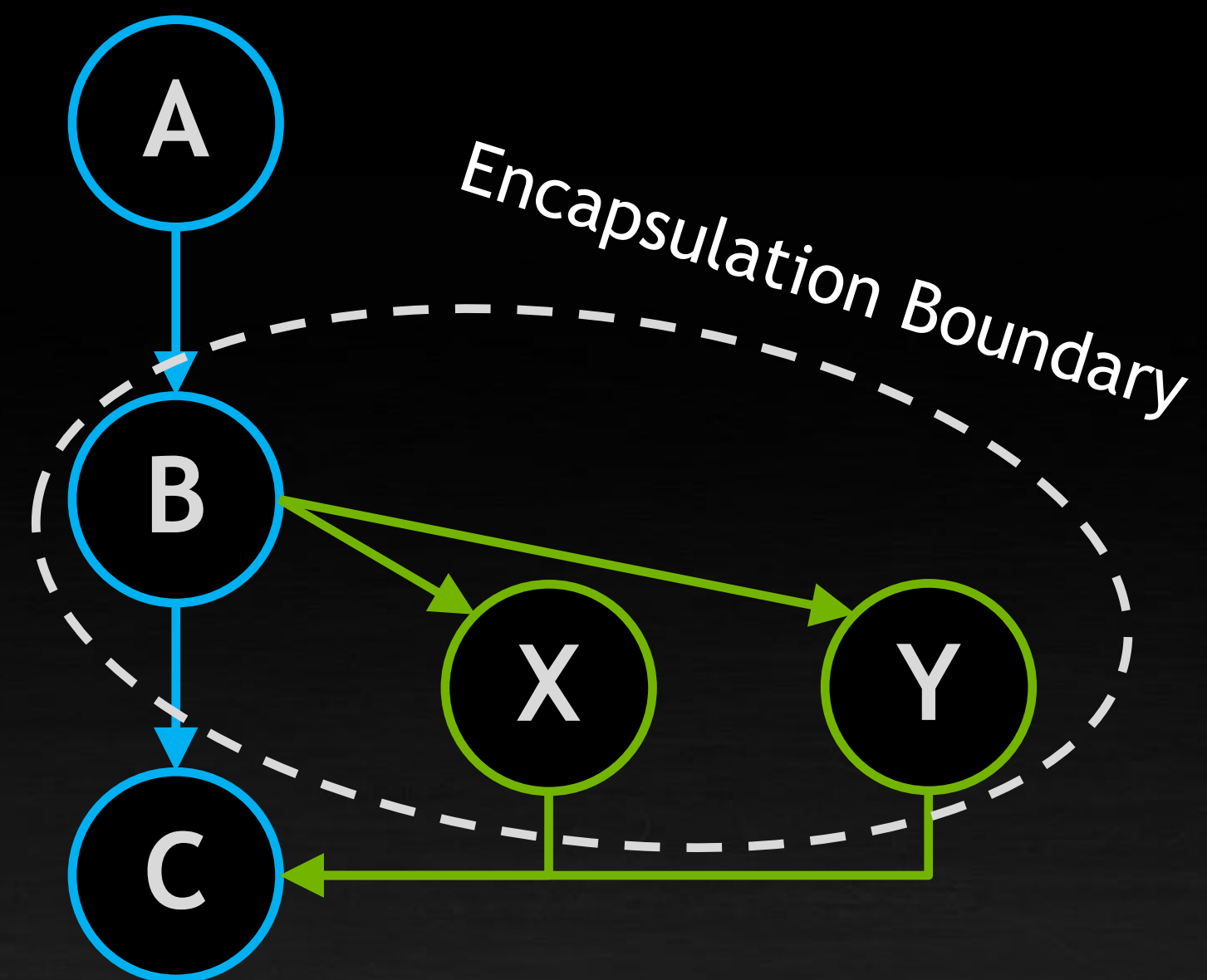
THREE NEW TYPES OF DEVICE-SIDE LAUNCH

Encapsulation rule always applies



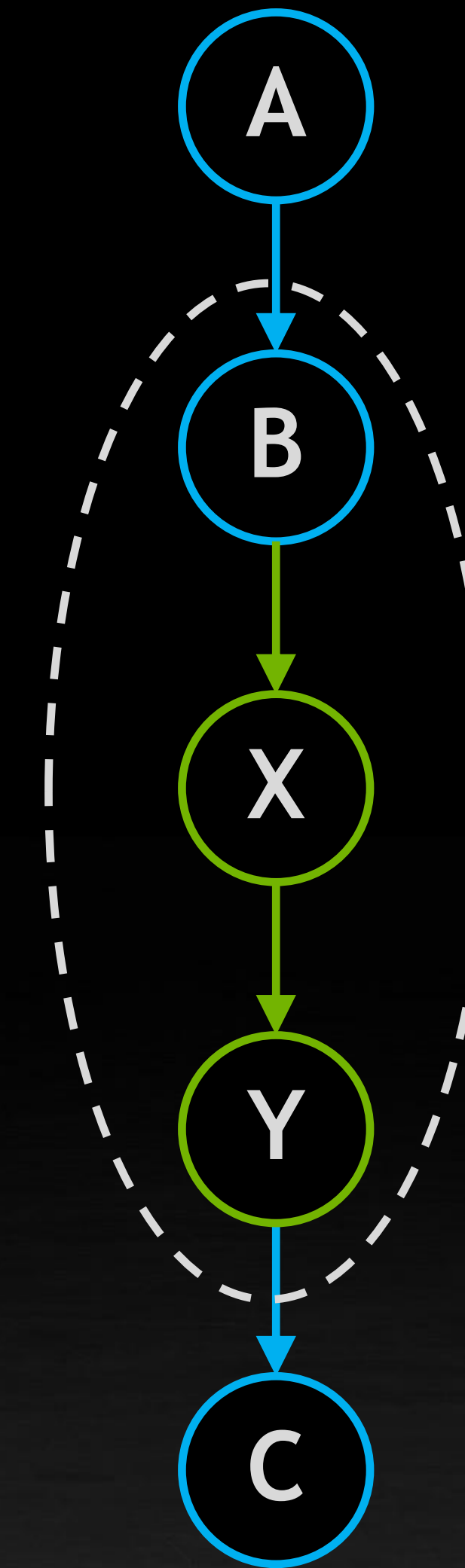
Per-Thread stream

X & Y execute **sequentially**, similar to existing stream launch



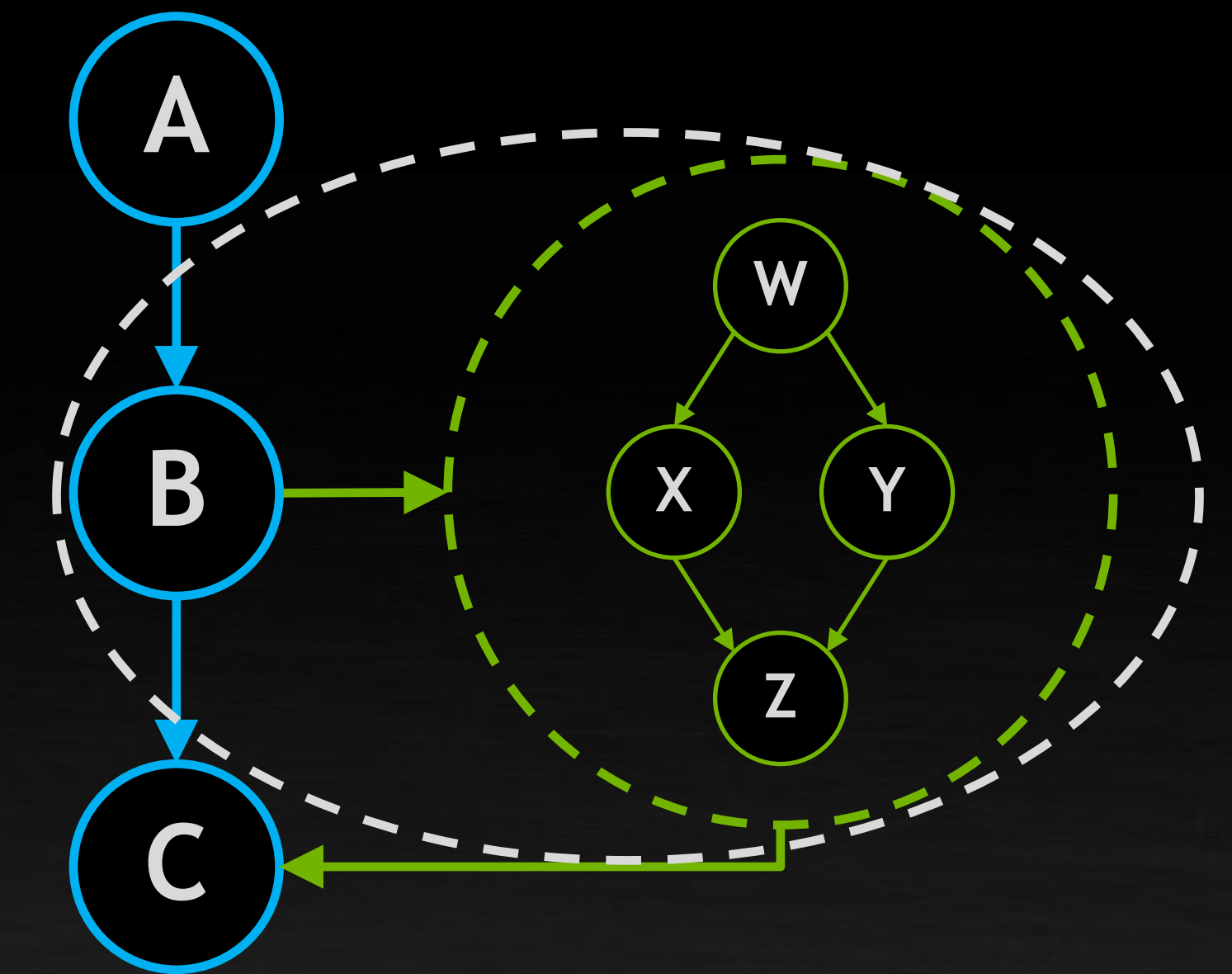
Fire-and-forget

X & Y execute **independently** as if launched in separate streams



Tail launch

X & Y execute sequentially **after** parent kernel completes

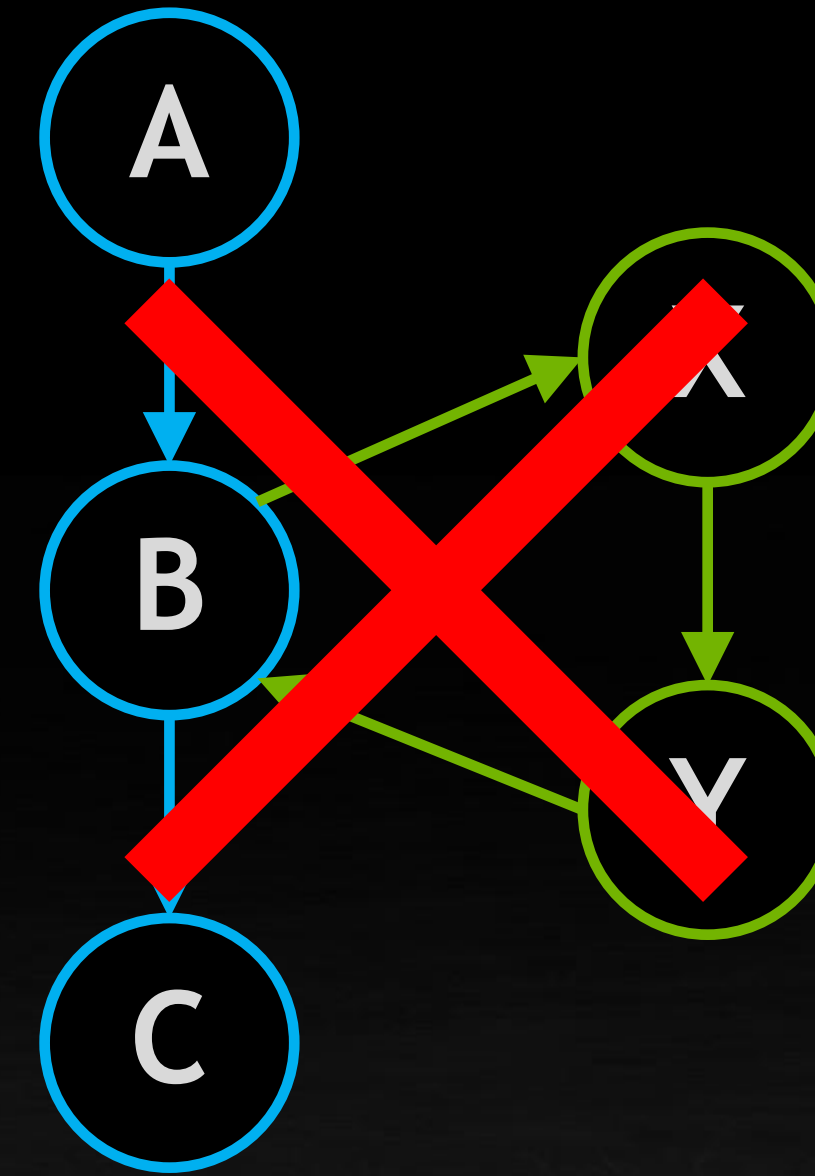


Graph launch

Can now launch whole graphs from a GPU kernel

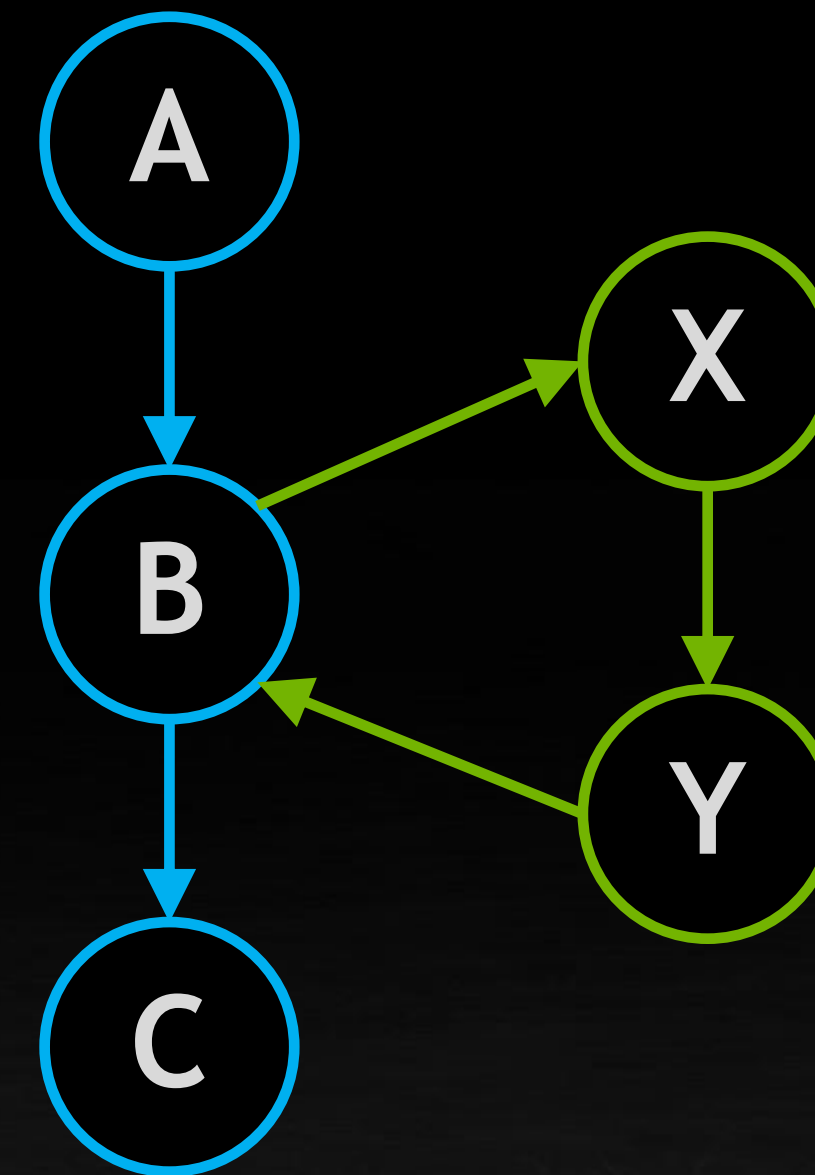
ELIMINATION OF FORK/JOIN DYNAMIC PARALLELISM PATTERN

```
__global__ void B() {  
    cudaStream_t gpu_stream;  
    cudaStreamCreateWithFlags(&gpu_stream,  
                             cudaStreamNonBlocking);  
  
    X <<< ..., gpu_stream >>>();  
    Y <<< ..., gpu_stream >>>();  
    cudaDeviceSynchronize();  
    do_something();  
}
```



LEGACY FORK/JOIN COMPATIBILITY MODE

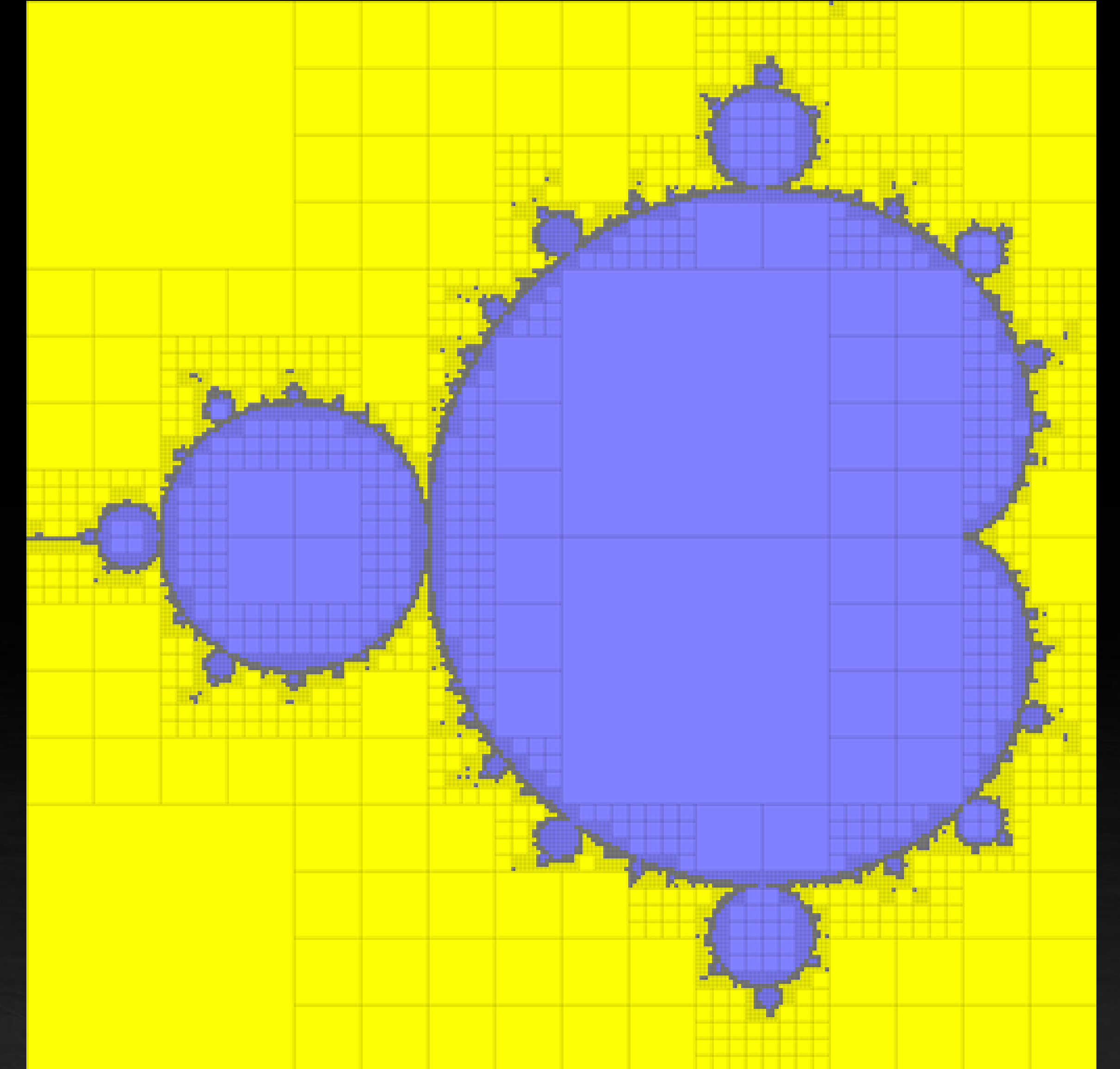
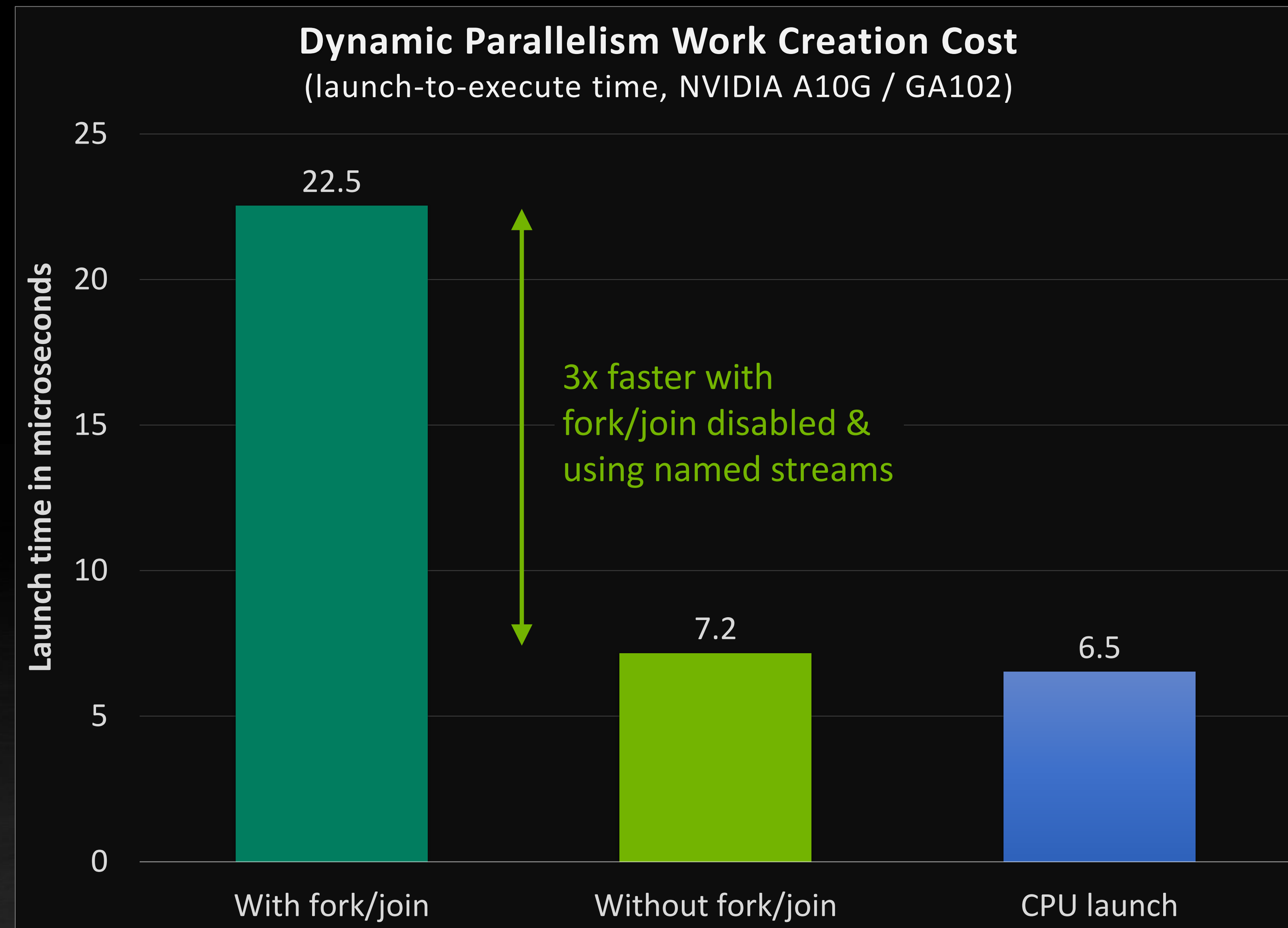
```
__global__ void B() {  
    cudaStream_t gpu_stream;  
    cudaStreamCreateWithFlags(&gpu_stream,  
                             cudaStreamNonBlocking);  
  
    X <<< ..., gpu_stream >>>();  
    Y <<< ..., gpu_stream >>>();  
    cudaDeviceSynchronize();  
    do_something();  
}
```



set CUDA_FORCE_CDP1_IF_SUPPORTED=1

***Not available on Hopper architecture GPUs**

PUTTING IT ALL TOGETHER



Adaptive parallel Mandelbrot → 14% end-to-end speedup

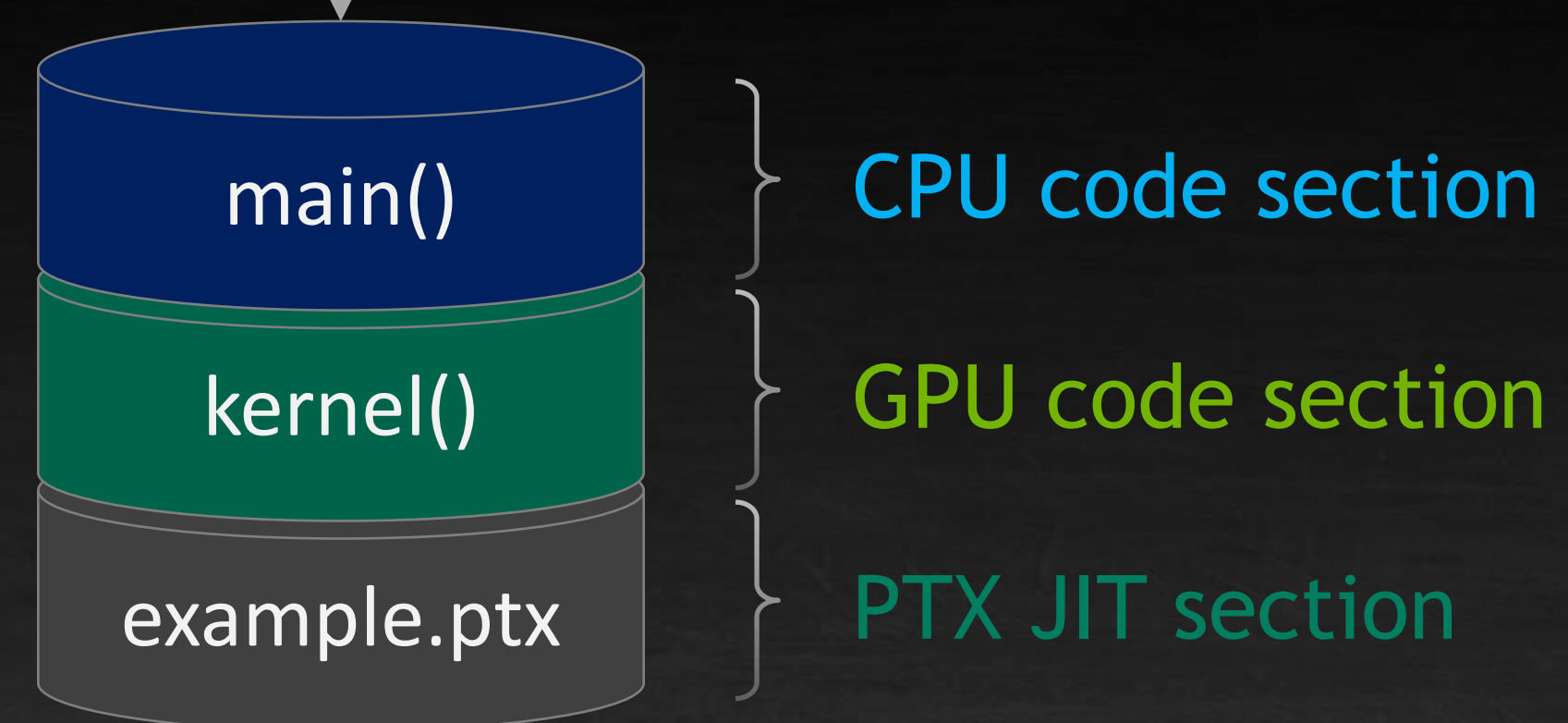
Mariani-Silver Algorithm on 16384x16384 grid, NVIDIA A10G / GA102

ANATOMY OF A CUDA BINARY

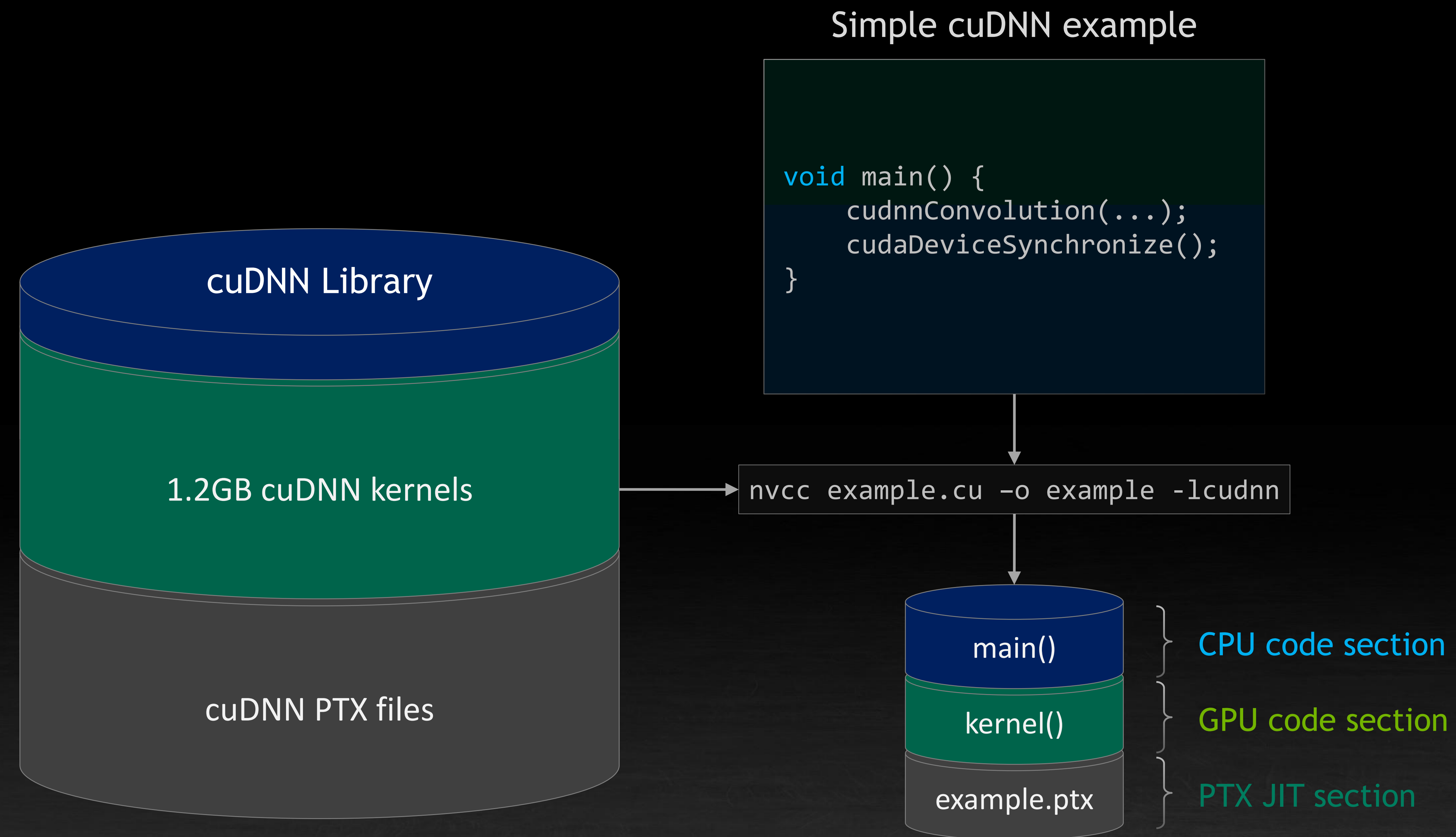
Hello world example

```
__global__ void kernel() {  
    printf("Hello, CUDA\n");  
}  
  
void main() {  
    kernel<<< 1, 1 >>>();  
    cudaDeviceSynchronize();  
}
```

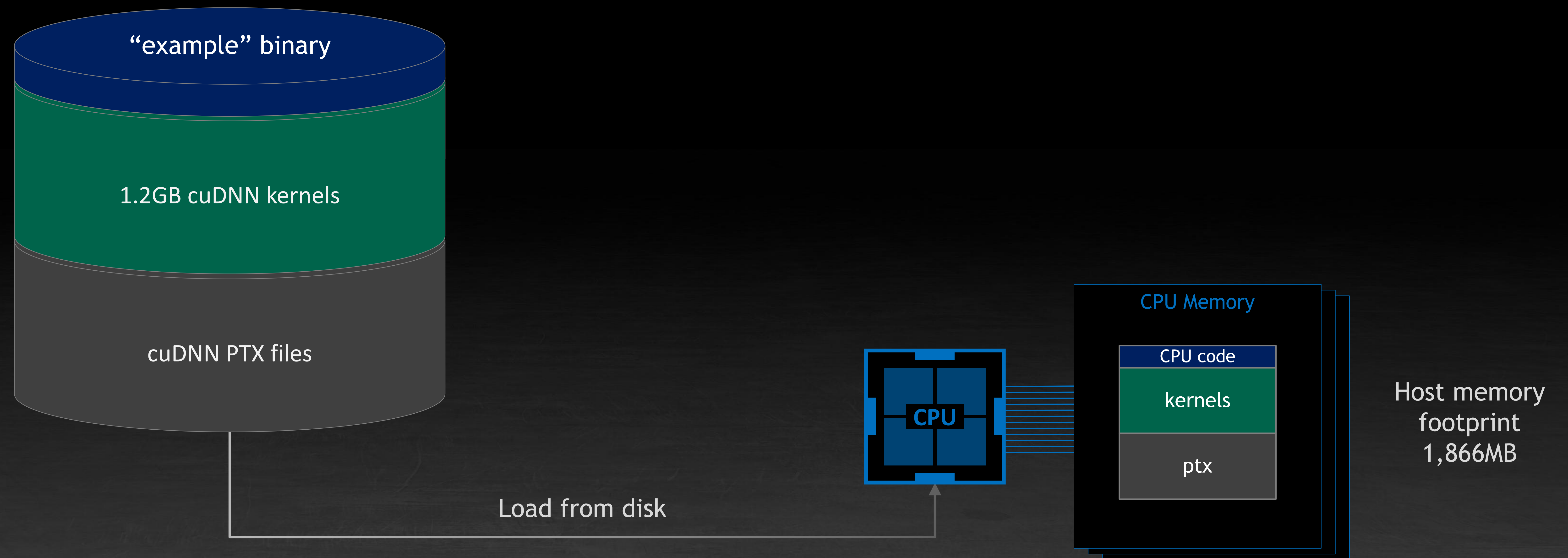
nvcc example.cu -o example



ANATOMY OF A CUDA BINARY

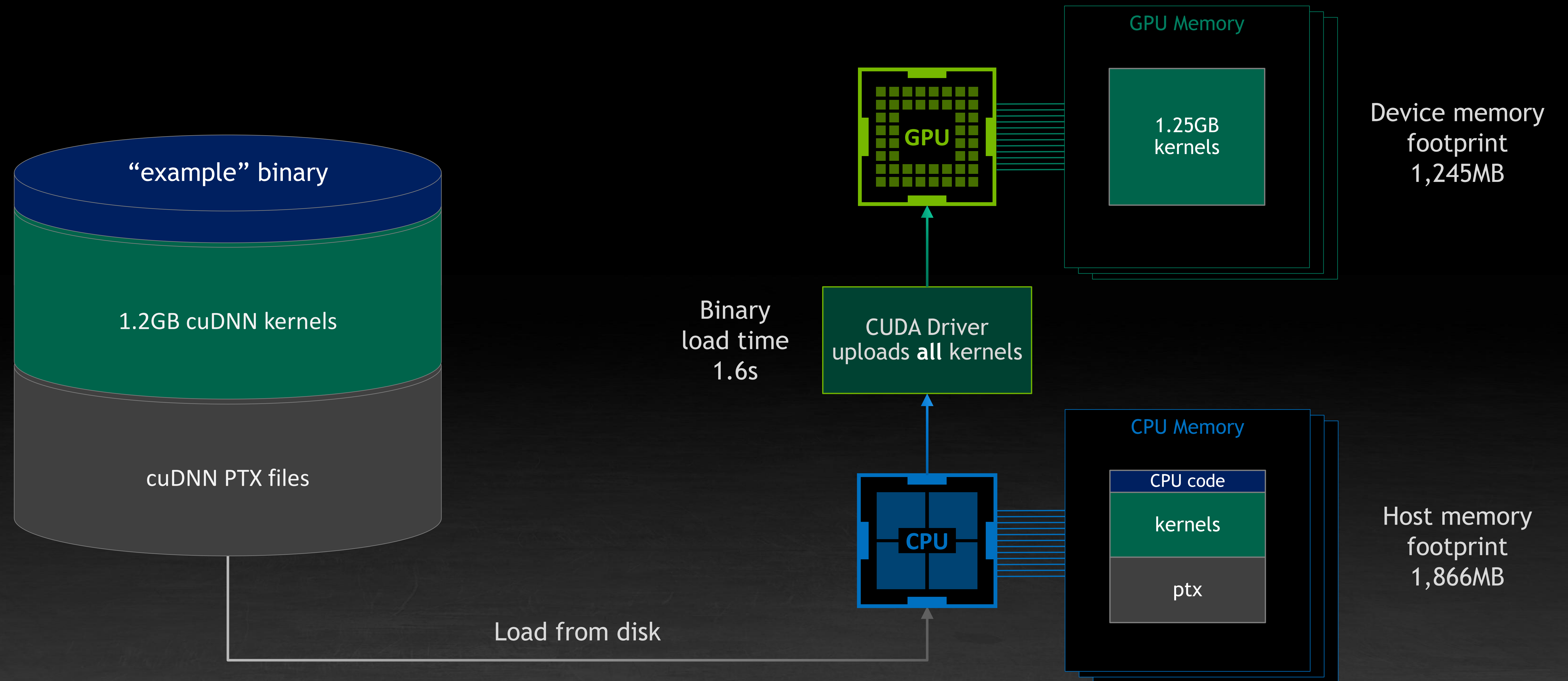


LOADING A CUDA BINARY



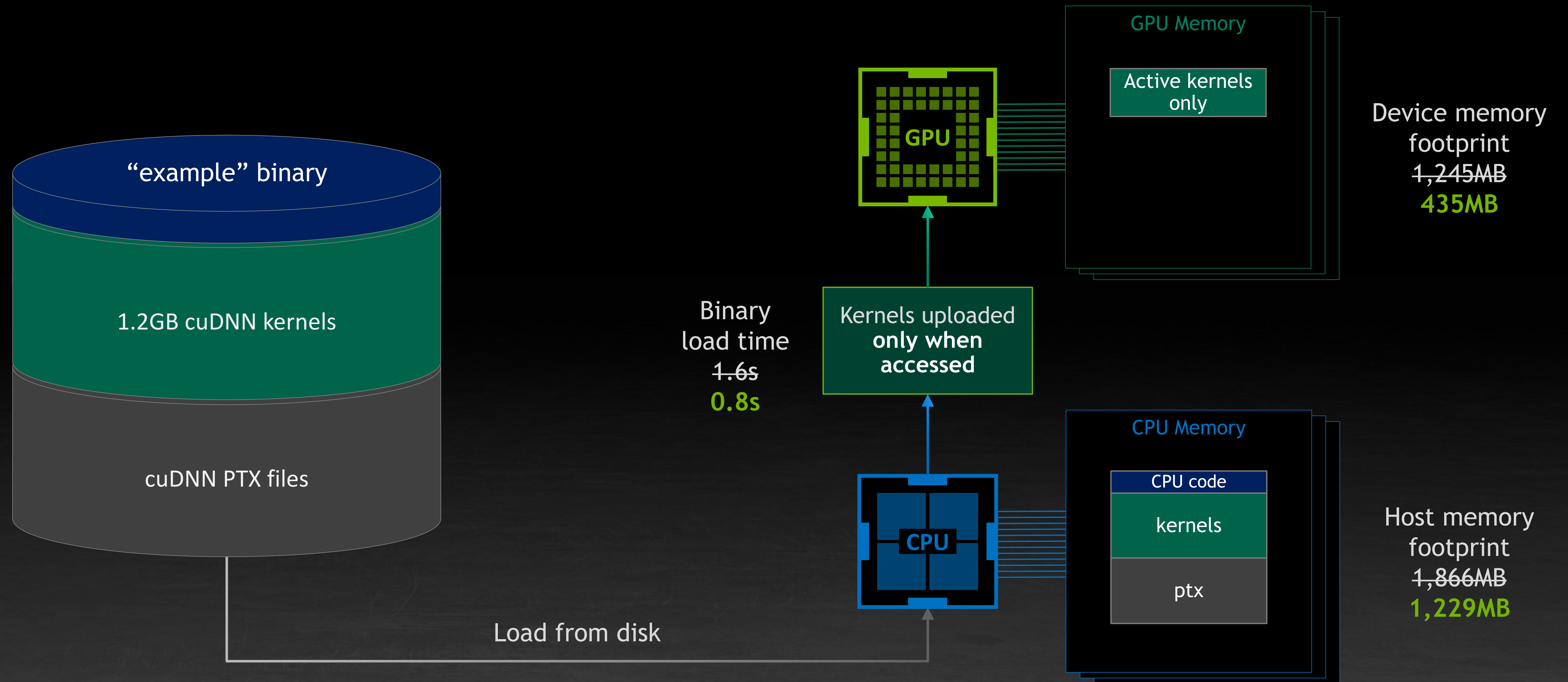
LOADING A CUDA BINARY

cuDNN Example - End-to-end run time: 2.9s



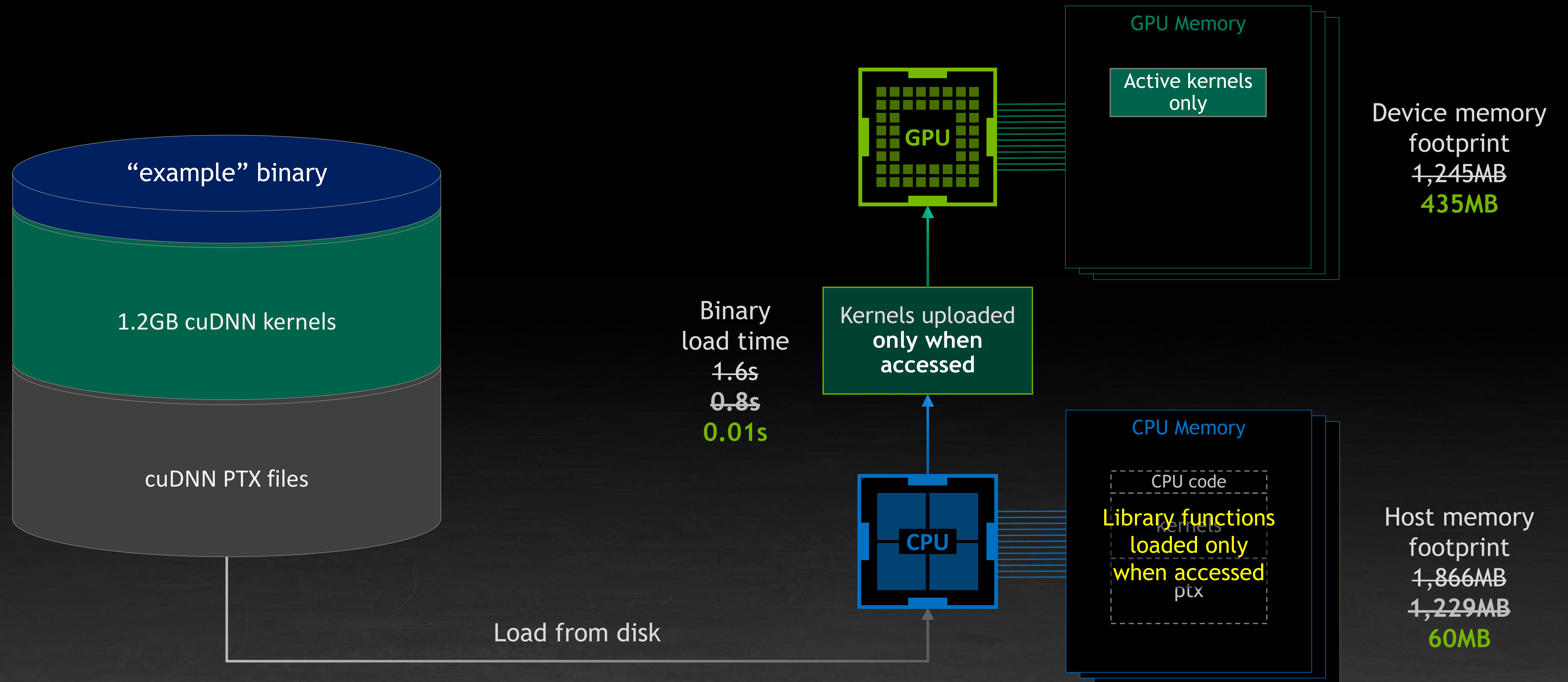
LAZY KERNEL FUNCTION LOADING

cuDNN Example - End-to-end run time: ~~2.9s~~ **1.7s**



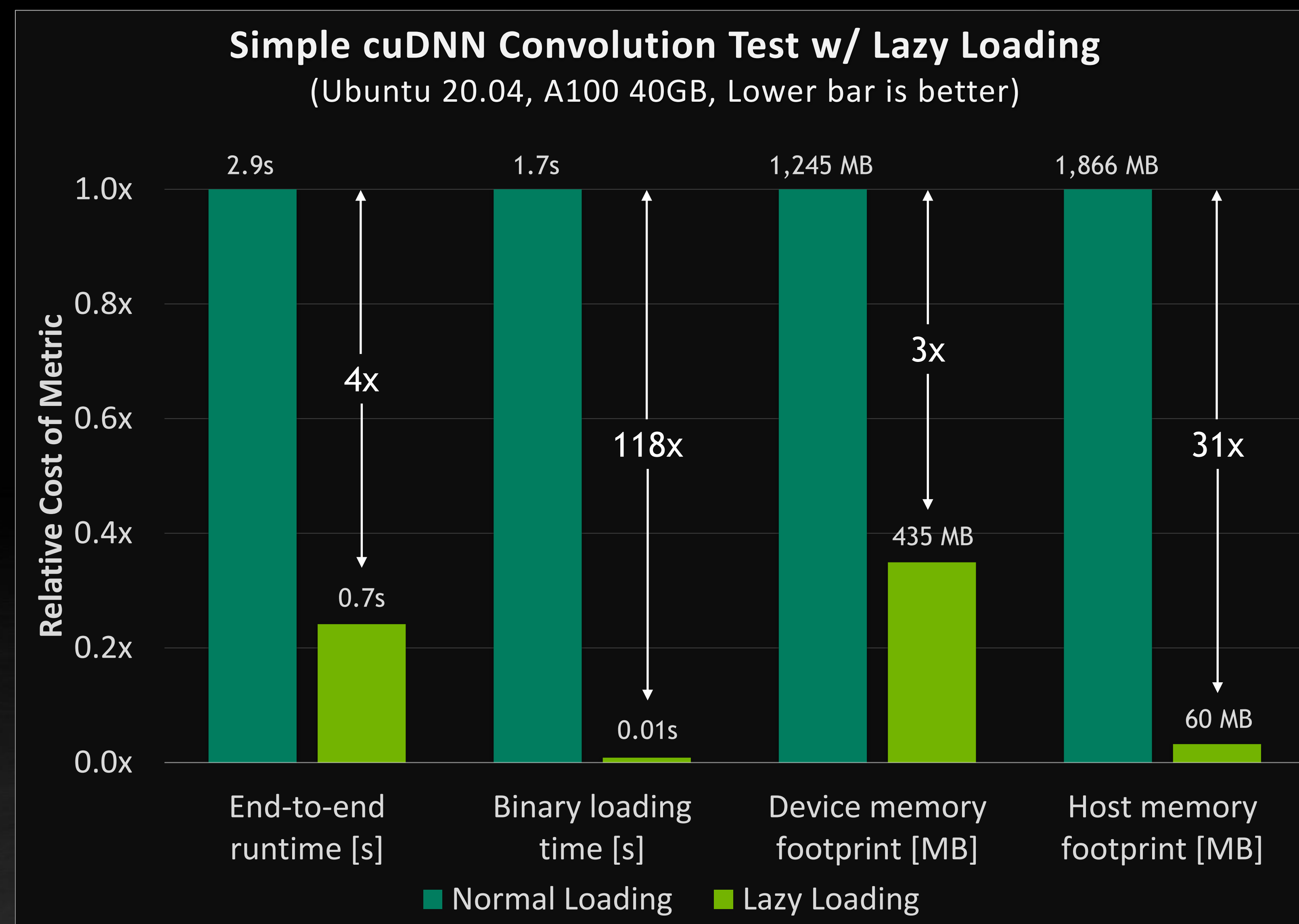
LAZY HOST MODULE LOADING

cuDNN Example - End-to-end run time: ~~2.9s~~ ~~1.7s~~ **0.7s**

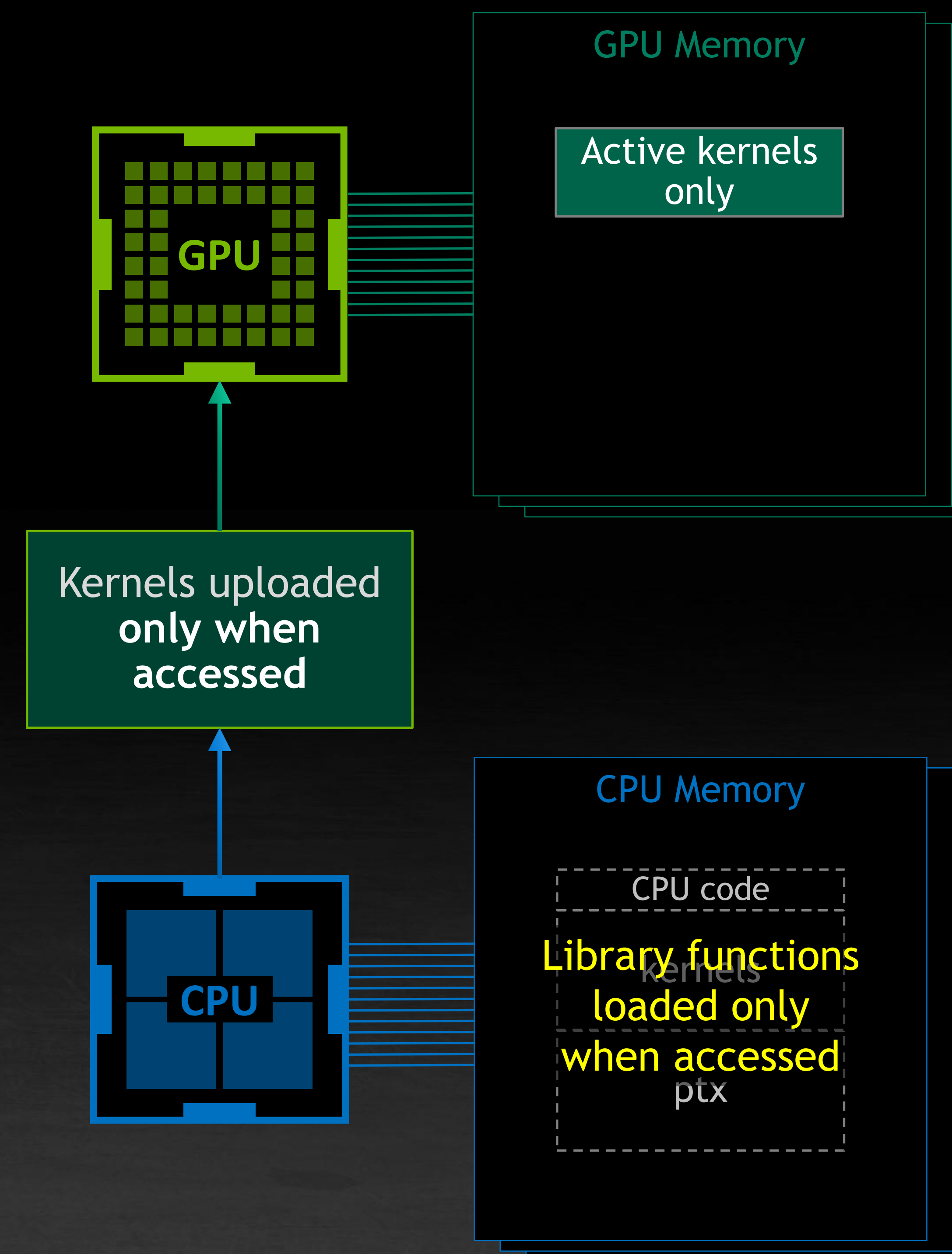


ACTIVATE USING AN ENVIRONMENT VARIABLE

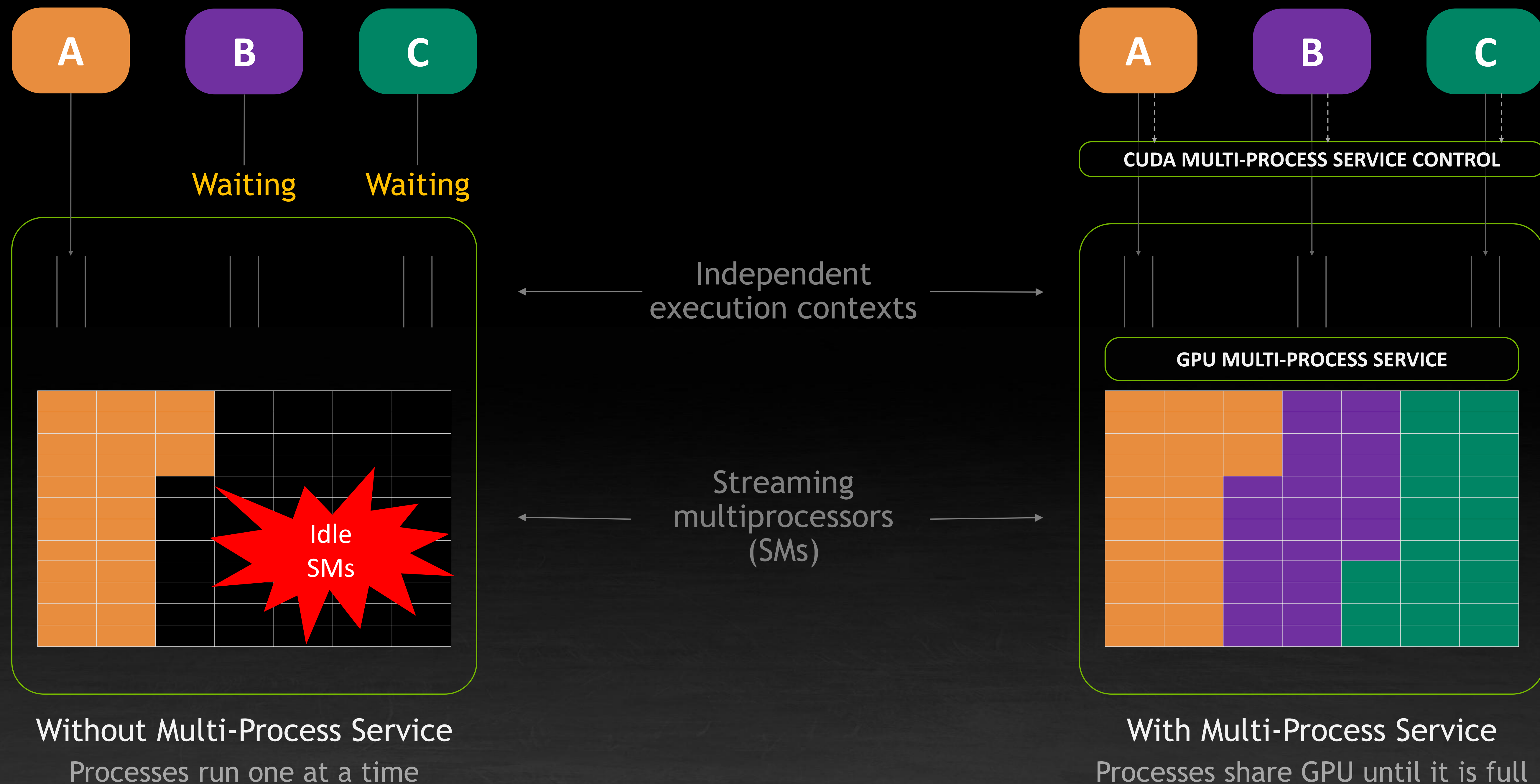
`CUDA_MODULE_LOADING=LAZY`



```
set CUDA_MODULE_LOADING=LAZY
```



MULTI-PROCESS SERVICE (MPS) ALLOWS CONCURRENT SHARING OF THE GPU



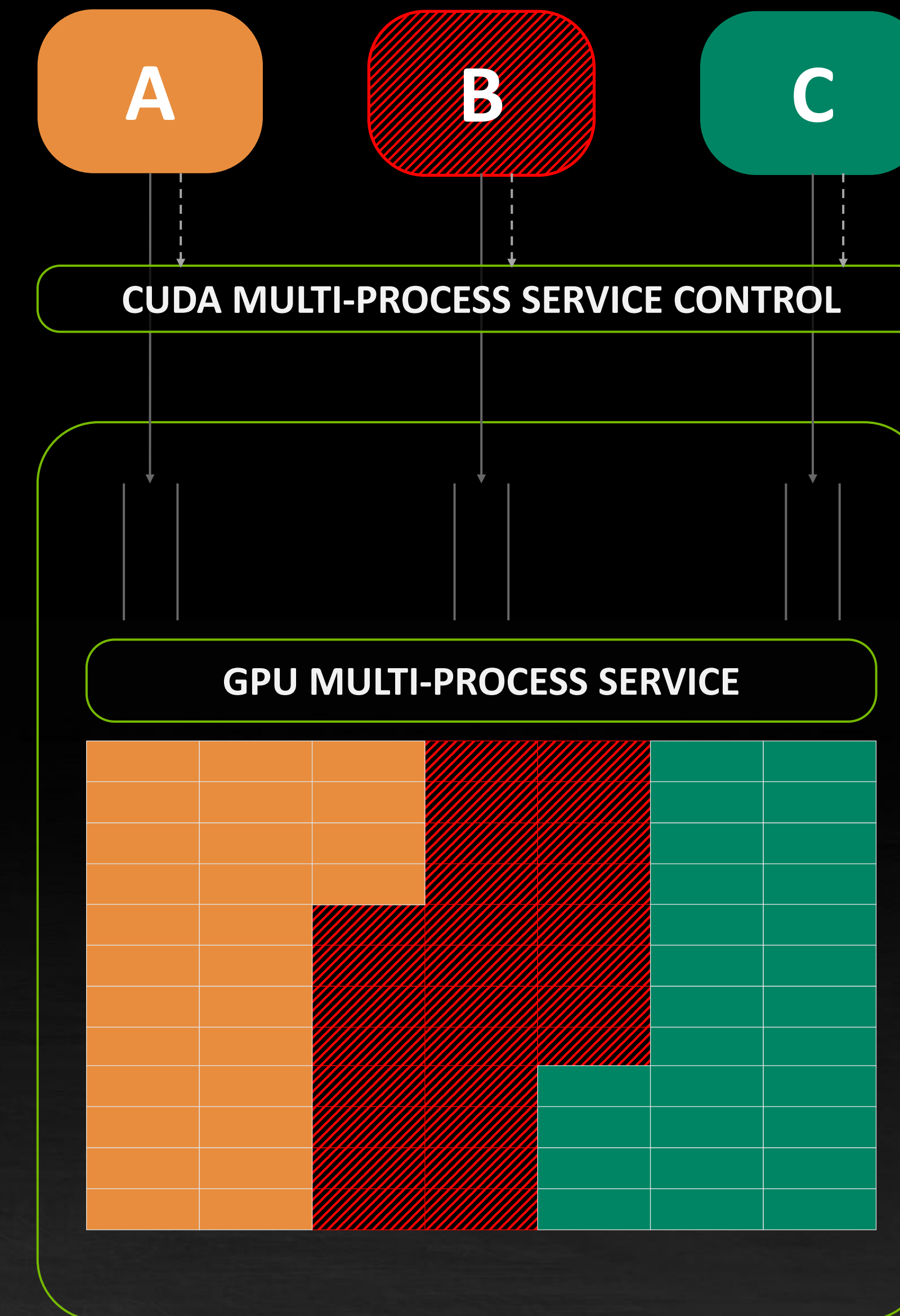
MPS INDIVIDUAL CLIENT CONTROLLED TERMINATION

Multi-Process Service clients cannot be killed while active

Clients are co-scheduled while managed by service

Unexpected shutdown may leave service in an indeterminate state

Difficult to terminate unresponsive / long-running clients



Targeted client termination

Shut down one attached process
without affecting the others

MPS INDIVIDUAL CLIENT CONTROLLED TERMINATION

Multi-Process Service clients cannot be killed while active

Clients are co-scheduled while managed by service

Unexpected shutdown may leave service in an indeterminate state

Difficult to terminate unresponsive / long-running clients

New MPS control server command allows graceful termination

1. Use new “ps” command to identify process IDs to terminate

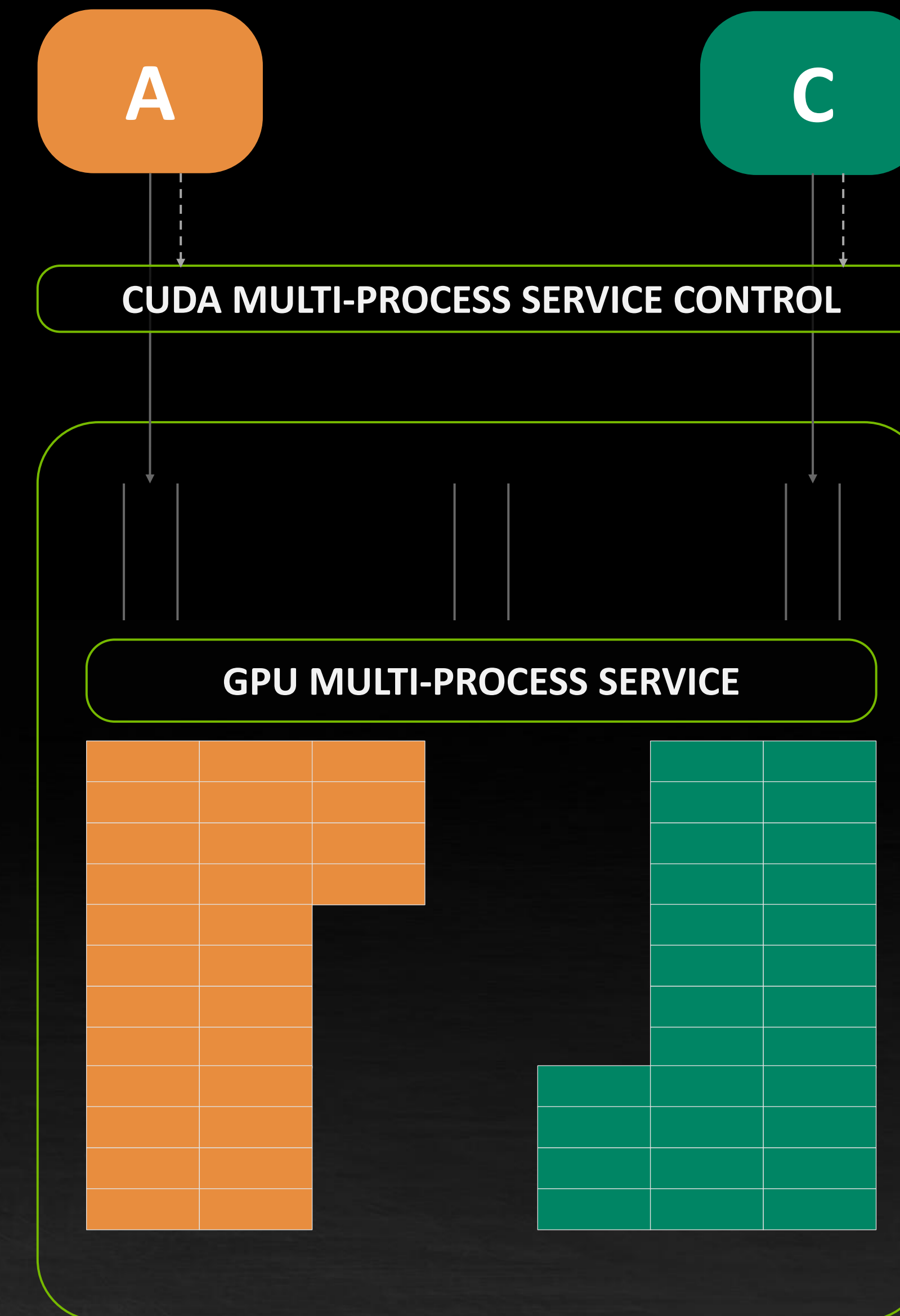
```
$ echo "ps" | nvidia-cuda-mps-control
```

2. Issue termination command to server

```
terminate_client <server PID> <client PID>
```

3. Now safe to kill / ctrl-c the client process

```
$ kill -9 <client PID>
```



Targeted client termination

Shut down one attached process without affecting the others

NVIDIA MAGNUM IO: GPUDIRECT STORAGE

- 1

```
pread (fd, cpubuf, 8192, 0);
```
- 2

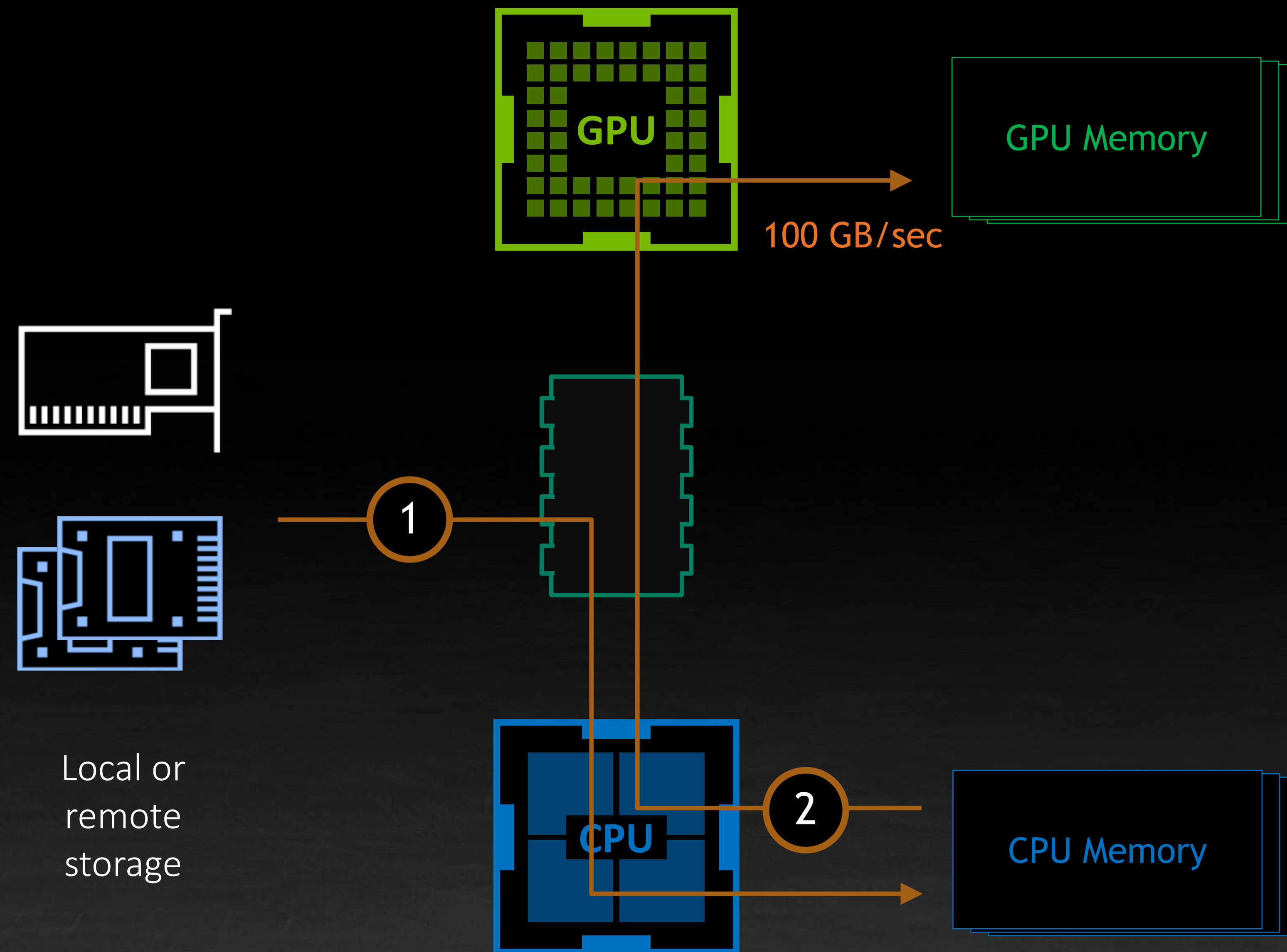
```
cudaMemcpy(gpubuf, cpubuf, 8192,  
           cudaMemcpyHostToDevice);
```

Traditional filesystem access

2 DMA operations

Requires bounce buffers

Halves CPU $\leftrightarrow$ PCI bandwidth



NVIDIA MAGNUM IO: GPUDIRECT STORAGE

1

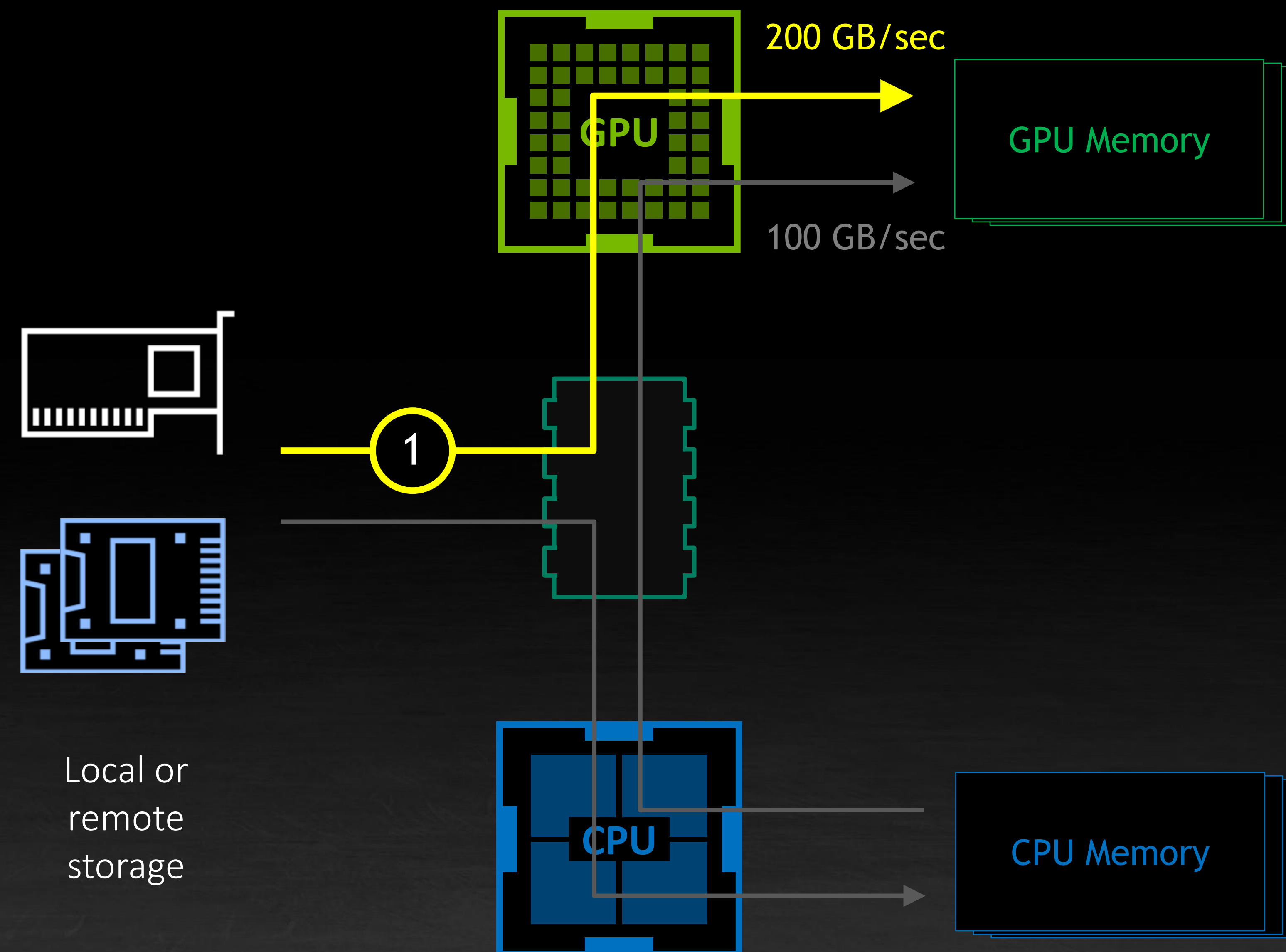
```
cuFileRead(fd, &gpubuf, 8192, 0, 0)
```

GPU-Direct Storage

Single DMA operation

No bounce buffer

Independent of CPU $\leftrightarrow$ PCI



NVIDIA MAGNUM IO: GPUDIRECT STORAGE

```
int main(void) {  
    ...  
  
    CUfileError_t status;  
    CUfileDescr_t cf_descr;  
    CUfileHandle_t cf_handle;  
  
    status = cuFileDriverOpen();  
  
    // Interop with normal file IO  
    fd = open(TESTFILE, O_WRONLY|O_DIRECT, 0644);  
    cf_descr.handle.fd = fd;  
    status = cuFileImportExternalFile(&cf_handle, &cf_descr);  
  
    cuda_result = cuMemAlloc(&devPtr, size);  
    status = cuFileBufRegister(devPtr, size);  
    cudaMemset((void *) devPtr, 0xab, size);  
  
    ret = cuFileWrite(cf_handle, devPtr, size, 0);  
  
    ...  
}
```

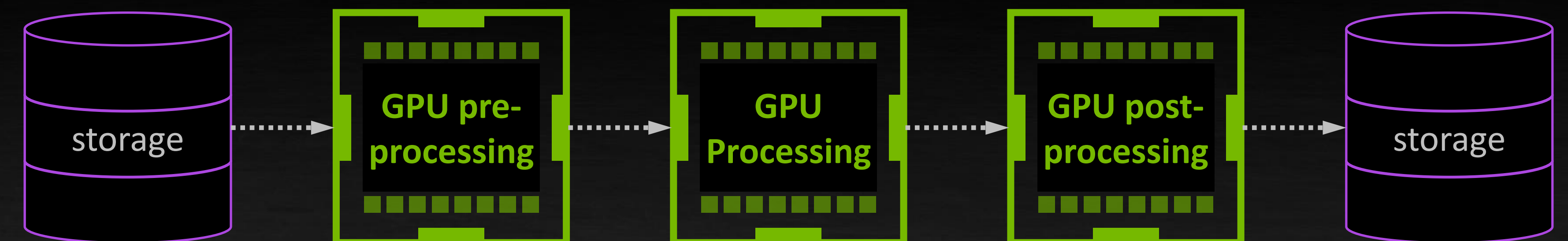
Adds file IO to CUDA without special HW

2x-8x higher peak bandwidth if PCIe switch present

Up to 3x lower CPU utilization

Supports unaligned & batched read/write

Keeps workflow & data on GPU



NVIDIA MAGNUM IO: GPUDIRECT STORAGE

HPC Visualization	8x
DeepCAM Inference	7.2x
Merlin	1.4x
Genotyping	3x-6x
Seismic Simulation	2.5x
RAPIDS/cuCIM	11x
HDF5	5x
KvikUI/Zarr	2.9x
Cosmoflow DL Training	1.2x

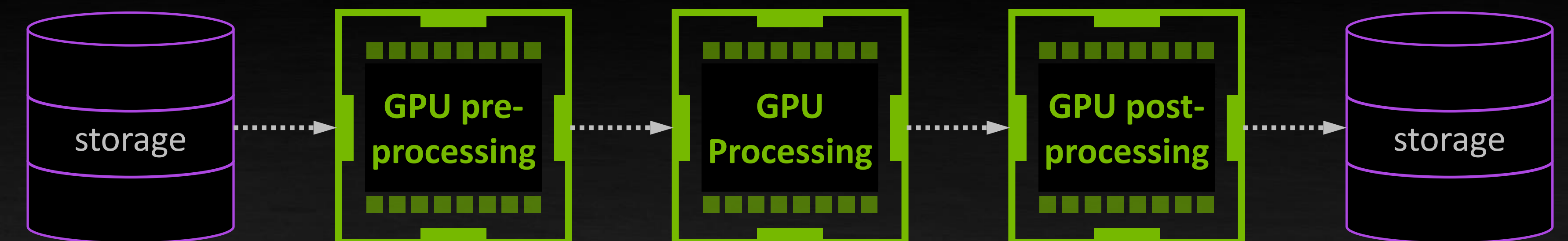
Adds file IO to CUDA without special HW

2x-8x higher peak bandwidth if PCIe switch present

Up to 3x lower CPU utilization

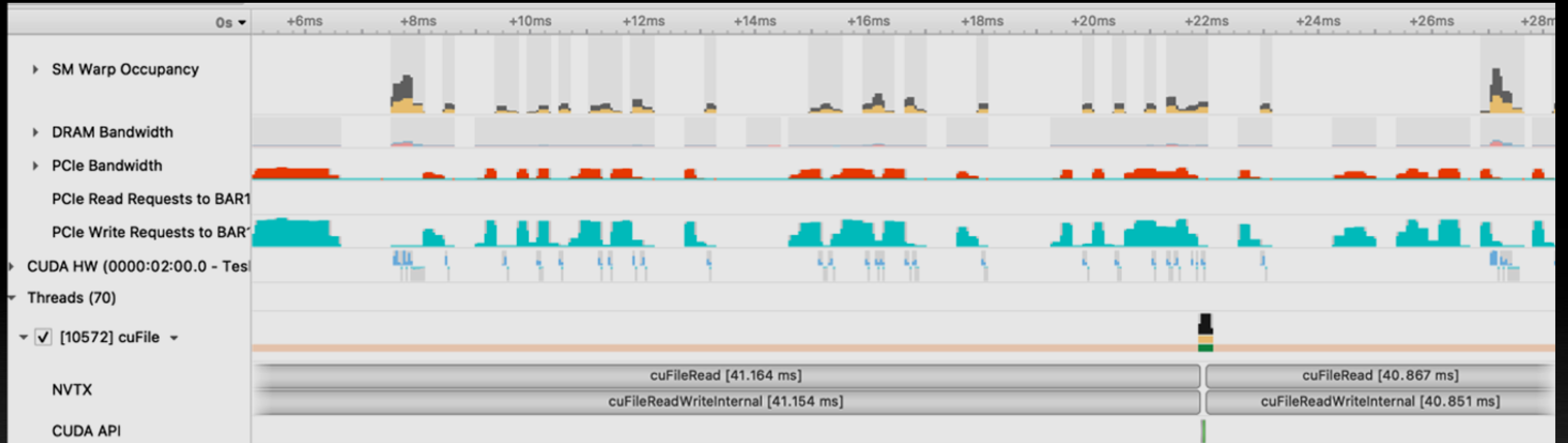
Supports unaligned & batched read/write

Keeps workflow & data on GPU

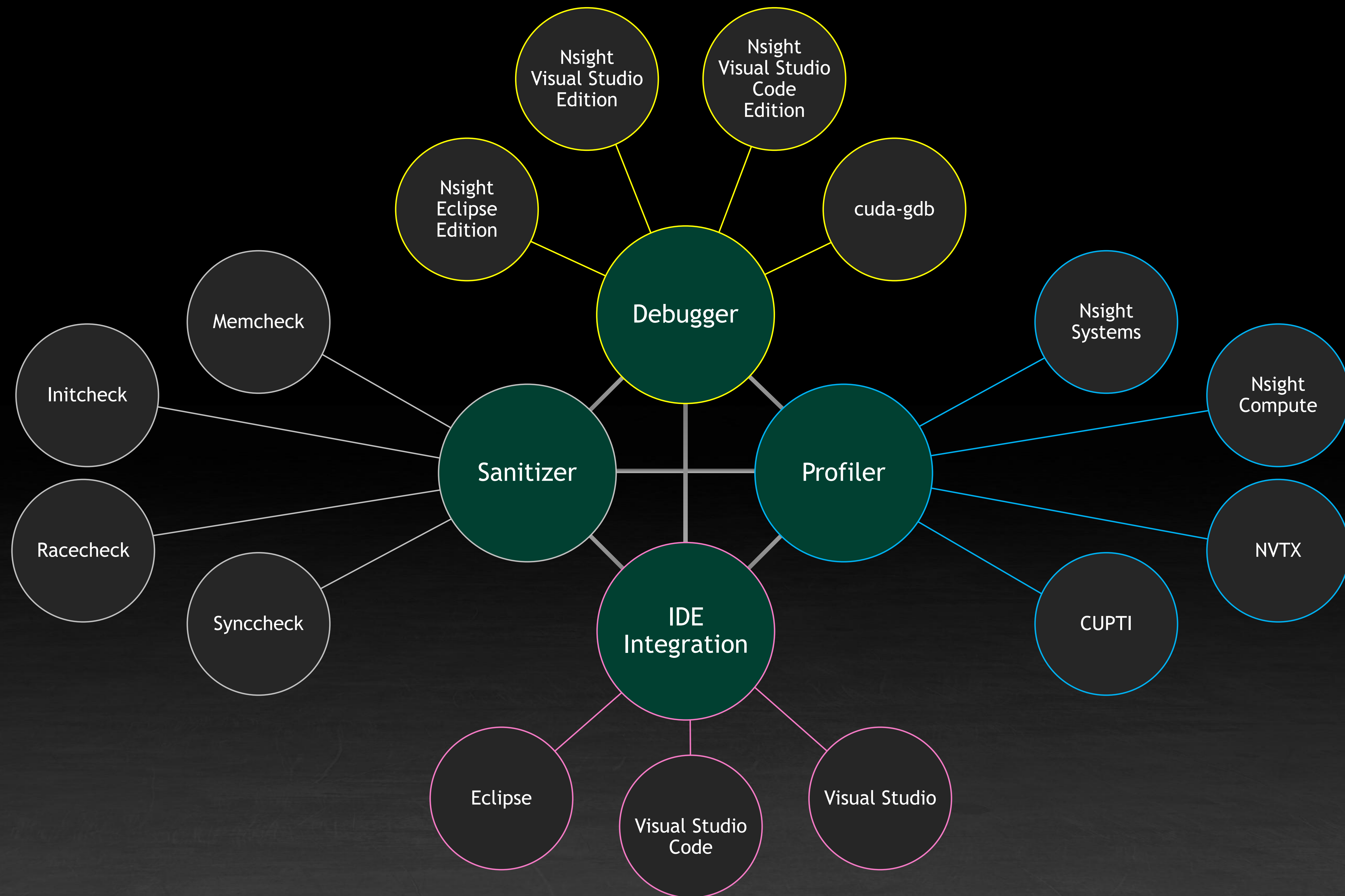


VISUALIZATION OF GPU-DIRECT STORAGE ACTIVITY

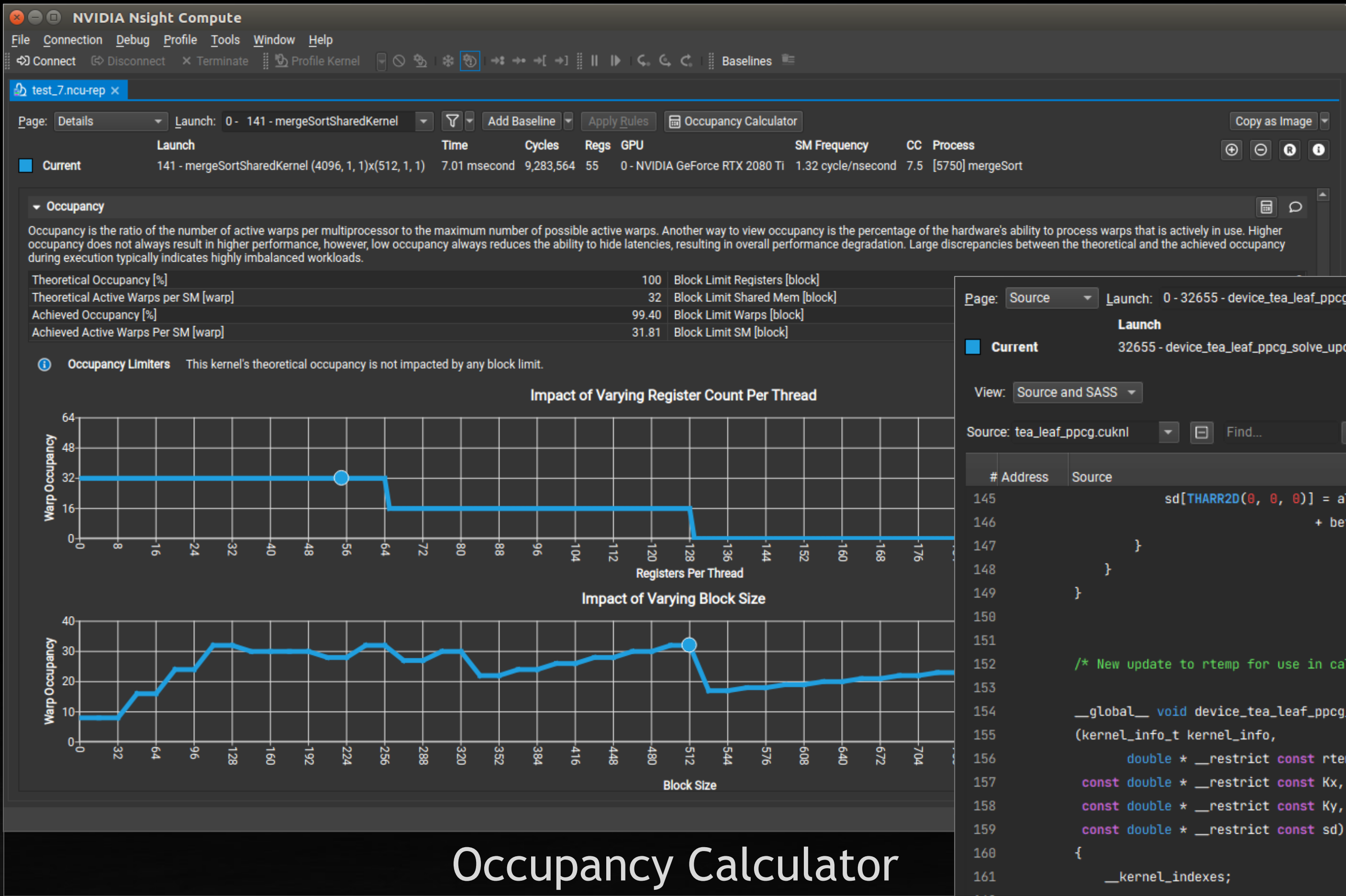
GPU metrics sampling of PCIe BAR1 requests & cuFile trace



FAMILY OF CUDA DEVELOPER TOOLS



NSIGHT COMPUTE: NEW CUDA PLATFORM DEVELOPER TOOLS



Occupancy Calculator

Page: Source Launch: 0 - 32655 - device_tea_leaf_ppcg_solve_update_r

Time Cycles Regs GPU SM Frequency CC Process

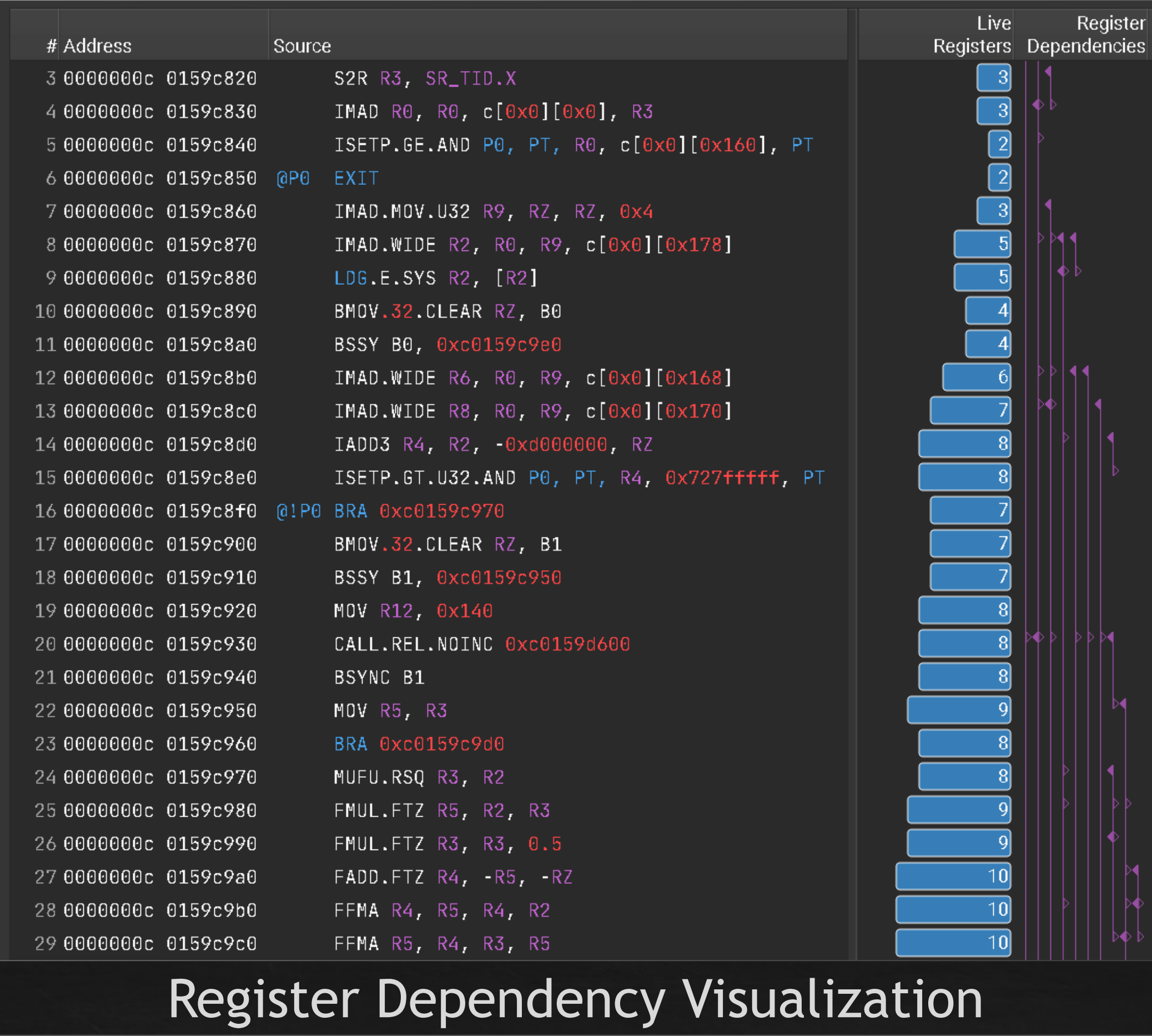
Current 32655 - device_tea_leaf_ppcg_solve_update_r (126, 1001, 1)x(32, 4, 1) 1.07 msecond 1,458,003 32 NVIDIA GeForce RTX 2080 Ti 1.36 cycle/nsecond 7.5 [10906] tea_leaf

View: Source and SASS

Source: tea_leaf_ppcg.cu.knl Find... Navigation: Instructions Executed

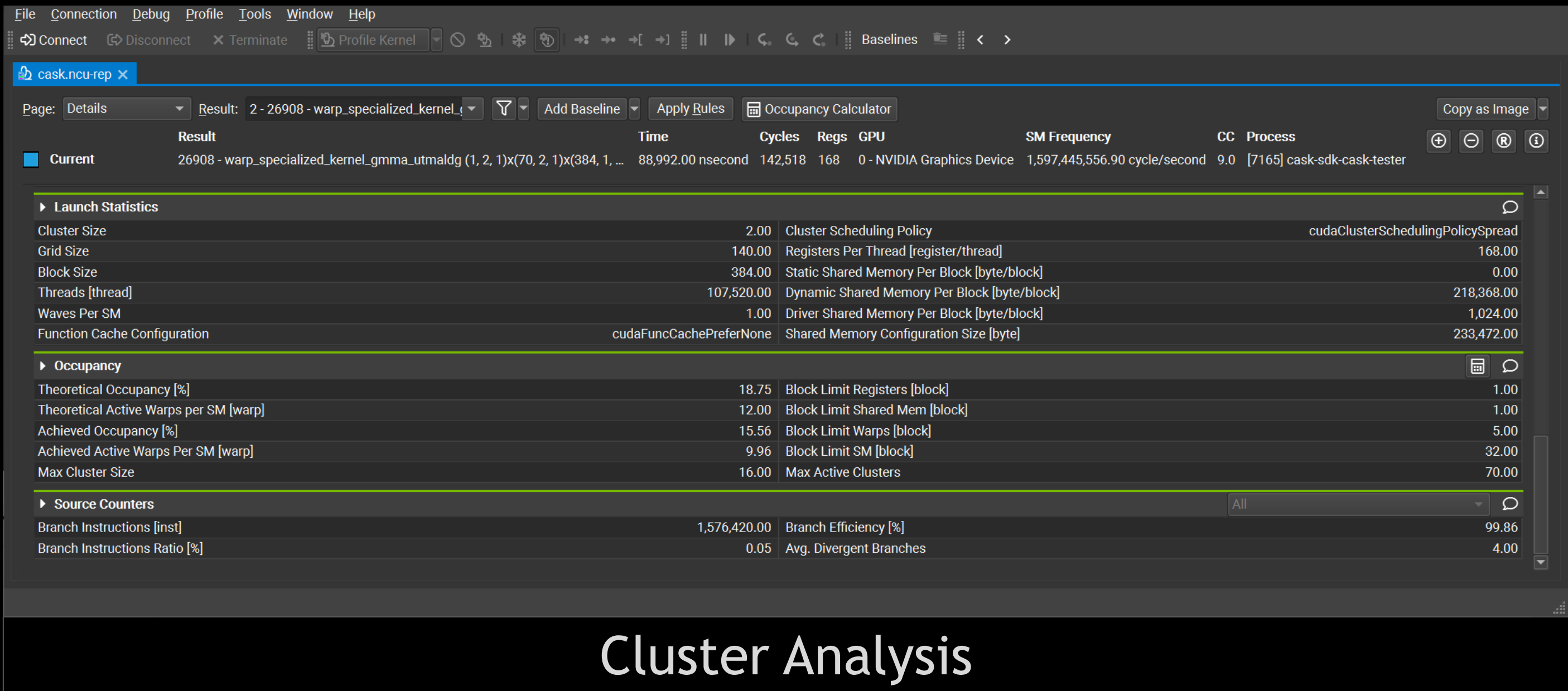
```
# Address Source
145 sd[THARR2D(0, 0, 0)] = alpha[step]*sd[THARR2D(0, 0, 0)]
146 + beta[step]*r[THARR2D(0, 0, 0)];
147 }
148 }
149 }
150
151 /* New update to rtemp for use in calc_sd */
152
153 __global__ void device_tea_leaf_ppcg_solve_update_r
154 (kernel_info_t kernel_info,
155  double * __restrict const rtemp,
156  const double * __restrict const Kx,
157  const double * __restrict const Ky,
158  const double * __restrict const sd)
159 {
160  __kernel_indexes;
161
162  if (WITHIN_BOUNDS)
163  {
164      const double result = (1.0
165          + (Ky[THARR2D(0, 1, 0)] + Ky[THARR2D(0, 0, 0)])
166          + (Kx[THARR2D(1, 0, 0)] + Kx[THARR2D(0, 0, 0)])) * sd[THARR2D(0, 0, 0)]
167          - (Ky[THARR2D(0, 1, 0)] * sd[THARR2D(0, 1, 0)] + Ky[THARR2D(0, 0, 0)] * sd[THARR2D(0, -1, 0)])
168          - (Kx[THARR2D(1, 0, 0)] * sd[THARR2D(1, 0, 0)] + Kx[THARR2D(0, 0, 0)] * sd[THARR2D(-1, 0, 0)]);
169
170
```

Standalone Source Viewer

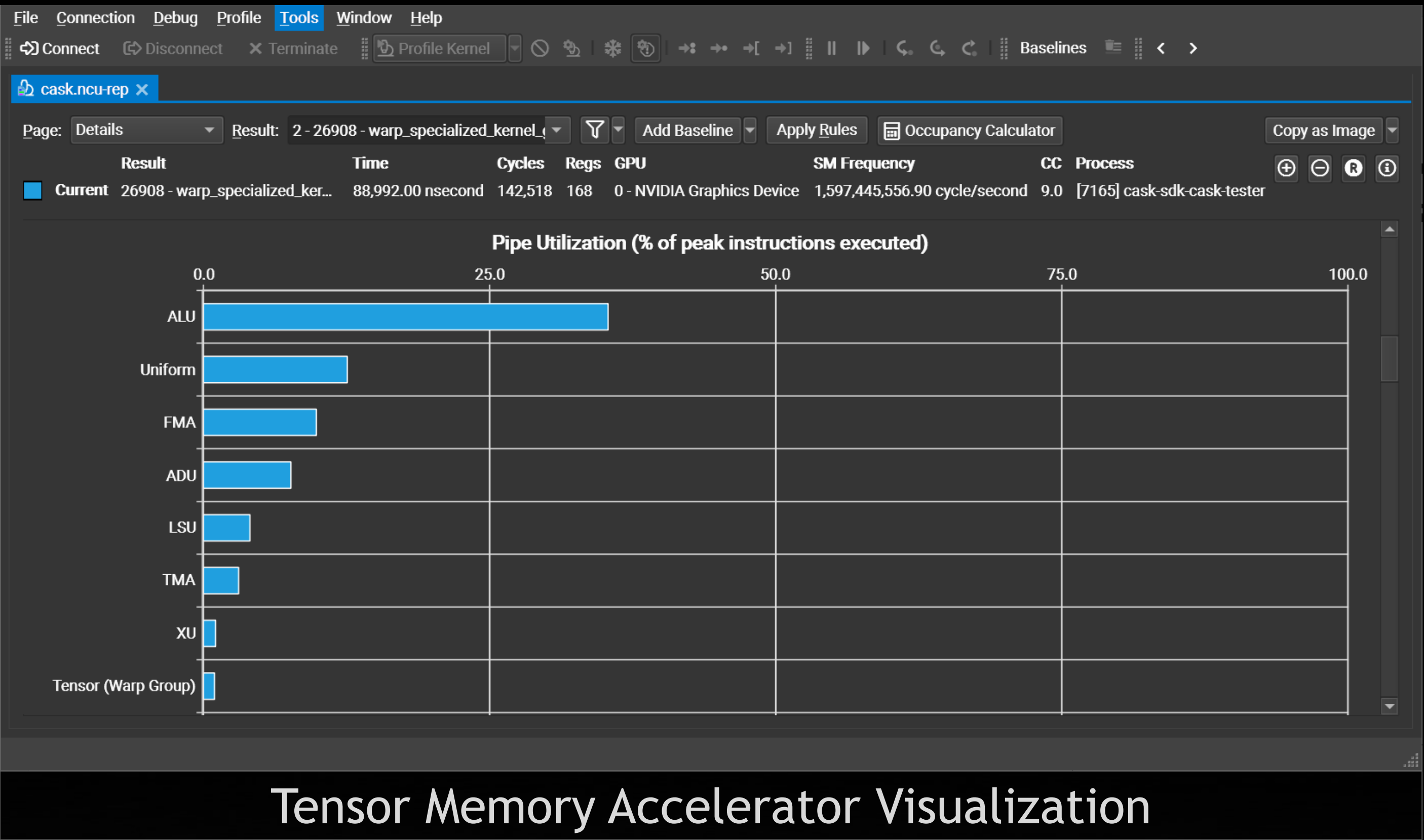


Register Dependency Visualization

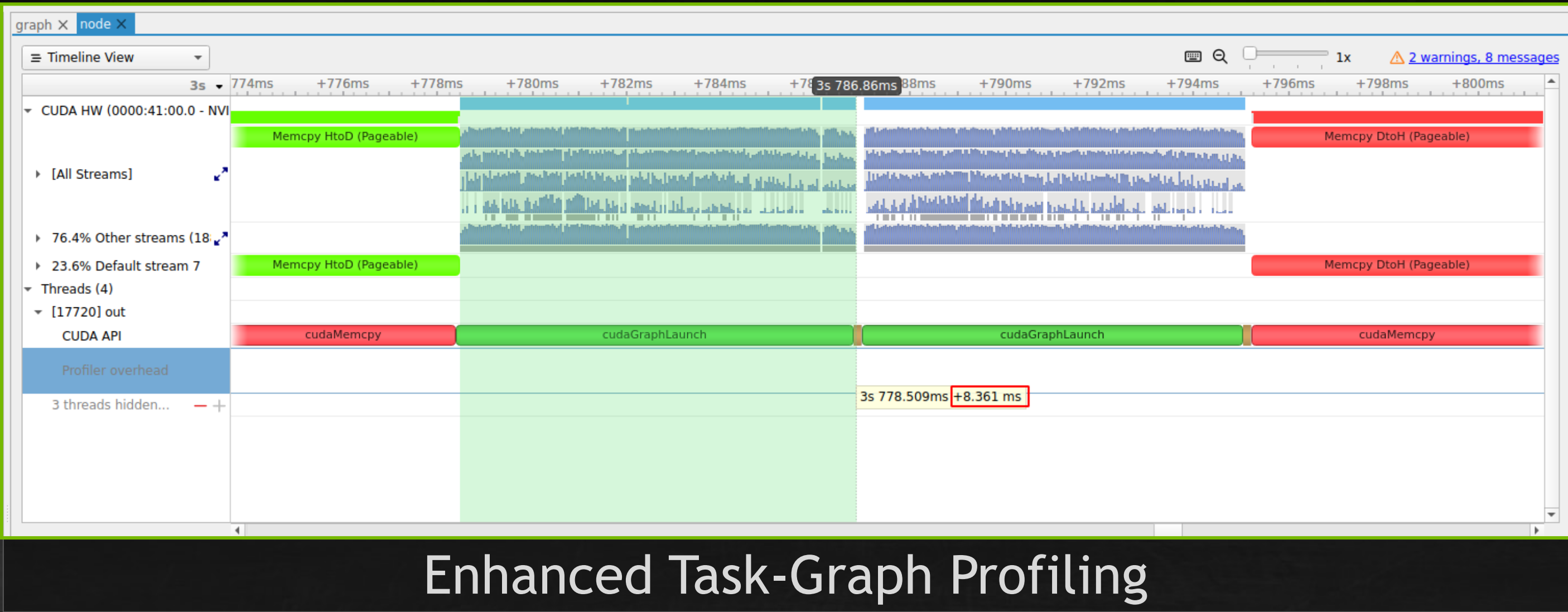
NSIGHT COMPUTE: TOOLS FOR HOPPER & CUDA 12.0



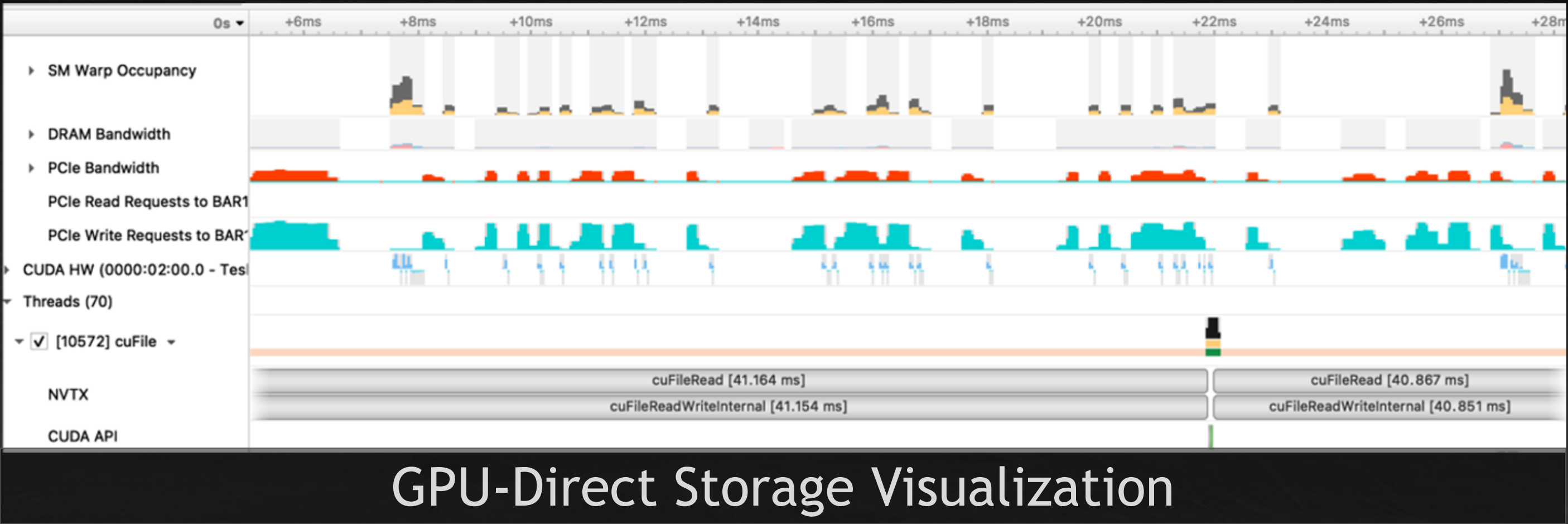
Cluster Analysis



Tensor Memory Accelerator Visualization



Enhanced Task-Graph Profiling



GPU-Direct Storage Visualization

REFERENCES

A41147 [Enabling Hopper-Specific Optimizations in CUDA Applications](#)

(March '22) S42663 [Inside the NVIDIA Hopper Architecture](#)

A41095 [CUDA Programming Model for Hopper Architecture](#)

A41090 [Tackling New Architectures and Workflows with the Latest CUDA Developer Tools](#)

A41133 [A Deep Dive into the Latest HPC Software](#)

Blog [Adaptive Parallel Computation with CUDA Dynamic Parallelism](#)

Docs [CUDA C Programming Guide: Cooperative Groups](#)

Docs [Magnum IO GPUDirect Storage](#)

